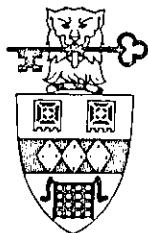


VECTOR

100+ PAGES OF THE BEST IN APL

- AGM report – your new committee.
- Standards before implementation.
- PC Graphics cards compared.
- FOCUS on APL.
- £50 prize competition.
- PC screen – ☐ WIN



*The Journal of the
British APL Association*

A Specialist Group of the British Computer Society

Vol.3 No.1 July 1986

Contributions

All contributions to VECTOR should be sent to the Editor at the address given on the inside back cover. Letters and articles are welcomed on any topic of interest to the APL community. These do not need to be limited to APL themes nor must they be supportive of the language. Articles should be submitted in duplicate and accompanied by as much visual material as possible, including a photograph of the author. Unless otherwise specified each item will be considered for publication as a personal statement by its author, who accepts legal responsibility that its publication is not restricted by copyright. Authors are requested wherever possible to supply copy in machine-readable form ideally text files on a 5¼" IBM-PC compatible diskette. For other standards, please contact the Editor beforehand. Program listings should indicate the computer system on which they have been run. APL symbols should be displayed on a separate line and not embedded in narrative. Except where indicated, items published in VECTOR may be freely reprinted with appropriate acknowledgement.

Membership Rates 1986-87

Category	Fee p.a.		VECTOR copies	Passes
	£	\$		
Nonvoting student membership	5		1	1
UK Private membership	10		1	1
Overseas private membership	18	27	1	1
Supplement for airmail (not needed for Europe)	8	12		
Corporate membership (UK)	85		10	5
Corporate membership (Overseas)	140	210		
Sustaining membership	360		neg	5

The membership year runs from 1st May to 30th April. Applications for membership should be made on the form at the end of the journal. Passes are required for entry to some Association events and for voting at Annual General Meetings. Applications for student membership will be accepted on a recommendation from a course supervisor. Overseas membership rates cover VECTOR surface postage and must be paid in £UK.

Corporate membership is offered to organisations where APL is in professional use. Corporate members receive multiple copies of VECTOR and are offered group attendance of Association meetings. Partaking individuals need not be identified but a contact person should be nominated for all communications.

Sustaining membership is offered to companies trading in APL products; this is seen as a method of promoting the growth of APL interest and activity. As well as receiving public acknowledgement for their sponsorship, sustaining members receive bulk copies of VECTOR, and are offered news listings in the editorial section of the journal and opportunities to inform APL users of their products via seminars and articles.

Advertising

Advertisements in VECTOR should be submitted in typeset camera-ready A5 portrait format with a 20 mm blank border. Illustrations should be black-and-white photographs or line drawings. Rates are £250 per page. A6 and A7 sizes are offered subject to layout constraints.

Deadlines for advertisement bookings and receipt of camera-ready copy are given beneath the Quick-Reference Diary.

Advertisements should be booked with and sent to Steve Lyus, whose address is given beneath the Index of Advertisers.

CONTENTS

	Page
EDITORIAL: The Joys of Spring?	3
APL NEWS	
Quick-reference APL diary	5
General Correspondence	7
British APL Association news	13
New Committee	15
News from sustaining members	17
British Computer Society news	21
International APL news	23
APL Press Review	27
APL Product Guide	29
APL Book List	37
Book Reviews	39
Software Review (AFM)	41
PC Benchmarks updated	44
Product Review (PortaAPL)	45
PC Graphics Cards (EGA and PGC)	51
RECENT MEETINGS	57
Introductory Notes	59
APL and OR (17th Jan 86)	60
Computing Environments for OR	62
Managing APLers (21st March 86)	65
GENERAL ARTICLES	67
Case Study: VSPC - The Last Rites	68
One Small Step for APL . . .	75
A Sharper Focus on APL	79
Steps to a Better BASIC	93
TECHNICAL SECTION	95
Technical Editorial	97
Technical Correspondence	99
Surely There Must Be a Better Way	101
Prize Competition:	
Get your Directories Updated	105
Introduction to Contributed Articles	107
Quick and Dirty Apportionment	109
Fast Fibbing	113
A Plea for Naming Conventions	115
Quad-WIN in APL*PLUS Release-5	119
APL Trivia	124
CONTENTS OF VOLUMES 1 AND 2	128
Index to Advertisers	131
Membership Application Form	132

NEW MAINFRAME SOFTWARE FOR IBM'S® APL

Now available in the UK, two new offerings from STSC that enhance IBM's mainframe APL implementations

If you're staying with VS APL ...

COMPILER

The first commercial compiler for APL compiles functions individually. Results in significantly faster execution. Interpreted functions can call compiled functions and vice versa.

If you're migrating to APL2 ...

SHAREFILE/AP

STSC's popular APL component file system is now available under APL2. Multi-user, nested array storage, libraries, access matrices. Multiple file system support. International language translations.

For full information, contact the APL*PLUS™ Product Group, Cocking & Drury on 01-493 6172.

Trademarks/Owners: IBM/International Business Machines Corporation · APL*PLUS/STSC, Inc.



COCKING & DRURY LTD.
THE APL PROFESSIONALS

16 BERKELEY STREET · LONDON · W1X 5AE
Tel: (01)493 6172 · Tlx: 23152 MONREF G

Editorial: The joys of Spring?

by David Preedy

So Spring is here at last; the birds are in full song; the trees are budding; the sap is rising, and every young buck's thoughts are directed to APL '86 in Manchester.

That's all very well for them, but for those of us whose BAA committee regulation beards are starting to grey, there are other more weighty matters that must take precedence. It's recruitment season, and we have to sit cloistered in darkened rooms, grilling the young aspirants whose very careers may depend on what we think of them.

For those of us who have deserted the big company scene to savour the delights of small business, the process is fairly nail-biting for the interviewers as well. After all the costs of a mistake are proportionately greater for a small team. Anyway, Curricula Vitae arrive by the sack-full and one's thoughts are divided between the desire to recruit some really able people and the fear of what geniuses these people must be, who have achieved such startling exam results.

However during the process of vetting all these applicants, I was struck by the singular inappropriateness of what they were being taught – particularly those on computer-related courses. Are they being given the breadth of vision to devise inventive ways to use the rapidly-changing technology of today, let alone tomorrow? Far from it! In most cases, they are being fed regurgitated techniques that were probably state-of-the-art when their tutors were being educated, souped up with a couple of lectures on the latest fad – normally expert systems.

Now I'm not pretending that it would be possible to teach only subjects that are likely to be widely applicable in the outside world – there's nothing to match commercial experience for that. However I would expect some of the students to emerge with an overview of the computing business. They should surely be exposed to the types of problem addressed by computer techniques and to the different sort of technique available. And that general background should at least include some study of the different types of programming language available. I don't expect, or even want, a generation of graduates fluent in Lisp, Prolog, Cobol and APL. However I would like some of them to have heard of APL – or at least to be able to find out something about it in the library when they get back to college.

Of course my own experience and prejudices predicate that I will be looking for an appreciation of APL, but the issue is more general. If students never get the chance to even hear about one of the languages that has led the way in many application areas – graphics, decision support, information centres, financial planning, to name a few – then how can they hope to develop new and imaginative applications themselves over the next thirty years or so?

So this issue's editorial ends with a three-fold challenge:

- ★ to the APL community, to approach your local college and offer to give a brief series of talks to interested students, explaining why you use APL; better still lend them some kit and an interpreter;
- ★ to the academic community, to ensure that all computing students at least can find out about APL and other minority programming languages – why not for instance subscribe to VECTOR and buy Gilman & Rose?
- ★ to the students themselves, to choose courses that offer them a broad perspective of the field of computing and whose practical offerings involve more than writing an operating system (in BASIC) and programming someone else's hidden-line removal algorithm.

Reminder

1986/87 subscriptions are now due. If you have not already paid your membership for the year, please return the enclosed renewal form with your cheque. Otherwise you will not receive any more copies of VECTOR.

Better still, you can take advantage of our special free membership offer (see VECTOR Vol 2, No 3 page 9). Send in your own renewal form with application forms and payment from three people who have not previously been Association members, and you get a free year's membership if you are a U.K. private member, or a £10 reduction on your subscription otherwise.

Quick-reference diary
compiled by David Preedy

Date	Venue	Event
1986		
1-3 July	Wembley	Knowledge-Based Systems '86 Organised by Online Conferences
7-11 July	Manchester	APL 86 - "APL in action"
10-12 September	Southampton	Software Engineering '86 Organised by BISL
19 September	London	British APL Association meeting
22-26 September	York	HCI '86 - People and computers: Designing for Usability Organised by BISL
17 October	London	British APL Association meeting
21 November	London	British APL Association meeting

Dates for future issues of VECTOR

	Vol.3 No.2	Vol.3 No.3	Vol.3 No.4
Copy date	25 Jul 86	17 Oct 86	30 Jan 87
Ad. booking	22 Aug 86	14 Nov 86	22 Feb 87
Ad. copy	29 Aug 86	21 Nov 86	27 Feb 87
Distribution	October	January	April



H.M.W. PROGRAMMING CONSULTANTS LTD

Why not discover more about

- ★ **Consultancy/Support Service**
- ★ **APL on the IBM PC**
- ★ **VM/CMS Packages**

Ring Ken Jackson at

H.M.W. PROGRAMMING CONSULTANTS LTD

142 FELTHAM HILL ROAD,
ASHFORD, MIDDLESEX TW15 1HN
Telephone: Ashford 41232

General Correspondence

The VECTOR working group welcomes correspondence on any topic affecting the APL community. All such letters should be addressed to the Editor and should indicate whether they are intended for the general or the technical section. Letters containing APL code will normally appear in the Technical Section of VECTOR, and authors are asked to observe the requirements on the inclusion of APL code stated on the inside cover. The Editor reserves the right to edit any letter unless the writer states that it is to be published in full or not at all.

APL – USP or RIP

From L J Devaney

15th April 1986

Sir,

In my opinion the APL milieu has a most unsatisfactory scabrous patter (MUSP). APL's inability to penetrate the mass market place is surely conditioned by:

- a) the complete lack of management control;
- b) the unintelligible texts that APL functions contain, whether keyword or symbolic;
- c) lack of portability;
- d) the inability to implement closed, secure systems.

Management control is important because of the need to conserve a firm's investment in software and to establish a firm control over program libraries. Change controls are necessary procedures in business and the cycle of "design, implement, test, change, re-test" is an important one. APL is unable to support properly established libraries, segregated responsibilities and incremental implementation in the way that COBOL, FORTRAN and other languages can.

The readability of code has in the past been promoted at the expense of function, but APL promotes function at the expense of readability. Mr Last's examples on page 98 of VECTOR, Vol.2, No.3, would not be improved by using keywords! Part of the problem is the use of one line of text to do so much work. Although COBOL, FORTRAN, etc. use more statements and would need intermediate variables to hold partial results, overall their texts would be more intelligible because of the stepwise presentation. Of course, APL could be so written, but the power of the language bodes ill for such considerations to be of more than a fleeting interest to APL "professionals".

Portability is important to some user-communities. When I worked abroad some years ago, we received a large FORTRAN program from a Burrough's installation. Although we were operating an IBM installation, all that was needed was to follow the few rules documented as comments at the beginning of the program and then compile link-edit, etc. in the usual way. No problems ensued. The text was received on magnetic tape. The only portability for APL that I know of is typing it all in again!

BASIC and APL have a common problem – the user can meddle with the programs if they know how to. Today it is not possible to have one's system developed in APL and secured against intrusion. This extends also to operator authorization/permissions and operational safety. "Runtime only" versions may help, but a more thorough solution is required for the long term.

Yours,

L J Devaney,
24 Canada Road,
London, W3 0NP.

(Editor: Thanks for your comments – no doubt they will stimulate a response from our readers!)

APL for newcomers

From Andre Faure

30th December 1985

Sir,

May I compliment you on your publication. I feel that VECTOR as now produced is an excellent journal. Overseas members do not probably have much use for the club news which takes up quite a bit of the publication space, but, after all, the journal must also serve the British Association. However, given the high calibre of writers involved in its production, it could perhaps reach a wider readership if it also tried to educate users and newcomers to APL. No publication to my knowledge does that at the present time.

There are two types of person interested in APL: those making a living with it and those looking for a tool to solve their computing problems. The former may not need much help with APL and usually have access to other experts. The latter need first of all convincing that APL is "it", and then help in getting started. Moreover they are usually isolated, especially if they intend to use APL in a personal computer environment. This is where VECTOR could do a lot of good.

It must be recognized that APL presents a higher starting threshold than most languages. For instance, simply listing a function on a printer with the STSC APL*PLUS/PC requires a function to be written with ☐POKEs and ☐ARBINs. I doubt whether any newcomer to APL can handle that with only the aid of the manuals. It took me two days and quite a few phone calls. In BASIC, you type LIST and are content with that – any kid can do it!

You could (and should) fill two or three years of VECTOR with tutorials for newcomers (and dare I say old-timers as well?) on how to manipulate inputs and outputs in APL. How do you build an entry screen which will support a normal keyboard and a "fill the blanks" data entry principle? How do you write reports? How do you take care of headers, footers, page numbers, etc.? This is not easy, or the text-books would not so blissfully ignore the subject.

My experience is that computing with APL does not present many problems. In fact, it is fun! Adding inputs and outputs to build a system that anyone can use on any machine is another story, and that story needs telling.

A theme which keeps recurring within APL concerns those curious little characters used in APL in addition to the normal alphabet. Your editorial policy appears to be that these symbols deter potential users and should be replaced with keywords. I disagree. Using the APL character set is fun and fast. What is not so funny is returning to a function a few days later and trying to figure out what that line of symbols actually performs. What I miss is a translator, which goes beyond mechanically replacing each symbol with a keyword: the translator should look at the context in which each symbol is being applied (the APL interpreter does that, to my understanding), and replace each symbol, or set of symbols, with a suitable plain language equivalent. The plain language equivalent need not be executable code, only documentation.

Since the translator would first parse a line of code into a set of unique operations (as does the interpreter) and then replace these operations with plain language equivalents (a table look-up operation), it becomes immediately obvious that the translation could be carried out in any language. Thus APL could offer the best of both worlds: a universally understandable and transportable set of symbols, that can also be translated into the user's own language. The compactness of the language actually makes translating easier, since phrases can then be used instead of single words.

Yours truly,

Andre H Faure,
Rasti 2 B 17,
40340 Jyvaskyla,
Finland.

(Editor: Thank you for your comments on VECTOR to which I would like to add my own thanks to all those who have contributed to it. We do try to remember the needs of APL beginners, but a regular feature sounds like a good idea.

The VECTOR editorial team as such does not take a policy position on such issues as symbols versus keywords; individual papers – and for that matter editorials – merely reflect the views of the author concerned. Certainly the independence of the symbols from a user's national language is a new and interesting aspect of the discussion.

On a personal note, I am extremely attracted to any idea that would effectively reduce the need for me to document my code! I would however disagree with the comment that the starting threshold for APL is higher than other languages; when my son was four I sat him in front of a micro for the first time and without prompting he correctly entered a valid line of APL. Unfortunately, he didn't know about the Return key!

APL research studentship

From David Livingstone

11th April 1986

Sir,

I wonder if any of your readers know of a student who may wish to do research in APL, leading to an M.Phil/Ph.D?

Such a project will be starting in the Autumn of this year at Newcastle-upon-Tyne Polytechnic's School of Computing and Informatics. It will have External Supervisors from IBM and Dyadic Systems Ltd.

The aim of the research is to develop the concept of domains in APL. Currently there are just 2 domains or types of value that may appear in a variable – numbers (the set of all numbers that can be held on the computer) and characters (the set of all characters stored in the Atomic Vector).

It would be useful if the programmer could tailor domains to meet the particular needs of an application rather than be limited to these 2 primitive sets of values. By specifying a variable's domain precisely, data can be validated automatically when it is assigned to the variable. The same approach can be used in debugging, to check a variable contains only permitted values at a particular point in a function.

The specific objective is to add to APL the optional facility to associate a user-specified domain of values with a variable. Whenever a value is assigned to that variable, a check will be made to see whether it lies in the variable's domain of permissible values. If it does not, a DOMAIN ERROR or appropriate interrupt will be generated.

As a domain is a mathematical set, it can be specified either by enumeration or by set conditions. Ordinary APL variables or functions respectively will be used to implement the 2 kinds of set definitions. Thus the full power of APL will be available to define domains.

Please phone or write to me if you know of anyone who may be interested in doing this project; or better still, get the prospective student to.

Yours sincerely,

David Livingstone,
School of Computing and Informatics,
Newcastle-upon-Tyne Polytechnic,
Northumberland Building,
Newcastle-upon-Tyne,
NE1 8ST.
Tel (0632) 326002 ext 3347

Vacation address:
210, Osborne Road,
Jesmond,
Newcastle-upon-Tyne,
NE2 3LD.
Tel (091) 281 3778

From A.G. Bright

1st May 1986

Sir,

I read with interest the recent article in VECTOR Vol. 2, No. 3, by Katie Williamson and Geoff Wells concerning external databases and APL applications. It was pleasing to see experienced programmers integrating I.P. Sharp databases with APL systems.

As a postscript to the article it was stated that there were inaccuracies in the share price data provided by ourselves and that the service was being withdrawn. Having had extensive contact with the authors of the article I can appreciate the accuracy of the statement from their perspective. It is however slightly misleading to the outsider and I would like to clarify the situation.

Through extensive use of our FTSTOCK database (London share price information is provided – in a raw form – to I.P. Sharp by the Financial Times), Katie and Geoff did indeed find several inaccuracies. Upon close examination of the database we realised that due to the extent of these inaccuracies we could no longer market FTSTOCK in good faith and therefore withdrew this database at the end of February 1986.

However, a suitable replacement is available on-line from us in the form of EXSHARE – Extel Computing's International Stocks Database. This contains coverage not only of the London market (including the USM, Eurobonds and options) but also all major non-American markets. The database is updated daily and data goes back to January 1983. EXSHARE complements our North American databases giving I P Sharp a complete coverage of the world's major stock markets. This is in addition to worldwide coverage of commodities, traded options, and over 100 other time-series databases . . . but then this isn't supposed to be a commercial!

So, it is true to say that errors were found in our London share price database. We therefore changed our source and in doing so we now supply a more comprehensive coverage. I trust this has clarified the situation.

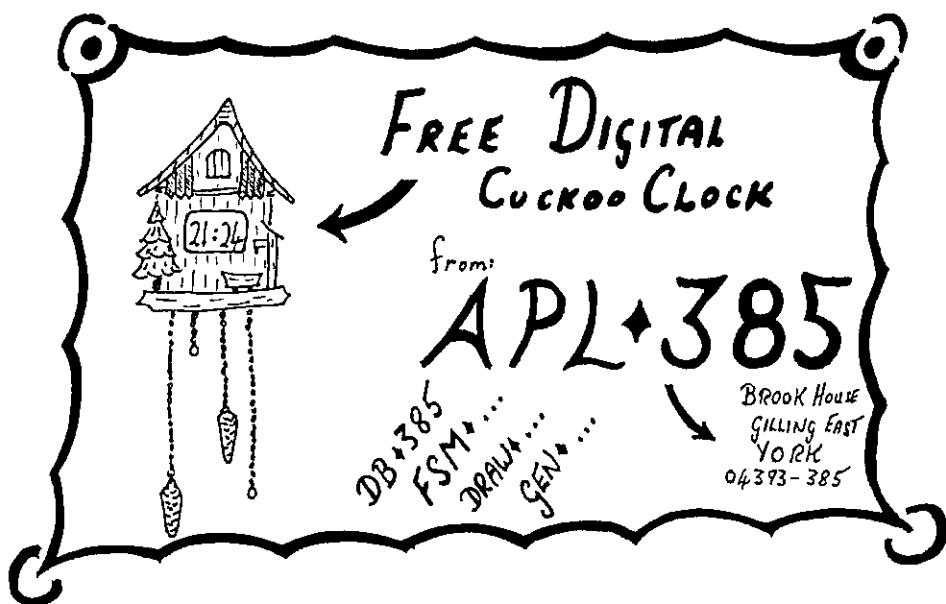
Yours faithfully,

A. G. Bright,
Database Consultant,
I. P. Sharp Associates Ltd.,
10 Dean Farrar Street,
London SW1H 0DX.

(Ed: Thanks for clarifying the issue. Geoff and Katie encountered these problems late after submitting their article for publication, so there was only time available to append a brief postscript. I'm sure that many readers will find your letter useful.

Two thoughts arise as a result of this. Firstly that the responsibility of ensuring data validity is an onerous one, whether the data is provided externally, as in this case, or is only made available within one's own organisation. Clearly I.P. Sharp's prompt action in withdrawing the particular service involved reflects their own recognition of this responsibility.

Secondly, that the issues involved in this particular case, as I understand them, were not the simple ones of whether a particular set of numbers were right or wrong, but concerned the exact mechanisms used to adjust historic data, e.g. to compensate for subsequent share issues. I wonder how many other data-sources there may be whose exact specifications are either unclearly defined or rarely validated and how many Decision Support Systems provide such data as an input for decision-making.)



British APL Association News

compiled by David Preedy

Treasurer's report

The maintenance and administration of the Association's membership database has now become the total responsibility of the treasurer. It is hoped that by having direct control over the system, rather than using a 'middle man', we will be able to provide a more efficient and responsive service to the membership.

At the same time it is intended to control VECTOR distribution more closely so that members will not receive copies of the journal until they are fully paid-up for the current year. Please ensure that you do not suffer this fate by forwarding the 1986-87 subscription without delay. Otherwise this may be the last VECTOR you see!

It is not intended to issue membership cards for the current year - nor for that matter for last year! Clearance of your subscription cheque should serve as acknowledgement of receipt of payment. Any members who require a formal receipt should enclose a stamped addressed envelope with their cheque.

As a consequence of the Data Protection Act, any member may request that his name and address be removed from the Association's database. Any member wishing to do so should contact the Treasurer in writing.

(Ed: At the A.G.M. the treasurer gave an interim statement of the Association's report and accounts for 1985/86 - the final figures are awaiting exact costings and receipts for VECTOR volume 2, number 4. The final report and accounts for 1985/86 will be published in the next issue of VECTOR.)

HMW Programming Consultants Ltd.

The Association are pleased to welcome HMW Programming Consultants as the latest company to join the growing ranks of our sustaining members. Unfortunately they have joined too late to submit an entry to our regular *News from Sustaining Members* column in this issue, but I am sure they will have lots of news for us in the next issue.

In case you want to contact them before their address is emblazoned on the back cover of the next VECTOR, their address is:

Ken Jackson,
HMW Programming Consultants Ltd.,
142 Feltham Hill Road,
Ashford, Middx.,
TW15 1HN.
Tel. 07842-41232



9 members of the new committee at the Royal Overseas League:

*Back row:
David
Ziemann
(Technical)*

*Roy
Tallis
(Activities)*

*Christine
McCree
(Recruitment)*

*David
Preedy
(Journal)*

*Gerard
Paul-Clark
(Publicity)*

*Front row:
David
Eastwood
(Projects)*

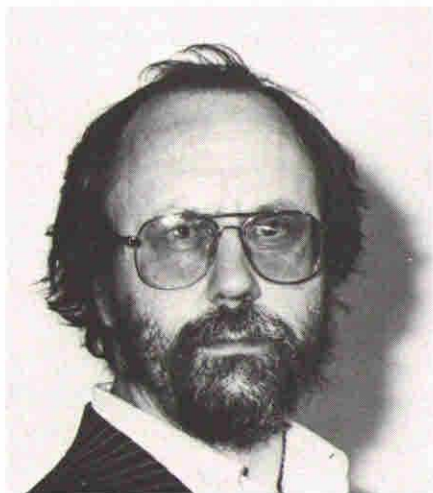
*Mel
Chapman
(Treasurer)*

*Dick
Bowman
(Chairman)*

*Anthony
Camacho
(Secretary)*



*Ken Iverson talking to the BAA on
"The split in APL"*



Dick Bowman - re-elected chairman

Annual General Meeting - Initial report

23rd May 1986, Royal Overseas League, London

At the AGM, the outgoing committee proposed that two new committee officers be introduced for the coming year. The two new posts are Recruitment Officer and Projects Officer. The Recruitment Officer will be responsible for initiating an active campaign to encourage wider Association membership. The Projects Officer will create a series of new projects to increase awareness and use of APL and enhance the reputation of the Association, building on the success of VECTOR and the anticipated success of APL86.

In the subsequent committee elections the following members were elected:

Chairman	- Dick Bowman
Secretary	- Anthony Camacho
Treasurer	- Mel Chapman
Journal Officer	- David Preedy
Technical Officer	- David Ziemann
Activities Officer	- Roy Tallis
Publicity Officer	- Gerard Paul-Clark
Education Officer	- Norman Thomson
Recruitment Officer	- Christine McCree
Projects Officer	- David Eastwood

Apart from Projects Officer, all remaining nominees were unopposed. In a show of hands for the post of Projects Officer, the votes were:

David Eastwood	16
Philip Goacher	12

and David Eastwood was declared elected.

A full report of the AGM, including the Chairman's and Treasurer's reports will appear in the next issue of VECTOR.

Application Prototype Environment

*an IBM program product for prototyping
and developing screen-based applications under VS APL or APL2*

This product is intended to be used both by the professional programmer and by business professionals, such as managers, planners, scientists, and engineers. The product is particularly suited for developing applications in Information Centre and Application Development Centre environments.

Three interactive functions are provided to easily and quickly define the end user interface.

Panel Design: For creating menu selection, action, and tutorial panels.

Chart Design: Charts (Plot, Surface, Bar, and Pie) are designed via an interactive dialogue.

File Handling: The interactive dialogue allows users to specify APL File, CMS, VSAM, QSAM, BDAM, and INSTOR-AGE access methods.

Over 70 cover functions are provided for building the application program. Some examples are:

Panel Operations

- Automatic Screen I/O.
- Automatic display of tutorials.
- Program function key control.

File Handling

- Reading/writing records from/to a file.
- Positioning the record pointer.
- Relative record access.

Object Library

The object library is used for storing, retrieving, and status checking of objects such as panels, functions, declarations, and variables.

Free 30-day Test Period

You have 30 days to test Application Prototype Environment to see how well it performs for you. If you are not completely satisfied, just send all materials back to your local IBM Branch Office within 30 days and you will not be billed.

For More Information: Contact your IBM sales representative or the nearest IBM Branch Office.

Technical Data

Application Prototype Environment is an IBM Licensed Program Product, Program Number 5668-896.

The program runs under CMS and TSO together with the following IBM programs or their equivalents: APL2 or VS APL, CDDM (Graphical Data Display Manager). Some examples of terminals supported are: 3277, 3279, 3270 PC/C and GX.



News from sustaining members

compiled by David Preedy

APL People Ltd.

APL People have got off to a good start in 1986 with orders coming in for PEFAC – the computerised estimating system acquired with the takeover of H. Walton Technical Services at the end of 1985. A lot of developmental work has been done since to make PEFAC even more versatile – such as writing microsoftware for PEFAC to run on networked computer systems, and creating the means whereby the engineer can write his own functions.

At the time of writing, APL Tran-Plan is beginning to take off with work already under way on the planning and design of several transportation projects in North America, and a lot more exciting projects set to come up in the near future.

In the guise of APL Software Ltd., APL People have now obtained the agency for a number of software products being offered to companies throughout Europe.

The consultancy side generates a steady flow of work, not only from the London area, but throughout the UK. The conversion market in particular is still buoyant, both into APL2 and from VSPC, with customers recognizing that an upgrade to a uniformly consistent and fully documented standard when an upgrade is being carried out can ultimately save them a lot of money.

On top of this activity, APL People somehow manage to find time to offer their placement and advisory service for APLers considering a job move, or simply seeking some unbiased career advice. At the time of writing they have something like a score of vacancies and a similar number of people looking – but of course they don't all match!

APL Software Technology (UK) Ltd.

1986 has got off to a good start for APL Software Technology with the first UK mainframe installation of their APL-based product RDS (Relational DataBase System). Although there are installations in Scandinavia and West Germany this implementation, for a large financial organisation, is seen as the start of yet another major product line here in the UK.

The business growth continues for them and the search is now on for a new London office from which they can base their attack on the international insurance and banking industry which forms the major part of their business.

POWERTOOLS/PC continues to be enhanced with further developments planned. The facilities now offered by this product make the design and running of PC-based applications easy, efficient and effective. Requests for Production Run Time versions continue as more and more applications are developed around the product.

As a result of their extensive experience in the 'bits and bytes' level of micro implementations APL Software Technology have been actively involved in the porting of APL onto portable micros and the design and implementation of 80286 and 80386 based micros for a major Scandinavian company. They hope to be able to mention names and make available full details in time for APL 86.

Cocking & Drury Ltd.

Cocking & Drury are now installing release 5 of STSC's Sharefile and Enhancements to VSAPL. This new release includes a library of compiled APL utility functions, many times faster than their interpretive equivalents. It also includes the feature necessary to execute user compiled functions within a workspace.

In addition, they have now launched in the UK, the first COMPILER for APL. Developed by STSC, it is available under both VM/CMS and MVS/TSO. It compiles individual functions written in VSAPL or VSAPL plus APL*PLUS enhancements. Improvements obtainable from compiling APL vary depending on the code, but results to date illustrate that a speed-up of the order 2 is not uncommon.

On the micro products side, as STSC's UK distributor, they are also launching the latest add-on to APL*PLUS PC, the SPREADSHEET MANAGER. It provides data preparation and analyses in a standard spreadsheet environment using APL formulae. It also includes transfers to and from APL, ASCII and LOTUS.

The company's in-house 'Introduction to APL2' and 'APL2 In-depth' courses are still in great demand. The in-depth course has been revised to incorporate a unit on APL2 system design – a badly neglected topic!

Cocking and Drury are proud to announce a new course: 'Intermediate APL*PLUS PC'. This public 3-day course is designed for existing APL programmers who need to increase their commercial applications development skills in the PC environment.

Inner Product Ltd.

Inner Product are maintaining and extending their APL support service, particularly to the financial institutions in the City of London.

Also they are making a major effort to provide, through packages like ALLWAYS and VIEWCOM, flexible communications on the IBM PC family of machines.

MicroAPL Ltd.

The Commodore Computer Show (May '86) may sound like an implausible venue for MicroAPL to display its wares but fighting hard next to "Yie Ar Kung Fu" and "Rambo" was APL.68000. On our stand in the Amiga village we received an encouraging welcome from the over-5s in the Commodore community, obviously hungry for some real software rather than the usual nonsense of games. Being gluttons for punishment the shell-shocked MicroAPL troops voted the show a resounding success and resolved to appear at the Atari Show in September.

Returning to sanity for a while I am pleased to announce that APL.68000 is now available for the NCR Tower-32. MicroAPL's Managing Director, Richard Nabavi, comments:

"The combination of MicroAPL's highly optimised APL.68000 product and NCR's 32-bit supermicro means that there are now very few applications which need the computational power of a mainframe. We come across lots of DP managers wanting to move APL applications off IBM mainframes on to an APL.68000 based system, this trend will accelerate now that our Tower-32 implementation is complete."

APL.68000 is now available on a wide range of systems other than MicroAPL's own Spectrum and Aurora supermicros. These include Sun, Hewlett Packard, Fortune, Apple, Wicat, Stride and TDI's Pinnacle.

Another new product to surface from the depths of MicroAPL's Research & Development department is Mac-APL, APL.68000 for the Apple Macintosh, the one you've all been waiting for, haven't you? You will be able to see and use Mac-APL on our stand at APL86 in July. If you can't wait that long then you can place an order with the Sales Department now. Mac-APL costs £295 (inc. VAT).

MicroAPL's third major announcement concerns support for Motorola's MC68020 and MC68881 processors from within APL.68000. Version 6.0 of APL.68000 will take advantage of the extended instruction set of Motorola's new 32-bit microprocessor and implements most floating point operations in hardware using the MC68881 floating point co-processor. Optimisation has also been performed on Boolean, integer and character monadic and dyadic scalar functions. The speed improvements gained are in some cases dramatic. Luan Thompson, Director of Research & Development, says enthusiastically, "On systems of equivalent clock rate integer addition is some 10 times faster, floating point multiplication is 15 times faster and logarithm is some 70 times faster than APL.68000/5.0." Since the MC68020 processor runs at twice the clock rate of the MC68000 these improvements are at least doubled when run on a 32-bit machine such as MicroAPL's new Aurora super-microcomputer. Work is still in progress on the upgrade, in such areas as formatting.

APL SOFTWARE TECHNOLOGY (UK) LIMITED POWERTOOLS/PC

*the software product which will transform your
APL *PLUS/PC workspaces*

The superb Forms Processor:

- Multiple forms that may be larger than the screen
- Inbuilt data validation
- Extended Numeric field and date editing
- APL function execution at the field level
- For the developer Form definition and editing with semi-graphic drawing facilities and interactive Forms testing
- Horizontal and Vertical subsidiary forms scrolled in parallel with the main form for allowing row and column headings to be in step with the data
- Recognition of ☐FMT overflow character in derived functions
- Screen time out, allowing the program to trap inactivity
- Mixed graphics and text.

PLUS

- A wealth of useful functions
- Matrix catenation, indexing and membership
- String handling, super boxing
- Sub-string searching and indexing
- Booleans implemented as bits rather than words

*As a result of our success in our major UK and Scandinavian accounts
we have now extensively enhanced the product and can now
announce Release 3.*

Powertools/PC is written in Assembler, is really fast and CPU efficient.

Powertools/PC has been specifically designed to act as a base around
which the application system can be built. It is NOT the typical
AP124-type screen manager

Powertools/PC, the workspace and CPU saver, is just £295 per copy
(excluding VAT).

Production Run Time versions available upon request.

Now available

*A fully compatible version of the Forms Processor (rewritten in C) for
DIALOG APL under UNIX.
Contact us for full details.*

14 Rosewood Avenue, Alveston, Bristol BS12 2PP. Tel: 0454 415737.

British Computer Society News

compiled by Philip Goacher

Following all the publicity for our own APL86 Conference, it should be remembered that there are another fifty or so BCS Specialist Groups organising meetings and conferences throughout the year. Two other major conferences are to be held in September.

Software Engineering '86
10-12 September, University of Southampton

This is to be the first of a series of annual conferences on software engineering organised jointly by the B.C.S. and the Institution of Electrical Engineers in conjunction with the Alvey Directorate. The conference will cover the practical application of software engineering as well as the theory of new and established methodologies, and is intended to be a showpiece for advances in software engineering from around the world.

The following topics will be covered:

- Technology of the software development process
- Management of large software developments
- Experience with software engineering methodologies
- Software development metrics
- Software development tools
- Teaching and training

HCI '86 - People and Computers: Designing for Usability
22-26 September, University of York

Human Computer Interaction is rapidly assuming a position of major importance in all aspects of Information Technology. Increasingly the IT market expects products with better human interfaces. Meeting such expectations is becoming a matter of survival.

HCI '86 is Europe's key international forum for advancing the discipline of HCI. Here industrial delegates can survey the advanced techniques being developed to help them cope with these growing demands. Here too, researchers will discern the important uses on which to focus their talents, and it is here that this synergy between industry and research will be harnessed to push the frontiers of HCI still further forward.

It is becoming increasingly difficult and dangerous to ignore the challenge of the human element in IT. HCI '86 is here to help you meet that challenge.

The conference will have tutorials, workshops, discussion groups, an exhibition and a varied social programme, as well as papers on a variety of topics including:

- Tools and techniques for developing user interfaces
- Formal models of Human Computer Interaction
- Empirical evaluation of interfaces
- Matching system functions to user tasks
- Applications, including expert systems, graphics and special purpose systems for the handicapped.

For further information and booking details for either event, please contact:

BISL Conference Department, British Computer Society,
13 Mansfield Street, London W1M 0BP. (01-637 0471)

ANNOUNCING THE



COPROCESSOR FOR THE IBM PC

The Dyalog APL coprocessor allows large scale second-generation APL applications to be developed and run on an IBM PC or plug-compatible computer.

The Dyalog APL coprocessor consists of:

- a 32-bit coprocessor board based on the NS 32000 chip set, hardware floating-point, up to 4Mb of on-board RAM, and 16Mb of virtual memory space.
- a complete implementation of UNIX* System V including C, f77, Fortran and Pascal.
- a complete implementation of Dyalog APL.
- software that integrates the Dyalog APL coprocessor subsystem into the PC-DOS environment.
- a Dyalog APL character generator and set of keycap stickers.

The Dyalog APL coprocessor is easily installed in an IBM PC or plug-compatible system with a standard BIOS and a hard disk. The board simply slides into a single slot in the PC chassis, and the software is installed via an interactive menu.

The Dyalog APL coprocessor provides both DOS and UNIX System V environments without compromise. Data can be shared between the two environments, and a simple keyboard command switches between operating systems. DOS commands and files can be accessed directly from Dyalog APL, and Dyalog APL can be run directly from DOS.

The Dyalog APL coprocessor supports very large APL objects and workspaces, to a maximum of nearly 16Mb; and floating-point performance is up to 3 times faster than an IBM PC AT.

Prices, including a multi-user Dyalog APL licence, start from around £3,000. A complete ready-to-run system with a fast 40 mb disk and tape back-up is also available for under £7,000.

For further details and ordering information, please contact **Peter Donnelly**
Dyadic systems Ltd., Park House, The High Street, Alton,
Hampshire GU34 1EN, United Kingdom
Telephone: Alton (0420) 87024 (10 lines) Telex 858811

International APL news

Compiled by David Preedy

APLBUG

We have received two recent issues of the APLBUG newsletter with brief papers as well as reports on their recent meetings. The March newsletter contains a solution to the Loss Carry-Forward problem raised in VECTOR Vol.2, No. 1. Tibor Horvath's function is reprinted in the "Surely There Must Be A Better Way" column. The April issue reports on "Object Oriented Programming in APL" and how to overcome date problems for the year 2000 – and for the next 2 billion years provided the Earth's rotation is unchanged. APLBUG membership costs \$15 and applications should be sent to Curtis A Jones (see address list below).

APL Quote-Quad

Volume 16/3 of Quote-Quad contains a mix of notices, product news, articles, reviews, algorithms and recreation. The articles cover a Boolean Array Based Algorithm, Emulating Prolog, Trees and Networks using APL2, and an APL Workshop. Orders for APL Quote-Quad should be placed with:

Fred Aronson, Coordinator for SIGS,
Association for Computing Machinery,
11 West 42nd Street,
New York,
USA 10036.

International APL addresses

Each year we publish the list of international APL user-groups and similar organisations. The VECTOR editor would be pleased to receive information from any of these groups for publication.

Austria

APL-Club Austria,
Siegfried Streit, Chair,
Wiener Stadtwerke,
Mariannengasse 4-6,
A-1095 Vienna,
Austria.

Belgium BACUS

An affiliate of the Computational and
Applied Mathematics Group
Belgian APL-CAM Users Society,
Joseph de Kerk, Chair,
Rooienberg 72,
B-2570 Duffel,
Belgium.
Tel. 015-314724
IPSA mailbox: KERF

Canada: Toronto CIPS APL SIG

An affiliate of the Canadian Information
Processing Society.
Toronto Special Interest Group on
APL,
Richard Levine, APL co-ordinator,
Xerox Canada Inc.,
703 Don Mills Road,
Don Mills,
Ontario, M3C 1S2,
Canada.
Tel. (416)429-6750, ext.2684
(after 4pm)
IPSA mailbox: RLEV;
group box: TSIG

Finland FinnAPL

Finnish APL Assn.,
Arto Juvonen, Chair,
P. O. Box 1005,
SF-00101 Helsinki 10,
Finland.

France

A subsidiary of the Association Francaise
de Cybernetique, Economique, et
Technique

Group APL, AFCET,
Gilles A Martin, Chair,
C.I.S.I.,
35 Boulevard de Brune,
F-75014 Paris,
France.

Germany APLCG

APL-Club of Germany e.V.,
Prof. Dr. Wolfried Stucky,
Postfach 6380,
D-7500 Karlsruhe 1,
Federal Republic of Germany.

Italy

A subsidiary of the Associazione
Italiane di Calcolo Automatico
APL Club Italia,
Giorgio Faconti, President,
C.N.U.C.E.,
Via S. Maria 36,
I-56100 Pisa,
Italy.
Tel. (50)45245

Mexico

Soc. Mexicana de APL, A.C.,
Erick Alvarado, President,
Teleinformatica de Mexico, S.A.,
Arenal 40,
Mexico 20, Dist. Federal,
Mexico.
Tel. (905)550 80 33

Netherlands APLWG

A subsidiary of the System-Programming
Section of the Netherlands Society for
Informatics (NGI)

Werkgroep APL,
Dhr. Bram Kornaat,
Rijks Computer Centrum,
Fauststraat 1,
NL-7323 BA Apeldoorn,
Netherlands.
Tel. (55)778822

Portugal

Ass. Portuguesa da
Linguagem APL,
Jose M. Henriques,
Rua Pedro de Barcelos 14,
Encosta do Restelo,
P-1400 Lisboa,
Portugal.

Sweden SIG-APL

A subsidiary of the Swedish Society for
Informatics

APL Special Interest Group,
Bo Lundmark,
Sperry AB.,
P. O. Box 1138,
S-171 22 Solna,
Sweden.

Switzerland SAUG

Swiss APL User Group,
Dr. J L Metzger, President,
c/o Diametal A.G.,
Solothurnstrasse 136,
CH-2504 Biel,
Switzerland.
Tel. (032)413971

United Kingdom

A Specialist Group of the British Computer Society

British APL Association,
Dick Bowman, Chairman,
C.E.G.B.,
85 Park Street,
London SE1,
England.
Tel. 01-634 7639

USA:**Dallas-Fort Worth SWAPL**

Southwest APL Users' Group,
Don Hatfield, Secretary,
2809 Apple Valley,
Garland,
Texas,
USA 75043.

USA: Denver DIGAPL

Denver Interest Group for APL,
P. O. Box 3931,
Englewood,
Colorado,
USA 80155.

USA: Houston

Houston APL Special
Interest Group,
Mary Hall, President,
Charter Oil Company,
P. O. Box 87535,
Houston,
Texas,
USA 77287/7535.
Tel. (713)640-4724

USA:**New York City NY/SIGAPL**

New York Chapter, SIGAPL,
P. O. Box 524,
New York City,
New York,
USA 10025.

USA: Rochester

Rochester Chapter,
ACM SIGAPL.
Robert Pullman, President,
P. O. Box 92487,
Rochester,
New York,
USA 14692
Tel. (716)244-4732

USA: San Francisco APLBUG

APL Bay Area Users' Group,
Curtis A Jones,
Secretary/Treasurer,
210 South 12th St. Apt. 1,
San Jose,
California,
USA 95112.
Tel. (408)292-4060

USA: Seattle

UW APL Users Group/SIGAPL,
J. R. Atkins,
University of Washington DH-05,
Seattle,
Washington,
USA 98155.

USA: Washington

Washington Chapter,
ACM SIGAPL,
Lawrence Dusold,
Dept. of Health, Education & Welfare
HFF-146,
200 C Street SW,
Washington,
District of Columbia,
USA 20204.
Tel. (202)245-1413

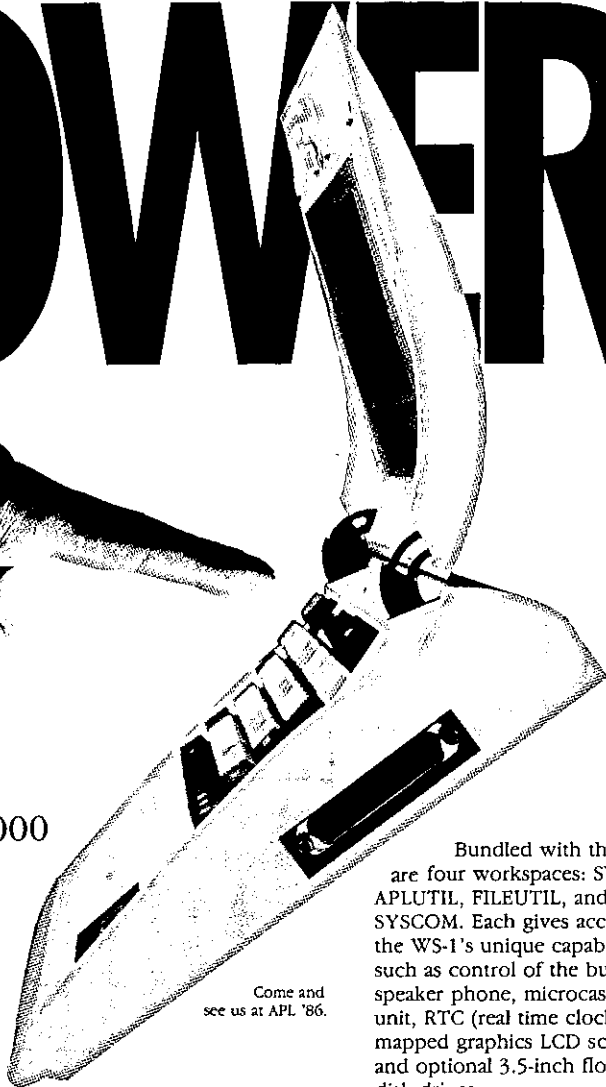
POWER



Enhancing APL.68000

An intriguing computer for an exciting language—the WS-1 and APL.68000. At last the APL programmer can have portability without sacrificing power or capability. Dodge the queue waiting for time on the main-frame and discover the sudden freedom of being able to try out programs anytime, anywhere.

The APL.68000 interpreter is implemented in 86KB of ROM, running under a multi-user, multi-tasking operating system called BIG. DOS. Speed is the essence of APL programming, and now the WS-1 makes development even faster.



Come and
see us at APL '86.

APL.68000 on the WS-1 has attractive enhancements such as a powerful component file system, QUAD. FMT function for alpha report formatting, QUAD. CC function for full-screen control, and extended error trapping facilities.

Bundled with the WS-1 are four workspaces: SYSFNS, APLUTIL, FILEUTIL, and SYSCOM. Each gives access to the WS-1's unique capabilities such as control of the built-in speaker phone, microcassette unit, RTC (real time clock), bit-mapped graphics LCD screen, and optional 3.5-inch floppy disk drives.

Compress these capabilities into a sleek footprint measuring less than 13 inches by 11 inches, and you have the ultimate definition of power.

ampère

FOR DISTRIBUTORSHIP INFORMATION AND PRODUCT DETAILS PLEASE CONTACT:

ampère
INCORPORATED

Asahi Bldg., 7-5-20 Nishi-Shinjuku, Shinjuku-ku, Tokyo, Japan. Phone: 03-365-0825.
Telefax: 03-365-0999. Telex: J33101 AMPERE. IP Sharp Mail Box Code AMP (Group Code APLWS).

APL Press Review

by David Preedy

Our irregular feature on the press coverage that APL receives is by definition limited to the publications that we, or our various spies around the world, peruse. This issue I feel sure that several articles must have slipped our net.

Our coverage was enhanced by a weighty bundle that fell through my letter-box, coincidentally all covering a certain supplier's new product announcements. Top of the pile was the extract from the December '85 issue of *Sinclair Gebruiker*. I just about struggled through the translation of:

"Micro APL voor de QL"

and discovered that the Dutch for "diskette" is "floppy", before abandoning the exercise for more productive tasks.

One of the English competitors of *Sinclair Gebruiker* must be *Sinclair/QL World*, and their April issue also had a two page introduction to this "more exotic language" now available on the QL. I'm not sure how many of the "floating voters" will have got far beyond the introductory paragraph:

"Though getting a little long in the tooth, having been around since the 1960s, APL . . . has a small but loyal following."

The rest of the article described some of the features of APL and of the QL implementation, concentrating on some aspects of the keyword facility. Unfortunately it never really explained why you would want to use APL instead of SuperBasic, so it's unlikely to bring about the mass conversion of all C5 users to APL.

Another of my spies sent me an extract from *Minicomputer News*, but unfortunately the date was cut off so I can't identify the exact issue. The context was quite interesting being one of those rare occasions when an application using APL is mentioned without a great song and dance about a strange back-to-front language written in Greek. The application was a foreign exchange pilot system for the Midland Bank Group Treasury. As the Microsystems manager said:

"We had three things going for us. A bunch of ATs; . . . some nine months' experience of the G/Net IBM PC local area network; . . . in-house expertise in APL, which is ideal for recalculating matrices of numbers."

Nearer to home, VECTOR itself has been receiving publicity from *Datalink*. The April 21st issue picked up my own call for a marketing campaign to boost interest in APL (Editorial – VECTOR Vol.2, No.3). I was indeed very impressed by the alacrity of their reporter who was on the phone the day after the issue had been posted. "Is it really as bad as that?" he asked. His article followed up my own comments with those of Romilly Cocking who:

"claimed there was a 'steady spread of understanding' which was enough to keep up the impetus behind the language."

VECTOR obviously gets passed quickly around the *Datalink* office, because by the following week, its GIGO column was doing its best to spread misunderstanding about APL and revealing major divisions within the ranks of our very own working group!

"Elsewhere in the same issue a press review by Steve Lyus seems to call some of Preedy's arguments into question in its extremely vigorous defence of all that is APL."

It seems that the merest hint that APL might make some sacrifice in terms of performance and machine efficiency (albeit to gain significant benefits in terms of development times), is tantamount to a statement that "APL gobbles up machine resources like nobody's business"! The tragedy is that such attitudes prevent anyone trying to develop a rational debate about the types of application where APL is or is not a suitable language to use.

One of the most enjoyable aspects of doing a press review is looking for the "ones that got away", those articles that cry out for a mention of APL and yet it never comes. This time the prize goes to *Computer Weekly* (May 1st) which unveiled "how we arrived at such a prolific number of programming languages". Despite this prolific number, the table headed "Significant programming languages" includes only 10 – Fortran, Algol, Cobol, Basic, RPG, PL/1, Coral, Pascal, C and Ada.

My thanks to all those who have sent in the cuttings they have noticed. No doubt next issue will report the worldwide coverage of APL86.

APL Product Guide

Compiled by Steve Lyus

VECTOR's exclusive APL Product Guide aims to provide readers with useful information about sources of APL hardware, software and services. We welcome any comments readers may have on its usefulness and any suggestions for improvements.

We do depend on the alacrity of suppliers to keep us informed about their products so that we can update the Guide for each issue of VECTOR. Any suppliers who find they are no longer included, have not bothered to reply to my two warnings. If they would like to be re-instated, would they please contact me to confirm their entries. Any suppliers who are not included in the Guide should contact me to get their free entry – see address below.

We reserve the right to edit material supplied for reasons of space or to ensure a fair market coverage.

The listings are not restricted to UK companies and international suppliers are welcome to take advantage of these pages. Where no UK distributor has yet been appointed, the vendor should indicate whether this is imminent or whether approaches for representation by existing companies are welcomed.

For convenience to readers, the product list has been divided into the following groups:

- ★ Complete APL Systems (Hardware & Software)
- ★ APL Timesharing Services
- ★ Other services
- ★ APL Interpreters
- ★ APL Visual Display Units
- ★ APL character set printers
- ★ APL-based packages
- ★ APL Consultancy
- ★ Vendor addresses

Every effort has been made to avoid errors in these listings but no responsibility can be taken by the working group for mistakes or omissions. The Guide has now been entered on our Wordprocessor, so there might be some this time!

Note: 'poa' indicates 'price on application'

All contributions to the APL Product Guide should be sent to:

Steve Lyus
2 Howard Road
Westbury Park
Bristol
BS6 7UT

(N.B. Change of Address)

COMPLETE APL SYSTEMS

COMPANY	PRODUCT	PRICES £	DETAILS
Analogic	The APL Machine	\$60,000+	AP500 array processor, 4 Mb data memory, 80 Mb disk drive.
APL Software Technology	Ericsson PC	2,000 -5,000	Authorised Ericsson dealer supplying complete APL systems based on Ericsson PC - fully IBM compatible including hardware add-on boards.
Cocking/Drury	MicroAPL SPECTRUM SAGE II SAGE IV	6,000 -35,000	Supplied as part of a turnkey system. See MicroAPL entry.
Gen. Software	Myriade	poa	TI computer + APL & APL operating system
Inner Product	IBM PC	2,000 -6,000	IBM PCs supplied for turnkey applications
M.B.T.	MBT Series 10 TORCH	poa poa	UNIX/68010 based multi-user APL system 68000/Z80 multiprocessor
MetaTechnics	—	3,000+	Details on application
MicroAPL	Aurora	23500	Multi-user APL computer using 68020 CPU. Std. configuration 2Mb RAM, 16 RS232 ports, 68 Mb hard disc, 720K diskette
	Sinclair QL with QL/APL	300+	Fully expandable APL system with colour graphics.
	SPECTRUM	11,000 -15,000	Expandable multi-user APL computer using Motorola 68000. Std. configuration 1 Mb RAM, 12/36 Mb disc, 12 ports.
	STRIDE 440	8,500	Multi-user APL computer, 1 Mb RAM, 12/18 Mb disc.

APL TIMESHARING SERVICES

COMPANY	PRODUCT	PRICES £	DETAILS
Boeing	Mainstream APL	poa	Enhanced IBM VS APL (CMS)
Mercia	APL*PLUS	poa	STSC's Mainframe Service - MAILBOX etc.
I.P. Sharp	SHARP APL	poa	International Network application systems and public databases.

OTHER PRODUCTS

COMPANY	PRODUCT	PRICES £	DETAILS
APL People	Employment Agency	poa	Permanent employees placed at all levels. Contractors supplied for short/long-term projects, supervised.
Mine of Information	APL Book Service		See booklist
I.P. Sharp	Productivity Tools	poa	Utilities for systems, operations, administration & analysts; auxiliary processors, comms software, international network.
	Databases	poa	Financial, aviation, energy and socioeconomic.

APL INTERPRETERS

COMPANY	PRODUCT	PRICES £	DETAILS
APL Software Technology	APL*PLUS/PC	600 - 695	See Cocking & Drury entry. Release 5 now available. Discounts available on multiple copies.
Cocking/Drury	APL*PLUS/PC	695	STSC's full-feature APL for IBM PC, PC/AT, Compaq, Olivetti, Wang, Apricot, Ericsson etc
	Release 5		
	Release 4 update	150	Extension upgrade from release 4
	Release 3 update	110	Extension upgrade from release 3
	Run-time	poa	Closed version of APL*PLUS/PC which prevents user exposure to APL
	APL*PLUS/UNIX	poa	STSC's 2nd generation APL for IBM PC/AT, DEC, AT&T and other Unix computers.
Dyadic	Dyalog APL	1,000 - 8,000	2nd gen. portable APL for UNIX systems, e.g. IBM PC AT, SUN, MC68000, HP9000, NCR, VAX, Gould, Amdahl.
Gen. Software	APL*MYRIADE	poa	Runs on Texas Instruments TI990 range.
IBM UK Product Sales	IBM PC APL	poa	Event-handling & APs for full-screen I/O disks, diskettes, asynch. comms.
Inner Product	VIZ::APL	250	8-bit Zilog Z-80 CP/M
		- 350	
	APL*PLUS/PC	600	See under Cocking & Drury
M.B.T.	Dyalog APL	poa	See Dyadic Systems entry
	MBTAPL	poa	Enhanced Dyalog APL for MBT hardware.
	VIZ::APL	poa	Customized for TORCH hardware
Mercia	APL*PLUS/PC	695	STSC's full-feature APL for IBM PC, PC/AT Compaq, Olivetti, Wang, Apricot, Ericsson etc
	Release 5.0 upgrade to 5.0	150	
	APL*PLUS/UNIX	poa	Interpreter for UNIX systems: WICAT, CADMUS, CALLAN, FORTUNE 32:16, HP, 9000/500, OLIVETTI 3B2
MetaTechnics	APL*PLUS Release 4	695	Discount on quantity.
MicroAPL	APL.68000	1,000+	Full implementation with component files, error trapping etc. for SPECTRUM, SAGE & other MC68000-based computers.
	QL/APL (keyword)	87	Full keyword APL for QL with many extra features.
	QL/APL (APL chars)	87	VSAPL compatible APL for QL with many extra features.
	MAC/APL	257	
	AMIGA/APL	326	
	ATARI/APL	131	
	APL*PLUS/PC-REL 5	495	
Portable	PortAPL	\$195	IBM PC Software
		\$275	Mackintosh
		\$2,995	DEC VAX
I.P. Sharp	Sharp APL/PCX	2,575	For IBM XT/AT
		1,000+	For IBM mainframes
	Sharp APL/PC	325	For IBM PC or PC/XT

APL VISUAL DISPLAY UNITS

COMPANY	PRODUCT	PRICES £	DETAILS
Farnell	Tandberg TDV 2221	995	Ergonomic design APL terminal, 50-19200 baud, 15" anti-reflex screen, low profile keyboard
Gen. Software	Tandberg TDV 2271	1,195	Combined APL/ANSI ergonomic terminal as above.
	Mellordata Elite 3045A	400	Second-hand
M.B.T.	various		Contact MBT for details
MicroAPL	Insight VDT-1	795	Inexpensive APL VDU
	Insight GDT-1	1,450	With monochrome graphics
	Tektronix 4105	3,545	High resolution colour graphics supporting APL character set on MicroAPL hardware.
	Concept 201	1415	APL VDU with 8 page memory
	Concept 201G	1935	Graphics VDU
Shandell	AVT-APL+	1,200+	ANSI 3.64; DEC compatible; full overstrike chars; 4/8 pages; 2/3 comms, ports; 80/132 cols; windowing; 12" screen; 46 PF keys; Tek 4013 graphics compatible.
	GVT-APL+		As above plus 15" screen; viewports; smooth scroll; NVM storage; 55 PF keys; Tek 4014, VT640/DQ640 & Visual 500 graphics compatible.
Textronix	4114B	13,500+	19" D.V.S.T.; Graphics: 3120 x 4096 displayable; Intelligent: up to 800K memory; APL keyboard (option 4E)
	4125	21,550+	19" 2D colour graphics; Workstation (1280 x 1024); Intelligent: up to 800K memory; APL keyboard (mod AP)
	4128	26,822+	As 4125 plus 3D wireframe

APL PRINTERS

COMPANY	PRODUCT	PRICES £	DETAILS
Datatrade	Datasouth DS180+	1,295	180 cps matrix printer with 4K buffer, 9 x 7 dot matrix and APL option.
	Datasouth DS220	1,695	Letter quality; graphics capability, APL option (both available with IBM Twinex or Coax Interface).
Inner Product	Epson FX80	500	Soft char. set, 160 cps, 80 column
	Anadex 9620	1,150	200 cps., 132 col., tractor feed
	Siemens PT88	620	180 cps., 80 col., silent
	TGC Starwriter	1,180	40 cps., letter quality
M.B.T.	Facit 4565	poa	40 cps letter-quality
	Facit 4510/11/12	poa	Matrix printers
MetaTechnics	Quen-data	295	Low-cost APL Daisy-wheel printer
MicroAPL	Datasouth DS180+	1,295	See Datatrade entry
	Philips GP300	1,924	Matrix printer with letter & draft quality and APL.
	Qume Letterpro20	549	APL/ASCII Daisy-wheel printer

APL PACKAGES

COMPANY	PRODUCT	PRICES £	DETAILS
APL ◊ 385	FSM 385	PC: 50	Screen development
	DRAW 385		Screen design
	DB 385	Mainframe: 125	Relational W.S.
	GEN 385		Utilities
APL Software Technology	POWERTOOLS/PC	295	Assembler written replacement function for commonly used CPU-consuming APL functions. Includes extended full-screen Forms Processor.
Boeing Beta-plan	TABAPL	poa	Hierarchical Planning System
	BETA-FONT	poa	Multiple font PC character generator. Dealers required for non-Scandinavian countries.
Cocking/Drury	<i>Mainframe</i>		
	STSC's SHAREFILE	poa	Component files, quad- functions & nested arrays for IBM VSAPL under VM/CMS & MVS/TSO
	SHAREFILE AP	15,000	STSC's sharefile for APL2
	EMMA	poa	STSC's database package.
	CALL/AP	3,000	Non-APL program execution from VS-APL or APL2 under VM/CMS & MVS/TSO
	AFM	10,000	High-performance shared files for VS-APL
	FORMAT	2,250	Enhanced report formatting.
	WSAIDS	795	Workspace doc. & development aids.
	COMPILER	30,000	First APL compiler. Available with APL*PLUS enhancements and Sharefile under VM/CMS & MVS/TSO
	<i>Microcomputer</i>		
	STATGRAPHICS PC	995	Statistics & Graphics for IBM PC, AT, etc.
	APL*PLUS PC Tools		
	VOL 1	325	Incl. 327x IRMA support, RAM disk, full screen data entry, menu input, report generation, games.
	VOL 2	125	Incl. file documentor, screen editor, exception handler.
	APL*PLUS PC Fin & Stat. Library	350	Financial & statistical routines
	APL/DIF	145	Interface between APL*PLUS PC and DIF file supporting spreadsheets.
	MetaScreen	245	See MetaTechnics entry.
	MetaPack	95	See MetaTechnics entry.
	POWERTOOLS	295	See APL Software Technology entry.
	SPREADSHEET MANAGER	195	APL-based spreadsheet for APL*PLUS/PC. Cell arithmetic; transfers to ASCII, LOTUS
E&S	PROTOPAK		Packages for prototyping management information systems – PC & mainframe
	consisting of:		
	RMS	Modules	Relational databases.
	AMS	250+	Multi-dimensional arrays
	RAMS		Combined RMS & AMS.
	BMS		Dynamic financial modelling & forecasting
	FMS		Full-screen handler for APL*PLUS/PC. (AP 124-based)
	CMS		Communications package.
Gen. Software	SOS	poa	Scheduled ordering and stock control.
	PROPS	500+	Spreadsheet system for Product and/or Project Planning.
H.M.W.	INPUT	poa	Matrix manipulation package for data entry & report generation
	PRINTPAK	poa	Block printing for VM/CMS
	VIEWPAK	poa	AP124 Protocol emulator for IBM/PC
Holtech	CASH	3,500	Accounting package & hotel management system on MicroAPL
		~10,000	SPECTRUM & SAGE CPUs.

Inner Product	Viewcom	150	Control Viewdata from APL
	APL/dBASE II	150	Interface APL with dBase II
	APL/dBASE III	150	Interface APL with dBase III
	APL/LOTUS	150	Interface APL with Lotus
	APL/WORDSTAR	150	Interface APL with Wordstar
	APL/MULTIPLAN	150	Interface APL with spreadsheet
	CEMAS	3,500	EEC monetary and agrimonetary analysis.
M.B.T.	RHOMBUS	poa	Integrated Office System
	HASLEMERE	poa	Hotel Accounting System
Mercia	STATGRAPHICS	995	Integrated stat. graphic system for PCs.
	APL*PLUS tools		
		VOL 1	325 IBM PC Utilities: IRMA3270 comms, full screen, RAM Disk report generator
		VOL 2	125 File documentation, screen editing. Exception handling
	FINANCIAL AND STATISTICAL LIB.	325	Financial and Statistical analysis
	INFO CENTRE	2,000	Full-screen entry, display & multi-dimensional analysis. Interfaces to other I.C. products. Runs under VM VSAPL on IBM mainframes.
		-20,000	
	EXECUCALC	4,000	Mainframe Spreadsheet with VisiCalc and Lotus 1-2-3 functionality requires VSAPL under TSO or VM.
	EXECUPLOT	3,200	Mainframe Graphics display system with VisiPlot functionality requires VSAPL under TSO or VM and GDDM.
	EXEC*U*STAT	495	Easy to use Statistics for management.
MetaTechnics	MICROSPAN	250	
	MetaScreen	245	Full-screen handler for APL*PLUS/PC, based on VSAPL AP124
	MetaPack	495	Comprehensive utilities package for APL*PLUS/PC. Includes MetaScreen, MetaWS, Browse, Toolbox, Numeric Editor
	ADAPTA DLS	poa	Production & purchasing scheduling for process manufacturing.
	ADAPTA MSP	poa	Job-shop loading & scheduling for multi-stage production.
MicroAPL	MicroTASK	250	Product development aids
	MicroFILE	250	File utilities and database
	MicroPLOT	250	Graphics for HP plotters etc
	MicroLINK	250	General device communications
	MicroEDIT	250	Full screen APL editor
	MicroFORM	250	Full screen forms design
	MicroSPAN	250	Comprehensive APL tutor
	MicroGRID	poa	Ethernet & other networking
	APLCALC	400	APL spreadsheet system
	MicroPLOT/PC	250	For APL*PLUS/PC product
	MicroSPAN/PC	250	For APL*PLUS/PC product
	PC TOOLS Vol 1	495	
	STATGRAPHICS	695	
Parallax	ExecuCalc	\$5,000	Mainframe-based electronic spreadsheet for VM/CMS & MVS/TSO with links to micro products.
	ExecuPlot	\$5,000	Mainframe-based colour graphics with micro links.
I.P. Sharp	ACT	poa	Actuarial system
	APS	poa	Financial Modelling
	BOX/JENKINS	poa	Forecasting technique
	CONSOL	poa	Financial Consolidation
	COURSE	poa	APL Instruction
	EASY	poa	Econometric Modelling
	FASTNET	poa	Project Management
	GLOBAL LIMITS	poa	Exposure management for banks
	MABRA	poa	Record maintenance/reporting
	MAGIC	poa	Time series analysis/reporting
	MAGICSTORE	poa	N-dimensional database system

MAILBOX	poa	Electronic Mail
MICROCOM	poa	Mainframe to micro link
SAGA	poa	General graphics, most devices
SIFT	poa	Forecasting system
SNAP	poa	Project management
SUPERPLOT	poa	Business graphics
VIEWPOINT	poa	4GL - Info centre product
XTABS	poa	Survey Analysis

APL CONSULTANCY

(prices quoted are per day unless otherwise marked)

COMPANY	PRODUCT	PRICES £	DETAILS
APL People	Consultancy	poa	All levels, most APL systems
APL Software Technology	Consultancy	poa	Technical & business systems, micros, networking & communications a speciality.
Boeing	Consultancy	poa	
Camacho	Consultancy	poa	Specialising in programming & manual writing.
Cocking/Drury	Consultancy	120-150 140-200 185-300 275-400	Junior consultant Consultant Senior consultant Managing consultant
	Courses	375 330 595 poa poa	3 day Fundamentals 3 day Advanced 5 day System Design Introduction to APL2 APL2 in depth
Delphi	Consultancy	poa	Specialising in management reporting systems and APL on microcomputers.
Dyadic	Consultancy	poa	APL system design, consultancy, programming & training for Dyalog APL, VSAPL, APL*PLUS, IPSA APL etc.
E & S	Consultancy	150 - 250	System prototyping: all types of information system.
FASTCODE	Consultancy	poa	Specialise in improving performance of APL applications on micros & mainframes.
Gen. Software	Consultancy	100+	
H.M.W.	Consultancy	100-250	System design consultancy, programming.
Inner Product	Consultancy & Training	200	On-site micro-mainframe APL, PC/DOS & Assembler
Lloyd Savage	Consultancy	poa	Decision support, particularly specialising in Sales & Marketing systems.
M.B.T.	Consultancy & Courses	poa	
Mercia	Consultancy & Training	poa	APL*PLUS & VSAPL consultancy. Onsite & Offsite APL courses.
MetaTechnics	Consultancy	poa	Management Information & Production. Management Systems & Process Control.
	MetaSys	poa	Prototyping APL-Hybrid Custom Programming Service.
MicroAPL	Consultancy	poa	Technical & applications consultancy.
Parallax	Consultancy & Training	\$750 poa	Introductory APL, APL for End-user & Advanced Topics in APL
QB On-Line	Consultancy	200	Specialising in Banking, Financial & Planning Systems.
I.P. Sharp	Consultancy	poa	Consultancy & support service world-wide.

VENDOR ADDRESSES

COMPANY	CONTACT	ADDRESS & TELEPHONE No.
Analogic Corporation	Denise Favorat	8 Centennial Drive, Centennial Industrial Park, Peabody, Mass. U.S.A. 01961 ☎ 617-246-0300
APL 385	Adrian Smith	Brook House, Gilling East, York. ☎ 04393-395
APL People	Valerie Lusmore	17 Barton Street, Bath, Avon. ☎ 0225-62601
APL Software Technology	John Hagger or Per Hultin	14 Rosewood Avenue, (UK) Ltd. Alveston, Bristol BS12 2PP. ☎ 0454-415737 46 Vicarage Hill South Benfleet, Essex SS7 1PB. ☎ 03745-50501
Beta-plan APS	Kim Andreassen	Stengrade 75, DK-3000 Helsingør, Denmark. ☎ 45 221 48 48
Boeing Computer	Anne Harding	19 Fitzroy Street, Services (Europe) Ltd., London W1P 5AB. ☎ 01-631 0808
Anthony Camacho		2 Blenheim Road, St. Albans, Herts AL1 4NR. ☎ St. Albans 60130
Cocking & Drury Ltd.	Romilly Cocking Brian Drury	16 Berkeley Street, London W1X 5AE. ☎ 01-493 6172 155 Friar Street Reading RG1 1HE. ☎ 0734-588835
Datatrade Ltd.	Tony Checksfield	36 Billing Road, Northampton, NN1 5DQ. ☎ 0604-22289
Delphi Consultation Ltd.	David Crossley	Church Green House, Stanford-in-the-Vale, Oxon SN7 8LQ. ☎ 03677-384
Dyadic Systems Ltd.	Peter Donnelly	Park House, The High Street, Alton, Hampshire. ☎ 0420-87024
E & S Associates	Frank Evans	19 Homesdale Road, Orpington, Kent BR5 1JS. ☎ 0689-24741
Farnell International Instruments Ltd.	R. Fairbairn or Roger Attard	Jubilee House, Sandbeck Way, Wetherby, W. Yorks. ☎ 0937-61961 Davenport House, Bowers Way, Harpenden, Herts. ☎ 05827-69071
FASTCODE	Andrew Dickey	P.O. Box 281, Croton-on-Hudson, New York 10520, U.S.A. ☎ (914) 271-3200
General Software Ltd.	M.E. Martin	22 Russell Road, Northolt, Middx. UB5 4QS. ☎ 01-864 9537
H.M.W. Programming Consultants Ltd.	Ken Jackson	142 Feltham Hill Rd, Ashford, Middx. TW15 1HN. ☎ 07842-41232
Holtech Ltd.	Jan Bateman	'O' Block 4th Floor, Metropolitan Wharf, Wapping Wall, London E1 9SS. ☎ 01-481 3207
IBM UK Ltd	Chris Sell	PO Box 32, Alencon Link, Basingstoke, Hants. RG21 1EJ. ☎ 0256-56144
Inner Product Ltd.	Dominic Murphy	Eagle House, 73 Clapham Common Southside, London SW4 9DG. ☎ 01-673 3354
Lloyd Savage Ltd	Philip Johnson	Cambridge House, Oxford Road, Uxbridge, Middx. UB8 2UD. ☎ 0895-59826
Mercia Software Ltd.	Gareth Brentnall Barrie Webster	Aston Science Park, Love Lane, Birmingham B7 4BJ. ☎ 021-359 5096
MetaTechnics Systems Ltd	John Stenbridge David Toop	Unit 216, 62 Tritton Road, London, SE21 8DE. ☎ 01-670 7959
MicroAPL Ltd.	Jonathan Matthews Paul Thornton	Unit 1F, Nine Elms Industrial Estate, 87 Kirtling Street, London SW8 5BP. 01-622 0395
Mine of Information	Richard Ross-Langley	PO Box 1000, St. Albans, Herts AL3 6NE. ☎ 0727 52801
Modern Business Technology Ltd. (MBT)	Michael Branson	P.O. Box 87, Guildford, Surrey GU4 8BB ☎ 04868-23956
Parallax Systems Inc.	Kevin Weaver	60 West 9th Street, New York, New York 10011, U.S.A. ☎ 212-475-4001
Portable Software	Richard Smith	60 Aberdeen Ave, Cambridge, Mass. U.S.A. 02138. ☎ 617-547-2918
QB On-Line Systems	Philip Bulmer	5 Surrey House, Portsmouth Rd Camberley, Surrey, GU15 1LB. ☎ 0276-20789
Shandell Systems Ltd.	Maurice Shanahan	12 High Street, Chalfont St. Giles, Bucks HP8 4QA. ☎ 02407-2027
I.P. Sharp Associates Ltd.	David Weatherby	10 Dean Farrar Street, London SW1. ☎ 01-222 7033
Tektronix UK Ltd.	Paul Morgan	Fourth Avenue, Globe Park, Marlow, Bucks SL7 1YD. ☎ 06284-6000

APL BOOKLIST**List of books stocked and on sale**

Title	Price	UK Post & Pack (£)
An Interactive Approach, Gilman & Rose	24.35	2.65
APL: An Introduction, H. Peele	15.00	2.25
Introducing APL to Teachers, K. Iverson	3.00	.40
Introduction to APL for Scientists & Engineers	3.00	.40
An APL Notebook, Barrie Wetherill	1.90	.50
A Source Book in APL, A. Falkoff & K. Iverson	8.75	1.80
APL: Design Handbook for Commercial Systems, Smith	13.10	1.50
APL and Insight, P. Berry and G. Bartoli	4.50	.55
APL in Practice, STSC	40.00	2.50
APL: The Language and its Usage, R. Polivka & S. Pakin	36.00	2.70
APL Quote-quad the Early Years	31.50	2.70
APL Business Technology '83 Proceedings	11.20	2.70
APL SV Reference Card	.50	.50
Designing APL Systems, Toronto Press Procs '82 Vol 2	8.50	1.80
FinnAPL Idiom Library	11.20	1.30
Sharp APL Reference Book	1.50	.50
Sharp APL Reference Manual, P. Berry	10.50	2.70
Yale University Idiom List	2.00	.50
APL in Exposition, K. Iverson	3.00	.50
Algebra, K. Iverson	12.00	2.30
Solutions to Algebra, J. Iverson	3.95	.50
Elementary Analysis, K. Iverson	13.50	1.50
Resistive Circuit Theory, R. Spence	20.00	2.80
Star Map, P. Berry & J. Thorstensen	6.00	.50
Reliable Software through Composite Design, Myers	15.00	1.30
APL Lapel Pin with gripping backplate	2.00	.50
Whizzbangs, Volume II, R. Sykes	10.00	1.80
Whizzbangs, Volume I, R. Sykes	10.00	.40
Information Centres Toronto Procs '84	8.00	1.80
Application Systems in APL, Gibson Levine Metger	30.00	2.70

Please order direct from Mine of Information:

APL Book Service, PO Box 1000, St. Albans, AL3 6NE, U.K. ☎ 0727-52801

This list replaces all previous lists

Prices are in pounds sterling and are as at 1st January 1986

Prices are subject to change without notice.

Terms: Access & Visa accepted, or sterling cheque with order;

Outside UK add an *extra* amount per book –

£1 to Europe, £2.50 Africa & Middle East, £5 elsewhere (all sent airmail).

APL CONSULTANTS

LONDON & READING

Account Managers	(6 years+)	to 23K
Senior Consultants	(4-6 years)	to 19K
Consultants	(2-4 years)	to 16K
Junior Consultants	(1-2 years)	to 11K

Are your APL skills and potential being recognised and rewarded?

Cocking & Drury consultants have been implementing successful decision support applications for 10 years, with clients who appreciate the productivity benefits of APL.

In our professional team you will experience a range of APL environments - APL*Plus, VSAPL, APL2 and Unix, on both mainframes and micros. You will also be developing systems which, increasingly, need to interface with non-APL Information Centre products.

Of course as the leading APL consultancy, in a rapidly expanding market, we offer a rewarding career with first class benefits - profit sharing, free health insurance, and a non contributory pension.



For further details call Brian Drury on 01 493 6172

COCKING & DRURY LTD.

16 Berkeley St, London W1X 5AE

Book reviews

reviewed by Paul Chapman

The APL Toolkit, published by CIPS APL SIG, First Edition

In the preface, the editor Richard Levine describes the book as the result of efforts by the Canadian Information Processing Society's APL Special Interest Group (CIPS APL SIG) from 1983 to 1985. It must also be said that he admits that the book is not yet finished, but a second edition will be forthcoming with the last two sections completed, based on feedback from the APL community.

The book is organised in six chapters. The first two provide a general introduction and a guide for the use of the book. A directory of functions grouped under categories follows, and then the functions themselves. Finally come the incomplete chapters, documenting the categories and the functions respectively.

The functions presented range from development tools (name list, function documenting, timing, etc.) through general programming utility functions (looping, timestamp formatting, the ubiquitous BOX, etc.) all the way to application functions (e.g. mortgage amortisation).

The function listings themselves reveal a healthy smattering of comments, and a refreshing avoidance of "one-liners". All functions are claimed to be standard-conforming and independent of global system variable settings, except, in the latter case, where dependence is natural (e.g. where the global setting of the index origin is used to produce a result containing indexes). Each function should, according to the introduction, contain a standard set of comment lines at the beginning, listing the function category, author, timestamp, version and example. Some of the lines are unfortunately missing from many of the functions, and where they are present the advertised format is not always adhered to.

The functions themselves are listed alphabetically. This seems to be a hangover from the way functions are listed in the active workspace. I cannot see why this is a better way than listing them in different sections according to category, especially in this case where each function belongs to just one category. An alphabetical index is very easy to produce, and presenting functions by category allows the reader to compare the different approaches available to solve his own problem. Much of the detail seems to have been machine-produced, and the on-line version of the utilities workspaces said to contain functions which directly support the idea of categories; so it would seem to have been easy to print the book in sections like this.

My major criticism of the book is the incompleteness of the last section, where the individual functions are documented. Of the 115 functions listed in the book, only 16 appear in this last chapter. Were the functions presented in a machine-readable form, my objection would be less, since one of the oldest (and still one of the strongest) APL maxims around would apply: if in doubt, try it out. However, most of the more interesting functions are more than 50 lines in length, and typing one of these in only to find that it doesn't quite do what you wanted could be a frustrating exercise.

The one thing that caught my eye in this otherwise straightforward book is a method for defining local functions inside functions, using comment lines whose second character is a single digit defining which local function the line belongs to. Where it is necessary to write monolithic utility functions which can be easily copied from workspace to workspace, this is the clearest way I have seen so far of presenting local functions.

The book is not easy to use, but some of the functions in it may well make it worth obtaining. Many of the functions have been introduced as similar system functions on some APL implementations, but where portability is a concern the book could be very useful.

FinnAPL Utility Library, published by FinnAPL, First Edition

No individual credits are given in this book of APL utility functions. The brief introduction explains that the book comprises the 101 most important functions in the "Special" workspace of the Finnish APL Association, FinnAPL.

The introduction also refers to the "widely accepted standard" of underscoring the last letter of each function name. I don't know of this standard, but I assume its purpose is to avoid name conflicts between utility functions and application functions. However, if it were to become "widely accepted", it would defeat its own purpose, since all publishers of utility functions would conform to it, and thereby introduce name conflicts between different sets of utility functions from different publishers. A better approach would be to start each function with the letters "FINN", or some such, to identify the origin of the function.

Apart from the one page introduction and four pages of contents, the book consists simply of one function per page or two. The functions are grouped alphabetically within categories. The categories themselves appear by name in the contents pages, but are not mentioned thereafter. No full alphabetic function index is provided, which would be a simple and useful addition.

The documentation for each function is consistent and well laid out. Its name and purpose are given, followed by its calling syntax. Then there is a detailed description of each of the arguments and the result, with some additional informal comments. One or two clear examples follow, and finally the function itself is listed. Where a function needs to call other functions listed in the book, their names are listed explicitly.

Many of the functions will be familiar to readers of FinnAPL's excellent idiom book. Some of the function lines tend towards the obscurity of those one-liners, particularly in their length and their use of embedded assignment. This is quite unnecessary within a function: an extra line or two spent in clarifying the inner workings would be well spent.

No independence from the global environment is claimed, but I suspect from looking at the listings that this is achieved in every case.

This is a most attractive and useful book. Its layout is excellent, and the functions themselves form a consistent set of utilities. Apart from adding an index, all I can suggest by way of improvement is: more please!

Software Review : AFM

by Adrian Smith

Introduction

AFM is a file management package which runs as an Auxiliary Processor with either VS APL or APL2. It provides IBM's APL with component files along the lines of the APL*PLUS shared file system; simple cover functions double for `□ FTIE`, `□ FREAD` and so on.

AFM files can be accessed concurrently by online APL systems and from conventional batch programs (typically written in PL/1 or COBOL). File and record-level locking is provided, with security handled through access matrices in the normal way.

A wide range of 'housekeeping' and 'administrator' functions are provided, some via an 'OPFNS' workspace and some as batch utilities.

The cost of AFM is around \$15,000 depending on the particular options chosen. It is marketed by Interprocess Systems Inc in the USA, Cocking and Drury being the UK agents.

Background to Use at RM

AFM was brought in to Rowntrees about a year ago. The motivation for doing this was threefold:

1. We needed to redesign an antique VSPC (ex CMS) system which made heavy use of direct files. We could have set it all up with keyed VSAM files, but AFM promised a much quicker solution at a very reasonable cost.
2. We had already anticipated a VSPC to TSO conversion. A pre-emptive move to AFM looked like smoothing the path very considerably.
3. Experience from RM Canada (under VM/CMS) suggested that we could expect significant CPU savings over our standard file routines.

In addition it was becoming clear that we would soon be faced with implementing APL systems on a variety of PCs, all of which had some variant of the APL*PLUS/PC file system. The idea of a compatible set of file utilities across all environments was obviously very attractive.

At RM the vast majority of APL filing is handled through two utilities called (with startling originality) DUMP and GET. DUMP simply takes a list of APL objects (a delimited vector or in `□ NL` format) and bungs them all on a file. Typically there will be 50 - 80 objects, say 100K of data in all. GET reverses the process; either everything on the file, or selected variables, reappear in your workspace.

Under VSPC by far the best way of doing this turned out to be to format and ravel everything, and then to write it out (via the EBCDIC processor) in 4K chunks. The potential economies from cutting out all those 'executes' and 'reshapes' are obvious!

Installation and Housekeeping

Getting AFM up and running was simply a matter of following the instructions! However getting it running fast required a fair amount of work from our technical support group; mainly a matter of getting the VSAM files defined right, getting catalogs in the right place and so on. The documentation was felt to be adequate, but could have been indexed better!

The utilities for file dump and restore, and the general overnight housekeeping jobs were well documented, and have caused us no problems. The OPFNS workspace is quite adequate, but can be improved out of all recognition by a bit of attention to the user-interface. Fortunately all the code is unlocked and commented, so simple patches like accepting lower-case responses (our administrator doesn't have an APL keyboard!) are very easy to do.

There are some minor beartraps, for instance if you mistype the TSO id which you want associated with a particular AFM user, you can run a batch import for that user, but can never again get at the files! You have to wipe them from the system and try again; not serious but avoidable I would have thought.

Using AFM from an APL Application

The first thing I must say is that the User Guide is absolutely first class. It is well organised and is readable as a tutorial as well as being a very adequate reference manual. Experienced APLers have been able to pick up the basics, and to write quite sophisticated systems, with no guidance other than a copy of the manual. I suppose all this proves is that component files are a natural extension of APL, and that it is absurd that they have never been made a native part of the IBM offerings.

My only gripes concern the very peculiar way the AFM cover functions attempt to reproduce APL*PLUS error handling. Functions such as FTIE are result-less, and leave various obscurely named globals lying around for inspection (if you've previously set up an equally obscure global to define the range of errors you want to detect . . . otherwise its DOMAIN ERROR time!). I'm afraid I rewrote most of them to return explicit results, again an easy task as the source code is very straightforward. Of course you have problems with things like FREAD; this sends back an empty vector if it fails, and you then have to go looking for that pesky global again. I could almost see a case for nested arrays!

Performance

It proved easy enough to get our DUMP and GET routines to work with AFM, but quite a task to make them as fast as they were before. The simple-minded approach of writing one object per component is hopelessly slow (in elapsed terms) when you have lots of little variables. It also leaves you wondering what to do with the odd whopper; you have to set your maximum record size somewhere, and wherever you set it (16K, 32K, 64K, . . .) sooner or later someone will bust it.

We eventually settled on a compromise of partitioning the file into character, integer and real and ravelling variables of each class together before guillotining at 16K. This still leaves us with the reshape, but does away with the execute on numerics. AFM now gives us about 30% better CPU cost (including time spent by the AP outside APL) and marginally slower elapsed times than we had before. We feel this is acceptable, if not quite up to our initial expectations.

I am more worried about that archaic ex-CMS system I mentioned at the start. Here the files have between 5,000 and 10,000 components, mostly quite small. Typically the application will tie half a dozen of these on signon, reading the odd record off each; this takes a minimum of 6 seconds elapsed, which is barely acceptable. We are too far down the road to start compromising on the design, but with hindsight I wonder if the decision to treat AFM as if it were 'native' might have been a mistake.

Summary

AFM gives plain old VS APL what it should have had all along; a usable file system. It is well supported with housekeeping utilities and in general the documentation is good. We have yet to find any bugs, and we had good 'hotline' support from the States when we confused ourselves during the TSO conversion. Performance is very adequate as long as you minimize interaction with the AP; feed it lots of trivial requests and response times are much less satisfactory. You can design round this, but you shouldn't really need to! On balance a good product and cheap at the price.

Benchmarks Updated

by Adrian Smith

Introduction

VECTOR (Vol. 1, No. 4) included some highly unrepresentative benchmarks of various APLs on the IBM PC and the MicroAPL Sage. Several of these were specifically set up to sniff out oddities in the various interpreters, and are probably worse than useless in comparing the speed of the computers themselves. Having had a chance to play with Dyalog APL on the DEC MicroVAX and also on the IBM RT PC (or 6150 as it wishes to be known in this country), I thought readers might be interested in the average performance of the most basic benchmarks across all the tested machines.

Benchmark Results

The times are all relative to the standard PC running APL*PLUS/PC with the co-processor fitted. It was not fitted in the PC/AT. The 'real' and 'integer' figures are averages of the three tests shown before; the 'function' figure represents a simple 1000-looper, which I have found (Dyalog would agree with this) is probably the best test you can perform to get a good all-round picture of application speed.

Computer	Integer	Real	Function
IBM PC(8087)	1.0	1.0	1.0
IBM PC/AT	1.8	3.2	3.0
Sage (APL.68000)	2.8	3.8	3.9
DEC MicroVAX	1.9	6.1	2.6
IBM PC/RT (Dyalog APL)	3.0	5.9	5.8

The DEC MicroVAX was the fastest floating-point machine, but was otherwise rather disappointing, being slightly slower than the PC/AT at function execution. I gather from Dyalog that the C-compiler is less than clever at procedure calling, so their APL (being a nice modular beast) is badly penalised.

The IBM PC/RT was (across the board) an impressive machine; marginally slower than the MicroVAX on intensive floating-point, but way ahead of the rest on all-round performance. Running it multi-user had a barely-detectable effect on elapsed times.

Product Review

PortaAPL for the Macintosh

reviewed by Normand R. Montour

Introduction

The Apple Macintosh computer has many features which could attract the inveterate APL user: its simplicity, highly interactive concept and a somewhat "futuristic" approach to Data Processing. It was lacking the only element which could make the APLer happy: APL.

The Portable Software Company of Cambridge, Massachussets, has remedied this situation. The PortaAPL interpreter is a lean, efficient and attractive package both with regard to the Macintosh and to APL. One already sees the horrible spectre of comparison showing its benchmark on the horizon. Being of a lazy nature, I felt that the wheels which were invented by others could turn just as well for me, so I decided to use two benchmarks previously presented in the April 1985 issue of VECTOR by Adrian Smith and David Ziemann. The latter was using what have become known as the "Harris" benchmarks.

Benchmarks

In the Harris/Ziemann benchmarks, PortaAPL performed extremely well and the vendor uses those benchmarks as a demonstration/marketing tool. I do not include here the findings of Mr. Ziemann, since most readers can refer to their issue of VECTOR.

PortaAPL		
THE HARRIS BENCHMARKS		
EMPTY LINE	0	UI←(500p0 1 0 0 1)/1500
Z←+/UI	10	MI←10 10pUI
Z←×/UL	5	UL←1 0 1 1 0 0 0 1
Z←r/111MI	12	UR←UI+0.1
Z←UI×.5	1550	MR←10 10pUR
Z←JUR	15	UC←'ABCDEFGHIJKLMNPOQRSTUVWXYZ'
Z←URJUI{1201}	15	MC←26 26pUC
Z←UI{pUI}	75	
Z←-2 1+MR	6	
Z←UI{UI}	205	FIB
Z←2 1pMC	20	Z←1 1
Z←UC°. =UC	50	L←-(100)pZ+Z,+/-2+Z)/L
Z←(150)°. +150	150	
Z←URL.+UR	490	
Z←MRB10+UR	450	ALL TIMES ARE IN MILLISECONDS.
FIB	2700	
Z←UR×3.14	345	
Z←UR÷3.14	470	
Z←eUR	5650	
Z←10UR×0.1	4230	

It is only when I started to execute the benchmarks devised by Adrian Smith for the IBM PC that I found a slight performance problem in the "scan" operator:

PortaAPL		
THE SMITH BENCHMARKS		
EMPTY LINE	0	U=1000P4 5 6 0
Z←+/U+U	90	M=100 10P2.5 6.7 0 0 8 -5.7
Z←+/U[1]	14	S=1000P'THE CAT SAT ON THE MAT '
Z←U+.XU	211	S[999+16]←'ADRIAN'
Z←+/U*2	250	I=100?1000
Z←+/M	1285	B=10000P4=6
Z←U+.X.M	3890	BF=800 5P'RASHJ RHF KJDSAJ '
Z←+/S='T'	1950	FF[601;]←'EJD...'
Z←'FAT'εS	77	
Z←Sε'FAT'	1550	
Z←B~B^B	45	FN N;CT;X
Z←~/B	1160	CT←0
Z←~/B	5774800	LP←(N<CT-CT+1)←EXIT
FN 10	350	X←+/2 3P16
FN 100	2950	-LP
FN 1000	29150	EXIT:'DONE'
Z←FF^='EJD...'	925	
Z←FF^='820 5P'EJD...'	15700	
ALL TIMES ARE IN MILLISECONDS.		

I have already reported the problem to the vendor and I do not believe this problem will linger very long. The other benchmarks, however, show that PortaAPL compares quite advantageously to other APL interpreters.

APL on the Macintosh

Once past the benchmarking obsession, we can consider the real beauty of a product such as PortaAPL. The Macintosh offers some excellent user interface tools: the Mouse, pull-down menus, windowing, and a very "solid" user interface. PortaAPL makes intelligent use of all these features plus a few more, including interfacing to other Macintosh products such as MacPaint, MacWrite (through the clipboard) and QuickDraw.

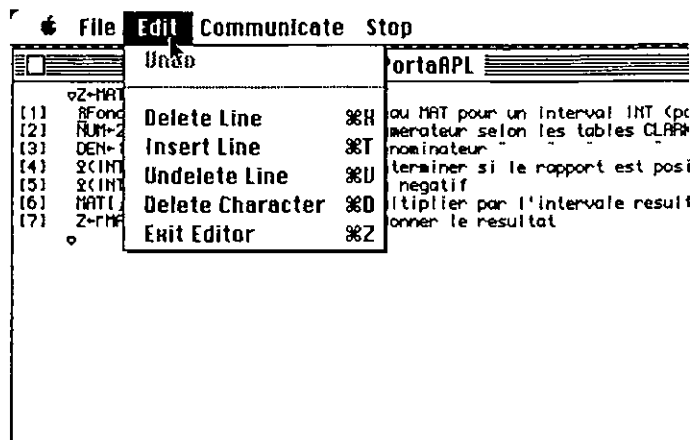
The file system is adequate, vaguely similar to the VAX/APL system (see reference 1), but with the unfortunate restriction of limiting the access method to a sequential file system. All primitives are supported including a few extensions. The PortaAPL interpreter does not offer a Nested Array facility, but this will hopefully reach the mind of the product designers. The most interesting feature of their file system is to access the Macintosh mini-finder from defined functions; this allows the user to select and open files and to choose the disks on which these files reside.

My main reason for buying a Macintosh instead of a more traditional computer of the IBM type was to relieve me of my day-to-day computerized job and to enjoy the plain and simple pleasure of a good and servicable tool to help me with writing music, composing love letters and developing my house floor-plan. I can very well see PortaAPL helping me make a musicological analysis program or getting me involved in the study of cryptography. The Macintosh is a very "visual" machine, and PortaAPL provides all the necessary functions to allow APL to take full advantage of those "visual" features.

The PortaAPL sound generation is very primitive and it does not allow any control of the advanced functions of the built-in Macintosh sound synthesizer. Polyphony is not available, nor is shaping of envelope, timbre and reverb. Other Macintosh applications already use these admirably, so it is regrettable that PortaAPL provides so little access to them.

Full-screen editor

The full-screen editor is substituted for the Del editor. It can edit functions and character arrays. The use of the Mouse and pull-down menus makes it very efficient and one can learn to live without sophisticated functions such as those provided by VM/SP's Xedit (and its popular interfaces to APL thanks to Norman Brenner – see reference 2). The twelve-fingered APLer may choose the clutch-shift-ctl-alt-esc-key editing style in lieu of the Mouse and menu operations.



The character set is crisp and clean, and no modifications are necessary to either keyboard or circuitry to obtain the proper APL characters for both display and printer (I used an Apple Imagewriter). Character arrays are edited using the  ED system function.

Usage and Feel

The PortaAPL interpreter "feels" very solid and the addition of little details such as "double-clicking" a workspace into the active workspace reveals the Macintosh dimension into the user interface. There is also a communications/terminal emulator mode which I was not able to test, but which seems to provide an adequate means of using host APL resources. The documentation is well written with plenty of examples and illustrations (all of which were drawn on the Macintosh). The supplied workspaces provide some cover functions for the most complex processes and usability functions for control of Mouse, files and external interfaces (Quickdraw, communications, etc.). Most of the other APL features are present, such as Trace and Stop, diamond, and a comprehensive set of System Variables and System Functions adapted either for the Macintosh or for the vendor's own flavour of APL. Comments are allowed on the right-hand side of the program and blank lines within functions are ignored. User-defined operators are not available.

PortaAPL also provides the ability to)EXPORT and)IMPORT workspaces to and from other systems.

Summary

Although it still contains a few bugs (which most APLers could circumvent anyway), PortaAPL is a very promising package because it opens a whole world of possibilities for the APL authors to spread their ideas in the Macintosh community, and to the Macintosh users to have access to the immense library of applications that were developed in APL during the golden years of Host Programming. I invite my fellow Iversonians to test drive this interpreter and to feed me comments. Perhaps we could even start a PortaAPL special interest group within the British APL Association. Anyone interested can contact me at:

Informaticien, 250 St. Clair Ave. W. 206, Toronto, Ontario, Canada, M4V 1R6.

PortaAPL is available from Portable Software, 60 Aberdeen Ave., Cambridge, Mass. 02138, USA.

References.

1. *VAX-11 APL Reference Manual*, DEC, Maynard, Mass., December 1982.
2. *Editing APL objects using CMS XEDIT*, by Norman Brenner, ACM APL84 Conference Proceedings, June 1984.



- SPECIAL OFFER for
APL '86
- Full Set of PLOTTING
PROGRAMS
- 2D & 3D GRAPHICAL
PRESENTATION of DATA
- SUPPORTS Screens,
Printers & Plotters
- EASY TO USE
- ONLY **£30.00** (inc. VAT)

MTS MetaTechnics Systems Ltd
Unit 216, 62 Triton Road,
London SE21 8DE, 01-670 7959
The Systems Builders

Experienced APL Consultants Required

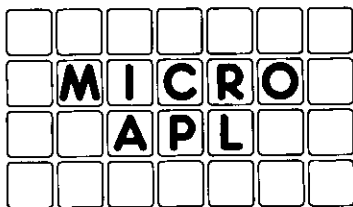
Up to £16,000 + benefits

MicroAPL require APL consultants with a minimum of 3 years' experience, preferably including user-support, to work on project development and client training and support.

Applicants must be self-motivated and able to act on their own initiative. The work will be on a variety of projects, the emphasis being on city-based applications.

An excellent salary is offered, together with free health insurance, a pension scheme and 25 days' annual holiday.

If interested, please apply in writing, enclosing a C.V.,
to Denise Sullivan,
MicroAPL Limited, Unit 1F, Nine Elms Industrial Estate,
87 Kirtling Street, London SW8 5BP. Telephone 01-622 0395.



Consultancy with Confidence

PCs, Graphics & APL

by Martyn Adams

One of the more enjoyable aspects of my job is to evaluate new pieces of kit and, if they check out O.K., write code for them. Recently I've been devoting my attentions to two such pieces of kit – both from IBM – the E.G.A. (Enhanced Graphics Adapter) and the P.G.C. (Professional Graphics Controller – also known as the PGS). Both have been around since last Summer, but the lack of availability will probably have prevented many readers from laying their hands on one.

IBM Enhanced Graphics Adapter

Now we at Metapraxis use APL*PLUS/PC on IBM PC/XTs and ATs and on the whole have been more than satisfied with it. (I have reservations about the choice of some ☐ functions and which way their arguments go and other niggly things like that but all in all I am pretty impressed.)

It was then with great delight that we received both the E.G.A. system and APL*PLUS release 4.6. (This was before the new release 5 but it supported the E.G.A.) We plugged it all in and hey presto! It worked first time! Now the E.G.A. supports many different modes – most of them the boring emulations of the basic IBM colour graphics card. These, as any discerning PC user will know, are little short of pathetic. The low resolution and poor choice of colours means that the basic colour card has few serious graphics applications. But the E.G.A. is a delight!

Our software package – RESOLVE – is designed to display very good quality colour images, graphs and such, to senior management. Consequently we are a bit fussy about the graphics devices we consider supporting and we had deliberately not used the standard colour card. By purchasing the additional memory expansion for the E.G.A., we were able to use the highest resolution possible on the E.G.A. This is obtained from APL by using mode 6 (APL graphics mode, not the E.G.A. graphics mode) which gives the 16-colour 640 by 350 pixel image. Once you've started using this, you'll never want to hear or speak of the colour graphics card again. Consequently that is the mode with which I am most familiar.

Having sung the praises of the APL interpreter and the E.G.A. I would like now to point out some practical considerations. Back at base, deep in the steamy Metapraxis jungle (our oversized office plants are rumoured to be carnivorous) we run the E.G.A. on a PC/AT. Speed, we thought, would be no problem with the extra horsepower of the AT as against our older XT's.

We used to have a performance problem on the XT with the primitive colour graphics card. It took several seconds just to write a line of text while in graphics mode! The problem seems to lie within ☐ GWRITE which is the function used to write text to the graphics screen. But the E.G.A. on the AT does not improve the speed of the text writing by much – although for most operations it is significantly faster. I think this is simply because there are more pixels to write to on the screen and so the extra speed of the AT is being traded for extra resolution on the card.

Oh well! We do get good graphics even if it is a mite slow. One advantage with the APL is that it takes advantage of the extra resolution and allows you to write 80 columns of text in full colour. The characters are fully programmable from within the APL but it is not a simple process. I have never used STSC's supplied functions so I cannot say if they supply a character design utility.

With release 4.6 of the APL (I am not sure but I think it is also true of release 5) we were supplied with a little program to define the characters to be used by the E.G.A. card. You see the E.G.A. has a fully programmable character set and STSC thoughtfully supplied a routine to program it. This means that you don't need the STSC APL character chip if you've got the E.G.A., even if you don't use the graphics options. I don't know what effect this has on any of the other programs that are lying around which use the PC's extended character set though. One could expect there to be a few funny characters lying around especially if you use a multilingual word processor. All of those horrid foreign characters are redefined as lovely APL symbols. To top it all the character quality, whether within APL, graphics or not, is very good and note that the E.G.A. still remains programmed even when in DOS.

Well done STSC on that one, but not so well done on the printing side! I have found a bug, or is it a feature? STSC have quite rightly put in a very nice and simple graphics printing function (`□ GPRINT`) which takes a snapshot of the displayed image and churns it out to the EPSON FX-100 dot matrix printer. You can decide what character represents what colour as a left argument to the function. This means that you can define black on white or white on black, by judicious selection of the appropriate characters from the STSC modified extended ASCII/APL character set.

I liked that. But the printing is rotated sideways on the printer, so to rotate it back to the correct way the manual recommends `□ POKE`'ing location 234. On simple black and white images this works beautifully – printing a very crisp image which really looks quite impressive. What the manual doesn't tell you is that your choice of shading patterns goes all funny and if you select anything other than black and white (black is a block character and white is a space character) then your picture will look as if it has put on a set of ghastly striped pyjamas.

This caused me much upset, but at the time of writing the Cocking and Drury Helpline were chasing up the problem with STSC and I am awaiting the answer with anticipation. It does seem a shame that such a nice and simple facility which has been well implemented should have this problem. In fact the other bug I mentioned to C&D was chased up and fixed pretty much straight away. I was very impressed! The bug is simple enough, if you `□ CR` or `□ VR` a function then the interpreter zaps the screen attribute character where the cursor lies. You don't normally notice it though because it sets the attribute to 'normal' display. It only shows up when you have a screen full of other attributes and then suddenly one of them switches off. However, there is a 'patch' available from C&D and later releases of APL*PLUS/PC should include that minor correction.

On the graphics side the line drawing, shading and circles all work amazingly well, although one has to be careful with the painting (colour fill). Many a time I have looked with utter dismay as the colour flooded out of the pretty little shape on the screen to obliterate everything else. Make sure your border's fully enclosed before using the `□ GPAINT` function.

A good alternative option would be to have a graphics function which allows one to specify an arbitrary shape by defining the corners and just say "fill it". It would make life so much easier. I have often needed to define a shape and fill it without knowing exactly where to start the painting. If you think about it it is not an easy task. What if the shape folds back on itself? What if there is a hole in the shape where the centre of gravity should be? Where do you start painting from? Perhaps STSC would like to consider that as a future development.

The `□ GINIT` function allows you to define the colours you want. All in all you can define up to 16 from a palette of 64. The 64 are not awe-inspiring and some of them look remarkably similar, but if you fancy 8 different shades of blue then you can do that. Actually I found that the choice of 16 colours was about right for nearly all of my applications and for most systems it should be enough. Using the `□ GINIT` function is very easy and you can even define the border colour – a thoughtful touch that.

One of the few nice things about the standard colour card was that after drawing your grotty picture you could `□ PEEK` it off the screen and compress the resultant variable, so that it held only the addresses and values where the screen was non-blank. By `□ POKE`'ing it back to the screen the image took just a second to recreate itself. If that variable was sorted by address or colour value then spectacular results could be achieved when it was written back to the screen again.

Unfortunately life is not so simple on the E.G.A. One can't simply peek and poke to the screen and get sensible results any more. At least we haven't found an easy way of doing it yet. Do you know of any way of doing it? It is a shame though, perhaps STSC could come up with a `□ GPEEK` or something which allows a quick read of any supported screen? Please!

Anyway, to compound the problem, the E.G.A. with 128k memory spills over the 64k segment size which was so thoughtfully built into the IBM PC architecture, and a fullish picture would more than likely take up more than 64k. There is no way one can fit a pint into a half-pint bottle. Or is there, STSC?

The Enhanced Colour Display – the special monitor that plugs into the E.G.A. (the older colour ones can't operate at a fast enough frequency) – has one really dumb feature. On the right of the screen are three knobs; one is the on/off, the middle one is the contrast and the third is the brilliance. Great. Except that when I first got the machine the contrast knob did not appear to work. Oh well, you can't win them all – I thought.

It turns out however that to activate the contrast knob you have to pull it out. Then it works quite well. As soon as you push it in again, the contrast returns to its factory default setting and you have to pull it out again to return to your desired setting. I think that's dumb. Why bother with a pull switch at all? And if you're going to have one, why not document it? Such are the wonders of modern technology I suppose. I wonder if the next screen will have an on/off switch which only works if you pull it out first. . .

To finish up on the E.G.A., I would have to recommend it as a serious graphics card which is more than I can say for the previous IBM offering. If performance is what you want, then you'll have to have an AT as the XT can be a bit slow. I can guarantee that you will be impressed with the picture quality of the device. I hope that the current rumours about IBM bundling the E.G.A. in with the new ATs as the standard monitor card turn out to be true.

By the way, a flood of E.G.A. compatibles seem to be hitting the market and we've successfully run all our software on one of these (Quadram) without having to make a single change.

IBM Professional Graphics Controller

The P.G.C. (Professional Graphics Controller) is a very different animal. IBM started selling them at very high cost because they are very high spec. However earlier this year the prices were slashed by about 50%. Don't get too excited about this though - we haven't been able to lay our hands on one in the U.K. since last Autumn, although they have been available to our knowledge in the States and in France.

The P.G.C. has a resolution of 640 by 480 and can display 256 colours from a palette of 4096. Not only that, but it has its own onboard 8088 processor! Unfortunately you can't speak to it by using the $\square$ Gfunctions in APL*PLUS, you have to send it commands.

I remember the days when my finances ran to an APPLE II. Remember them? I later upgraded it with a keyboard that had its own processor but technology had advanced so fast since I had bought the APPLE that the processor in the keyboard was more powerful than the CPU! It nearly had more memory too! I seriously considered programming the keyboard to be an interpreter for the CPU. Pipelining on the APPLE? No. I'm not that good unfortunately. But I digress. . .

The P.G.C. is one of the products supported by IBM's version of GKS called VDI. To translate into English that means that Big Blue wrote a packaged offering called Virtual Device Interface which emulates the Graphics Kernel System for its PC range of computers. That should, in theory, permit pictures drawn on one device to be transportable to another. In practice, as I have explained in a previous article (see VECTOR Vol. 2, No. 3), that doesn't work very well.

Virtual Device? I have visions of a printer zipping about the office on its little wheels, wires dangling, plugging itself into each PC workstation and accepting a line of printed output from each. Providing it doesn't bump into the virtual graphics screen (a P.G.C. perhaps) also skidding about trying to display at least one frame of a graphics image from each machine – it might work. Until the manager walks in and gets a flying RS232 in the mouth.

In the future, despite my previous article in VECTOR on the uselessness of GKS, it may well be worthwhile to write a VDI/APL interface, as VDI currently supports the P.G.C., E.G.A. and that other blotchy thing. For simple drawings it may prove a useful transporting tool but don't get carried away. Apart from the difficulty of defining pictures using VDI it also has a serious performance overhead. Oh roll on the 80386!

Rumour has it (lots of rumours around these days!) that the P.G.C. VDI driver doesn't quite work properly and that's why you can't lay your hands on a P.G.C. in the U.K. (*Ed: It must be something to do with the weather if it works in Silicon Valley and on the Riviera!*) Apparently IBM do not think there is much demand for a new graphics screen which uses a different protocol (yes – another graphics protocol) without having the VDI programs.

Anyway we – the amazing APLers – do not need the new proposed Esperanto of the graphics world in order to get the P.G.C. working. Besides which, unlike real Esperanto, VDI can slow down your application by as much as three times – rumour has it of course. I tried to get VDI into the APL workspace but VDI didn't cooperate and I'd be pleased to know if anyone else has any success in this field.

The P.G.C. is the biggest PC card I've come across; it takes two complete expansion slots and there is a little half card stuck between them. It certainly feels like you are getting your money's worth when you install it. In PC mode the P.G.C. emulates a feeble graphics card, which is not the E.G.A. but that older one, and it will only switch to its high-res mode by a specific command. The graphics commands are sent to it by accessing a memory buffer in the PC memory map and setting pointers. Unlike the E.G.A., the P.G.C. has a command processor on board and so it has to be sent a series of command strings. For this reason APL*PLUS doesn't yet support the P.G.C. with its $\square$ functions. I have written a function to send the command strings and if anybody wants a copy of my code (unsupported of course), just ask.

Using my function we have been able to draw incredibly pretty pictures with incredible resolution and in incredibly quick times. The P.G.C. does support arbitrary polygon fills and you can dynamically change colour palettes enabling you to emulate rotating planets and flashing arrows and such like. There is a suite of 3-D graphics functions, but be warned – they do not always work properly. You can rotate defined objects through a three-dimensional plane and end up with a really screwy picture. The P.G.C. doesn't support hidden line/surface removal. Pity that. . .

It will draw in 2D, 3D, convert from one to the other, circles, lines, polygons etc. You can even draw with absolute coordinates or relative coordinates. You can define command lists and run them whenever you want. You call them back in a translated form and you can read pixels from the screen as well as write them. You can program variable sized characters as well and the amazing thing about the P.G.C. is that it *could* look remarkably like an IBM 3279 graphics terminal if it wants too.

The P.G.C. is a very good device but is not for the casual user unless you really require the resolution or the greater colour selection. Development of programs can have its difficulties. Remember that the P.G.C. is a computer in its own right so that if you lose communications with it and the monitor is displaying the P.G.C. screen – not the PC screen – then tough! You can quite happily display a graphics image and while doing so return to APL with a SYNTAX ERROR and do frightening things, without the P.G.C. batting an eyelid. It still hasn't been told to return to PC mode and so will quite happily display its graph without you knowing what is happening under the bonnet. Even a soft re-boot won't help and you may have to switch the PC off and then on again before you can continue. Remember that rebooting the PC will restart the PC but not the P.G.C. IBM have thoughtfully not supplied a reset button on the card.

On the other hand you can draw a graphics image while still in PC mode and then suddenly flash to the graphics image without the user having to watch the graph being built up.

A big let down with the P.G.C. was the proportional spaced lettering which is supposedly able to cope with any character size. I don't know why I was led to believe that, it certainly isn't explicitly stated anywhere. With all those CPU cycles to spare you would have thought that the P.G.C. could draw properly proportional characters at any size anywhere on the screen – but it doesn't. As you increase the character size the spacing within the character string gets bigger until the P.G.C. decides that it can fit a double size character in. Then *bang!* – the characters are twice as large and closer together.

In conclusion then, I have to say that overall I am impressed with the E.G.A. (especially with the extra memory) and APL*PLUS's integration of it into their offering, even to the point that they support the extra memory without the manual saying so! STSC could do a little more in the way they handle graphics from APL and I would be only too pleased to make a few constructive suggestions.

VDI does not seem so impressive. The P.G.C. is very good for those who need the extra specification but remember that it didn't come cheap (and now it doesn't seem to come at all!). I suspect that there are already a few P.G.C. clones around that support hidden-line removal and one or two little extras which will make them superb for animated graphics.

RECENT MEETINGS

This section of VECTOR is intended to document the seminars delivered at recent meetings of the Association, particularly for the benefit of those members based away from London who often find it hard to find the time to attend. It also covers other selected events which are likely to be of interest to the wider APL community.

We are dependent on the willingness of speakers to provide us with a written version of their talk, and we would remind them the "a picture's worth a thousand words". Copies of slides and transparencies will enhance their articles.

The Activities Officer (see inside back cover) will respond enthusiastically to offers from individuals who wish to contribute seminars and supporting papers.



H.M.W. PROGRAMMING CONSULTANTS LTD

Why not discover more about

- ★ **Consultancy/Support Service**
- ★ **APL on the IBM PC**
- ★ **VM/CMS Packages**

Ring Ken Jackson at

H.M.W. PROGRAMMING CONSULTANTS LTD

142 FELTHAM HILL ROAD,
ASHFORD, MIDDLESEX TW15 1HN
Telephone: Ashford 41232

Introductory Notes

by Adrian Smith

1986 got off to a cracking start with two very well supported meetings at the Royal Overseas League. On 17th Jan we had a batch of papers on APL and OR, and on 21st March a set of talks on APL system design.

Much of the detail from the talks will be available in full as VECTOR articles (Tony Cooper's is included after these notes – many thanks) so I shall be very brief in giving little more than an outline of the papers.

Many thanks to Nic Cook for taking very thorough notes at the March meeting (and my apologies for not using more of them!).

APL and Operational Research

January 17th 1986

A Computing Environment for OR

Tony Cooper (British Airways)

Tony is a member of a small group in BA who support OR (and others) on APL*PLUS (mainframe), DEC-10 and so on. His talk covered the themes of:

- ★ what aspects of computing are helpful to OR.
- ★ how well does BA measure up?

He concluded with the interesting quote:

"All the really undisciplined people have congregated in the groups that do APL."

I wonder why . . . see Romilly Cocking's paper from the March meeting.

Optimum Pallet Loading

Eileen Dyson (Rowntree Mackintosh)

The evolution of a pallet-loading model at Rowntree Mackintosh fell into three phases:

- ★ 1976 (or thereabouts) saw the design of a 'best-fit' algorithm which was implemented in batch PL/1. The output from this was a 'layout chart' giving a complete packing schema for a particular pallet size.
- ★ 1983 saw an APL front-end, which used GDDM graphics to select and display the choice of layouts for any given box size. Using this the product development department could (for example) superimpose two candidate patterns to check interlocking and hence stability.
- ★ 1985, and the APL package moved down to APL*PLUS/PC for distribution to several factory sites. The graphics lacks the flair of GDDM, but is entirely adequate in day to day use.

Eileen brought along a disk of the model, which attracted an interested gaggle of people when run on a nearby Compaq.

Linear Programming and APL: Pains and Pleasures

Phil Chastney

APL effectively grew out of linear algebra; ergo it ought to be ideal for tasks like LP! Unfortunately LP is intrinsically procedural and highly iterative; FORTRAN programs may run for 8 hours . . . what can APL hope to offer?

It is a nice environment in which to *use* LP, and it does have all the right primitives. In fact the whole basis of LP is two inner products, a division, and an outer product. The problems lie in the implementation of the typical matrix operations when the data is very sparse (1 – 10%). This leaves a niche for APL in relatively small problems, where the answer is wanted interactively, and where the user is willing to pay for the privilege.

Pouring Water on Troubled Oils

Dominic Murphy (ex Texaco)

This was a typical Dominic half-hour! Ostensibly it covered the ‘human element’ in the design and implementation of a refinery simulation. As he put it, the motivation went something like this:

UK to USA: We need some more tanks

USA to UK: No you don’t

UK to USA: Oh yes we do

USA to UK: Oh no you don’t . . . we’ve done a simulation!!

UK to OR : We need a simulation – and make it snappy!

The whole thing ended up as a vast board game; it cost some phenomenal amount of money to run on IPSA, as everyone collected their printouts and flung them off to the States.

Unfortunately, after all this effort, all they proved was that they really didn’t need the tanks after all. Oh dear, they never spoke to OR again!

Computing Environments for OR

by Tony Cooper

This talk was an attempt to characterise what makes a computing environment for OR effective, followed by an assessment of how APL measures up when compared with a number of alternatives. It is based on 15 years of experience with British Airways' large O.R. group, which has been working with APL for four years.

The main point is to stress that it is the whole environment that influences whether O.R.'s computing is productive and whether high-quality, maintainable, developable programs, that do what the user wants, get written. This environment includes the language(s) used, editors, the operating system, hardware, and in general everything that contributes to making the computer a useful tool that is fun to use rather than a source of problems.

The choice of computery often depends on such factors as what's already in use, what we can afford and whether I.B.M. make it. I'm ignoring all this to concentrate on what is most effective at the job of using computers to solve complex airline problems. I identify five main areas and a number of sub-headings. Many of these should need no elaboration – and there is neither the time nor the space to give any – but these are not necessarily the less important ones. Here is the list:

Interactiveness:

The more interactive the better: terminals, workspaces, interpreters, . . .

Environmental and External:

Availability.

Response times.

Control: who decides what to buy and how to use it?

Portability and Upgrade Paths.

Diseconomies of scale: it costs £10,000 per annum for BA to provide a 3279 terminal on an IBM-compatible NAS mainframe, and £4,000 for a similar terminal on a 10-user mini.

The Padded Cell: users don't want other users to slow them down or get in the way.

Communications: but they need data from each other in all directions.

General Design Principles:

Re-usable software: the potential cost savings are enormous.

User-friendliness.

Orthogonality: a minimal number of independent ideas or features that work well together.

Clarity.

Simplicity.

Learnability: when users are intelligent and mobile, enable them to learn generally useful techniques from individual applications. (This was a main theme of Paul Berry's plenary presentation at APL83.)

Technical issues:

- Power.
- Up-to-date devices.
- Editors.
- Text- or word-processing.
- More than one thing at once.
- High-level debuggers.
- Windows.

Software availability:

- End-user products: spreadsheets, word-processing, etc.
- Development aids: e.g. code control systems.
- Uniform interfaces: all this software, purchased and written, needs to be interfaced together. Adoption of standards such as IGES, GKS, etc., helps; you also need the sort of glue provided by Unix pipes.

These requirements lead in two directions that often conflict: firstly a desire for a perfect system, and secondly a desire for an open evolving system, with access to the thousands of useful programs that others have written and will write, and security that one's investment in writing code will be preserved. The first drives you towards a **Language System**, an integrated operating system, language, debugger, editor, etc. Perhaps the best-known of these is Smalltalk, but Ada in an Apse has similar aspirations, as do several A.I. environments such as Poplog. Even BASIC achieves it for very small-scale problems. APL also falls more or less into this category. The second direction leaves you many fewer options: I would say that PC DOS/MSDOS, Unix, VAX/VMS and IBM-compatible mainframes constitute an exhaustive list of sufficiently open operating systems.

How does APL measure up against this long list of criteria? Moderately. Much better than IMS, say, or than PL/I under CMS; but no better a suitable Unix- or PC DOS-based toolkit. The strengths of APL are well-known to VECTOR readers, so I merely present a personal list of those that seem most important:

- ★ easy to learn initially;
- ★ fast development time for small problems;
- ★ well standardised (until spoilt by nested arrays);
- ★ generates enthusiasm amongst programmers;
- ★ structure and brevity helps create clear, transparent code;
- ★ workspace environment with very good debugging.

The weaknesses are serious, however, and many seem to get disregarded by the APL community:

- ★ programming in APL requires continual mini-inspirations. This makes it fun, but also hard to write well, and often unclear at the detailed level. Consider the idiom for deleting trailing spaces; it is unnatural and obscure compared with an iterative loop that repetitively looks at the last character and deletes it if it's a space.

- ★ consequently, APL programmers are very expensive to recruit (at BA we've never managed to hire one), and many existing programmers loathe it.
- ★ non-standard hardware: we can't use 9800 of our current stock of 10000 terminals.
- ★ the workspace orientation makes it very hard to share code across a group of programmers.
- ★ the widely-felt need to try out code to see whether or how it will work militates against a disciplined approach to program quality and the use of structured methodologies.
- ★ APL is very expensive on CPU time; BA has two 10-mips class mainframes dedicated to APL and a semi-permanent task-force investigating why they give such poor response times.

It is this last that is the main reason for two or three medium-sized disasters at BA, and the main reason why OR in BA is moving away from APL.

The talk concluded with a comparison of APL with C/Unix and Focus/PCDOS in respect of the effectiveness factors listed above. This was meant to highlight the different strengths and weaknesses of the three alternatives and generated a good deal of animated discussion.

Technical Meeting: Managing APLers

21st March 1986

An APL Banking Application

Per Hultin (APL Software Technology)

This talk concentrated on the value of a relational approach to data, and of end-user involvement in the system design. The banking system he outlined was (in any terms) big:

- ★ 8,000 accounts, with ½ million transactions per quarter
- ★ 967,000 records in the database
- ★ 31 files/relations with 258 attributes
- ★ 300 Mbytes of storage.

To some extent the development was trapped in APL because the volumes quoted were much lower! Once committed it was cheaper to buy a bigger CPU than to rewrite in COBOL!

Exercise: "ANZAC Provident Loans"

Nic's comments (unexpurgated) were:

"Quite good fun, got people discussing things – but some cliques didn't want to play!"

I don't know who thought this up, but it was obviously a good idea. Issues like profit potential, data accuracy, machine capacity, timescales and division of work were well brought out. Should you prototype? Without one you can't get off the ground, but it could lead you into an oversimplified design.

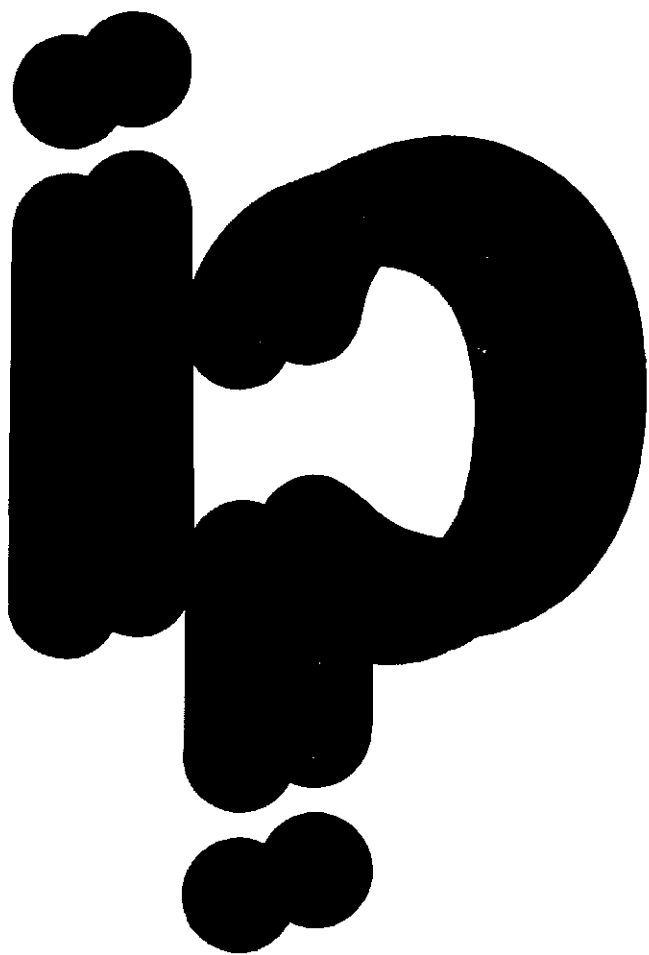
APL System Design

Romilly Cocking (Cocking and Drury)

This was basically a justification for, and potted version of, the C&D System Design course. I hope Romilly can be talked into writing this up in full; there is far too much material for me to have any hope of doing it justice here.

"Just telling us again to do what we know we should do, but usually don't."

... was Nic's parting shot on the day.



**THE SOLUTION TO
YOUR APL QUESTIONS**
on IBM PC or Mainframe.
INNER PRODUCT

For further information, contact JAMES MANNING, Marketing Manager, Inner Product, Eagle House, 73 Clapham Common Southside, London SW4 9DG, or telephone 01-673-3354.

GENERAL ARTICLES

This section of VECTOR is oriented towards readers who may neither know APL, nor be interested in learning it. However we hope you are curious about how, under the right conditions, such impressive results can emerge so quickly from APL programmers.

Case Study

by Adrian Smith

Author's Note

As promised, here is part-2 of the VSPC conversion saga. Much of what follows is lifted straight from my Email diary; a potentially fascinating archive for the software archaeologist of 4086!

If there is a moral to be drawn from this tale, it is probably "Panic sooner rather than later". In general a problem perceived was a problem solved; once we had wound ourselves up to face the fact of TSO the actual conversion rapidly became a well-drilled routine.

The 'wake' was scheduled for April 1st at 8.00pm; some of us made it with rather less than 2 hours to spare!!

VSPC : The Last Rites

by Adrian Smith

```
IEC161I 056-084, TX02001, IEFPROG, OPTION,,, SYS3.ISPF.OPTIONS.CLUST
IEC161I SYS3.ISPF.OPTIONS.DATA, ICFUCAT3
IEC161I 056-084, TX02001, IEFPROG, OPTION,,, SYS3.ISPF.OPTIONS.CLUST
IEC161I SYS3.ISPF.OPTIONS.INDEX, ICFUCAT3
IEC161I 062-086, TX02001, IEFPROG, OPTION,,, SYS3.ISPF.OPTIONS.CLUST
IEC161I SYS3.ISPF.OPTIONS.DATA, ICFUCAT3
```

To recap briefly I left my tale of woe suspended in mid-air towards the end of September 1985. The things that were really bugging me at the time were:

- ★ the TSO environment, in particular the session manager, and the way workspaces were saved and (?) shared.
- ★ VSAM record-locking. Could VSAM files be held at the record-level rather than by reserving the whole file?
- ★ the sheer bulk of data to copy across and check through.
- ★ the persistent rumours of heavy increases in CPU cost.

In no particular order, this is what we did . . .

VSAM Record Locking

The first problem to be tackled was the knotty one of multi-user VSAM files. We depend absolutely on these for all our APL printing; a routine called VTAMPRT scans constantly through a shared VSAM file, pulling off completed prints and dishing them out to the appropriate destinations. Typically up to a dozen users could be spooling prints concurrently to the file, so to try to work with file-level holds would be quite impossible.

Diary: 24/10/85

"Current state of play on VTAMPRT:

As feared the VSAM record locking does not hold up under TSO. We really do need shareoptions 3/4 as otherwise every user locks the whole file while they update. Ergo there are two lines of attack:

1. Andy writes an assembler routine to book the record for us. This could use shareoptions-4 and then ENQ appropriately. I suspect that we would need a product called CALL/AP (from the same stable as AFM) to do this.
2. Having booked the frame we then read it back to check that it hasn't been double-booked. This adds a bit to the VSAM I/O, but is hardly significant as Andy is increasing the record length to 22K. Most prints (168 by 20 = 3360 lines) will fit on one frame anyway.

The only worry is that each user will simply re-read a buffer, rather than the real record! As soon as Andy has set up a file I suggest we check this. The only way round might be to close/re-open the file . . . yuk!"

Diary: 28/10/85

"More on TSO and VTAMPRT . . .

It appears that with shareoptions-4 we can re-read the control record to check that we really have grabbed it. DJW & I did a quick check of the protocol on Friday, and it behaved as expected (as did shr-3 which simply re-read the buffer and is therefore quite useless). Essentially this is a 'last-in first-served' queue; hardly fair but quite effective!

I am now recoding a version of the APL side to fit in with this new protocol, and also to make use of the 22K record length (168 lines of print per record).

To do this efficiently I feel I need to change the way print data is buffered in APL (catenating to the end of a buffer this big could get very slow . . . the cost goes up as the square of the object size). I shall experiment with some alternatives and do a few timings . . . we should be able to cut a fair whack off the CPU cost of printing (as well as chopping the VSAM overhead by about 3/4) if we take a bit of care over the design."

Diary: 29/10/85

"I think that we are home and dry with TSO/VTAMPRT.

Andy has written a little assembler command which books a frame number and returns the frame number it booked. This is able to queue sensibly on the records, and there is no possibility of a double-booking. It also simplifies the APL routines and seems rather faster. All the APL side now does is generate a random slot number (1 - 99) as the starting point for a sequential search for a free slot.

I don't think that this is likely to lead to any more searching than the present strategy of trying all 99 slots at random. Apart from this very minor change the new routine is functionally equivalent to the old one. I shall now try some comparative timings of the two systems. I don't expect to see any significant difference."

Diary: 4/11/85

"The final timings on the new VTAMPRT for TSO suggest a reduction in CPU (as recorded by APL) to approx 70% of what it was. This is largely due to the 22K blocking, and only really shows up for files of 2000 lines or so. However the new booking arrangement does mean that even for small files (1 line) the new system is ahead (92% of the old).

I think we can safely put this one to bed."

As you can see, the final solution (found after several blind alleys had been explored) was simple and effective. Because we could 'book' a slot on the file with a simple TSO command (the numeric return code suffices to get back the slot number) we could get away with AP100 as the assembler interface. Of course it was much easier for APL to generate the random starting point, so this is passed as a (formatted) number, along with the user number, date and time.

As an aside, I got caught out by the behaviour of $\square$ RL when you start with too small a seed. Due to an attack of sloppiness I set the seed to $\square$ TS[5 6] encoded to base 100. If you generate ?99 from this the maximum number you get varies from 5 to 48 depending on the number of minutes past the hour! Consequently users were getting dreadful response to print requests just after the hour, because the booking routine always started to search from the very low end of the file. Slots above 49 were never used, which is why the problem soon became painfully obvious!!

Workspaces and Files

One little note in the TSO-APL manual brought on a bad attack of fear. It pointed out that a system problem in the middle of a)SAVE would not only lose your immediate changes, but would by then have wiped the previous copy! This looked like a sure-fire way to lose a day's work, and as true believers in Murphy's Law we felt that we had to do something about it. The obvious line of attack was to use 'Partitioned Datasets' (PDSs), rather than simple sequential files, for workspace saving. These preserve the old version during a)SAVE, but have a nasty habit of filling up with old copies and needing compressing just when you don't want to.

Once again an angel of mercy came to the rescue, in the unlikely guise of PDSMAN. This little wonder will reuse old slots and generally attempt to keep things tidy; it can't always make full use of the space available, but at least the need for regular compresses is avoided.

Once we had bitten the bullet and patched APL to use our PDS libraries we began to think round our 'ideal world' solution to the whole area of project libraries and data sharing. What we needed was some set of conventions which would protect private data, but allow easy read access for crash recovery, and controlled write access for routine maintenance. In general we wanted to get away from our VSPC habit of logging on to other people's IDs for routine updating.

The structure we settled on gave each TSO user two APL libraries:

```
APL.GLOBAL.AW002001    . . . shared data for user 2001
APL.PRIVATE.AW002001   . . . private data
```

In addition there are any number of 'Project Libraries':

```
APL.SHARED.AW100012    . . . updated by anyone in (say) OR
```

. . . and of course public libraries which are updated only by a select band of initiates:

```
APL.PUBLIC.AW000020     . . . Adrian's utilities on )LIB 20
```

From an APL point of view, this works quite nicely once you get accustomed to the distinction between 'LIB' and 'LIB 2001'; obviously you need to remember 'WSID 2001 MYWS' if you want the fruits of your labours to be generally available. However it is a bonus over VSPC to have the command 'LIB 2012' return the contents of someone else's global library; again this saves one from logging on to the owner's Id to find out what he called things! For the first time, we really don't need to know each other's passwords . . . this must be a good thing in the long term.

The APL Session

I felt that the VSPC Session Manager had been written by someone who used session managers; the TSO equivalent gives quite the opposite impression. It has that unwieldy feel which is so characteristic of 'design from the outside'; it just doesn't come comfortably to hand.

The only real bonus is the facility to queue a bunch of lines for re-execution. Particularly during the 'peak' of our conversion activity I found I was constantly going through 'LOAD this; COPY that; WSID the other; SAVE' routines; the session manager was very helpful in automating a lot of this.

Now for the gripes . . . the colours you can mitigate (by setting 'Highlight Off') or improve (by sneaking on to page 1001 and resetting them) to be quite acceptable. The behaviour on <CLEAR> still seems inexcusable. To be faced with the rigmarole of <RESET> <PA2> and <ENTER> to recover from a perfectly reasonable assumption (to clear the screen . . . press the key labelled 'CLEAR') is absurd. So far I have failed to find any way round this; closing and re-opening the GDDM device with a different 'clear/PA1 protocol' leaves you unable to interrupt APL loops. Has anyone else got any bright ideas??

Developing lines of APL 'at the terminal' is far less convenient under TSO. VSPC had a very handy <Last> key which pulled back whatever you just did, for possible modification and re-execution (APL*PLUS/PC has the same thing). Under TSO you have to go and find the damn thing (it might be several screens back) or just give up and re-type it. Oddly enough you often want to repeat a line without changing it at all; to be forced into overtyping (say) blank with blank to get the line back . . . well what can I say??

In summary the session screen would be improved out of all recognition with two extra facilities:

- * a 'last command' facility available via SM command or PFkey.
- * a 'mark' key to re-execute a line without the nonsensical modification.

IBM, are you listening?

The Conversion

TSO does have some nice things, one of the most useful being a sniffer-dog called 'APL WSID' which goes poking about in your workspace to find the time and date it was saved. Having discovered this, I did a bit of hacking and came up with the following:

Diary: 26/11/85

"To whom it may concern . . .

I am now in possession of a Trojan Horse routine which will copy into your workspace any utilities which are:

- ★ already there

- ★ are on the utility library with a saved date after the date when the workspace itself was saved.

I suggest we adopt this across the board as the latent exec, and will thus be saved all the hassle of recopying changed utilities under TSO"

This simple notion had the enormous benefit that we could 'preset' the conversion in existing workspaces. In systems where we needed to move around 20 users 'simultaneously' we just wouldn't have had time to load up all the workspaces, make the changes and resave, in one weekend. All we now needed to do was (at leisure) to patch in AUTOEXEC and let the system cope with the upgrades. Wonderful. What's more it really did go alright on the night! I think we had between 3 and 5 cases where someone had been 'naughty', and had changed a standard function without renaming it. I'm afraid we also had one example of a utility which was not (quite) upward compatible; it suddenly started returning 1-element vectors instead of scalars!! These minor snags apart, everyone carried on as if nothing had happened.

I had been quite right in seeing AFM as a major saving grace; in almost every case we were able to stick to the same user-number (VSPC 32670 became TSO PX32670=AFM 32670), so all those embedded user numbers:

'18750 BUDGET' GET 'VAR1,VAR2,. . . ,etc'

. . . caused us no hassle at all! In general we had completed the AFM conversion several months before the final move to TSO, so it was literally a 5-minute batch run to copy over all the files for a set of target users. Apart from one spectacular cock-up when we accidentally deleted the entire TSO/AFM system, all went astonishingly smoothly.

Postscript

TSO is undoubtedly a more 'powerful' environment than VSPC. The easy access to batch, and the linkage to packages like 'Focus', are major improvements. It is also much less 'protective'; the occasional splurge of red messages is bad enough, however it is quite possible to (say) 'abend' the session manager or get hung up somewhere between APL and SPF; in every case your only escape is to ring up and get thrown off the system! With a lot of work, we made TSO bearable; in the long term we may well be better off, but there is a lot of lost ground to make up.

I would like to conclude with two more diary entries, both slightly facetious:

Diary: 19/02/86

"In checking through the vector and image symbol editors I have come across a slight oddity in that my sheep have mysteriously turned green! This turns out to be a minor change in the use of the GSIMG command and is now OK. As far as I can see there is no problem in exporting VSPC-created symbol sets to TSO and building your own library of vector/image symbols."

Diary: 10/03/86 (Reference VDU menu entry for TSO)

"Most of us tend to see TSOB as short for TSO - B

Most users tend to see it more in terms of T - SOB, which loosely translates as 'That Son Of a Bitch'. Is this a good idea??"

1986 (May): Concluded

"One Small Step for APL"

by David S. Moffat

It was a warm sunny August morning in 1982 when I entered Imperial House as the newest member of the Chief Accountant's Department. My role, *inter alia*, was to look after the department's newly acquired Spectrum computer, and to learn a new language – APL. I had done a little research before joining Imperial and discovered that it was a language programmers either loved or hated – and most people I spoke to hated it.

My previous experience had been gained with Micromodeller, running on an Apple, and the PPL general ledger package running on an IBM mainframe. Both are environments where the user develops his own system without programmer intervention. I had also played with ALGOL and BASIC, but a long time ago.

The Chief Accountant's Department had little experience of micro-computers and due to redecoration of its offices was temporarily relocated. For at least one of the above reasons the Spectrum had been housed in a disused photographic darkroom. The environment, whilst not affecting the operation of the machine (provided the windows weren't opened) did little to engender my enthusiasm for the new task. I was shown how to switch the machine on, most people knew how to type B to boot, but few knew how to use APL.

A week later and in one fell swoop I was introduced to the teach yourself system, provided by MicroAPL, Gilman and Rose's "APL an Interactive Approach" and Adrian Smith's book "Design Handbook for Commercial Systems", and then I was left to my own devices. These aids to learning either had a soporific effect on railway journeys or left me confused, so I reverted to John Le Carre but he couldn't help. I found, later, that the non-fiction books helped enhance existing knowledge but lacked the stimulation I needed to gain initial expertise. Therefore, I decided to embark on a series of one day courses on a child's guide on how to write the ultimate "one liner". The hieroglyphics gradually began to make sense, but I was not filled with a sense of wonder at the dawning of a new age, and I would have been quite happy if this American import could have been re-incarcerated with the Essense's Dead Sea Scrolls, in the Qumran Hills.

My new found knowledge – if that's what three days' training in a classroom environment could be called – was soon to be put to the test by building a simple system. I had two basic problems:

1. I had to replicate an outdated manual system.
2. There was no detailed specification.

A task which would now take me a day to complete, took me two weeks. The system is still running but it is unsophisticated – and it has more than one line!

The whole experience left me wondering, yet again, why had we chosen a language, which took so long to learn and was so unfriendly. This was not a unique experience as other members of my department felt likewise and resorted to pencil and paper or flocked to the PC.

It was about six months later that I began to get involved in producing a full system specification to be implemented by a trained Analyst/Programmer, whom I watched build the system, and only then did I begin to see what all the fuss was about.

During the subsequent three years I have been closely involved in specifying and building complex accounting systems, and during that time I have acquired a reasonable knowledge of the language. However, I still would not class myself as an APL programmer because I primarily hold an accounting role, the demands of which prevented specialisation in "a programming language".

I now realise what can be achieved with the language. It has the following obvious advantages:

- ★ It has a short development time for even the most complex systems.
- ★ Routines can be developed into small building blocks or functions which can be interrelated and are easy to debug.
- ★ Data is easily controlled within the workspace.

However, it does tend to have a few disadvantages:

- ★ It is so brief as to be curt.
- ★ The characters are not obvious and cannot be read by anyone but programmers and other dead races.
- ★ It requires a special keyboard and printer.

A well built APL system can give great benefits to the user. However, in the real world, systems are rarely static, the continuing changes in company structure, legal requirements and commercial demands necessitate frequent changes. APL systems are often not easy to change quickly, even for a skilled programmer, unless he is the author. APL allows each person his own style, which partially evolves from his level of competence but also from his approach to the problem.

I have also found that APLers adopt a self-documentation approach to their systems, with a line of text at the start of each function, thus avoiding tedious manuals but not satisfying auditors. Self-documentation benefits those familiar with the system but is probably inadequate for a newcomer, and hence usually results in only authors changing their systems. Because only the originator can make changes, Murphy's Law says he is usually not at hand when needed to make amendments, as his expertise is in use elsewhere. This criticism is true, to some extent, of other languages but despite their failings these are easier to learn or are less hostile, because of their use of near English commands. Novices gain a fillip from success and users like to gain familiarity with their systems but APL does not encourage them. Harassed users do not always have the time to brief and control their systems analyst, particularly late at night with an early morning deadline, and inevitably there is a desire to control your operating tools.

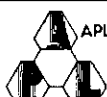
If a company has made a heavy investment in a computer system it wants to gain the full benefit from not only its main workhorse applications, but also from ad hoc analytical tools for business decision making usually called the "what-if". This need has led to the growth of the spreadsheet and PC market and a stampede for freedom from programmers, but it seems wasteful to have at least two computers, two terminals, two printers, two languages, etc, etc. The ideal solution to quote ICL would be to have "one per desk" – one machine, one language, at least within each culture. This is not a universal solution to chain everyone to only one supplier, but it can be both expensive and confusing for a non-expert to have to use two computers. Familiarity and dexterity helps speed up the decision-making, particularly with a universal tool – but that tool needs to be both flexible and easy to learn. There ought to be standardisation for the non-expert within a company or department. The guy at the sharp end probably doesn't care whether he is using APL, Basic or whatever, what he wants is to get the job done in the most efficient way, with the minimum of fuss.

There is no doubt that APL has many advantages over other languages but I believe you experts are not developing it to its potential. There could be great satisfaction and one should not also forget financial reward, in developing the ultimate solution. Could you develop the quantum leap from Lotus 123 et al. I know you are developing many bespoke systems for major customers which form building blocks with other systems. I have also seen interfaces with spreadsheets and heard suggestions about using the PC as a dumb terminal, but is anyone developing for one or two generations out, and more importantly is it likely to expand outside the small world of VECTOR?

My dictionary defines vector as "any quantity requiring direction to be stated as well as magnitude in order to define it properly". Most people in the business community have heard of Lotus, few will know of APL. Perhaps you in the APL community need more magnitude and more direction. If, as I believe, the product is a good one, you should be able to go out and sell it in the right way with easy to use systems which are well marketed. You are the prophets but have you been left in the wilderness?

If I show my ignorance of what is available, then I am probably representative of the vast majority of your potential customers, who have insufficient information about the power of APL. Don't wait for the Big Blues of this world to lead the way, expand outside the comfort of the VECTOR clique and develop more user-friendly, generalised products which can help the novice build his own routines, which should enable "what-if" modelling and may eventually interface with other bespoke systems. The development of cheap hardware has created the most receptive audience you could ask for. The keyword system developed for the QL was a good attempt to meet this demand, but it did not go far enough to simplify access to the APL environment, but it did get reviewed outside the computer press.

In writing this I have attempted to be controversial and draw together the adverse comments of others. If you are stimulated to respond please remember you should address the silent majority.



APL People Ltd.

17 Barton Street,
Bath,
Avon,
BA1 1HQ
Tel: 0225 62601

APPOINTMENTS

A placement service solely for the APL community.

Use our extensive knowledge, contacts and experience in the APL world.

We have vacancies NOW, both permanent and contract and our confidential advice on career moves is always freely available.

CONSULTANCY

A consultancy service designed to solve your APL problems.

Consultant's Toolkit and Supervision available.

Current projects include:-

- * graphics
- * project management
- * databases
- * system design
- * system 'rescue'
- * documentation
- * VSAPL under VSPC, CMS, TSO
- APL*Plus/PC, Sharp APL

To discuss your requirements: — Call Valerie Lusmore on 0225-62601

Registered employment agency licence SE6440

A Sharper Focus on APL

by Sylvia Camacho

Introduction

There is a faith and a hope, often expressed in the data processing and lay press, that it will only be a short time before software is written which will let computers be instructed in plain English, and that this will allow end users to interact directly with their machines.

This faith is mistaken and this hope is misplaced. The faith is mistaken because it will be a very long time before computers are able to interpret plain English with anything like the adequacy of a child of five and the hope is misplaced because, even were computers to have the linguistic ability of Dr. Johnson himself, plain English would be an inappropriate language in which to address them – at least for commercial purposes.

What follows is an example of a normal commercial purpose, used to illustrate and justify the above heresy and to put the case for more appropriate, immediately available and cheaper software. It is argued that most software currently used is written in code which is long on bastard English and short on logical and mathematical notation. This is a reflection of the simplistic approach adopted by the first language writers who worked within the constraints of tape and punched cards and minimal storage. Their world was "just one damn thing after another" and most languages have moved ever since with the tottering gait imposed by their bound feet. Hardware is now cheap – the bindings have gone. It is time we learned to leap.

A taxing problem

Consider the following specification for the calculation of Head Office charges:

"Head Office Charge is a percentage of a Store's annual turnover. The percentage varies with the bands into which the turnover falls. For example, the current bands and percentages for Grocery are:

From	To	%
0	5000	5
5000	8000	2.5
8000	15000	1
15000	30000	0.5
30000	-	0.1

There is nothing unusual about this requirement; accountants and analysts are expected to make, or to provide programs to make, such calculations every day of the week. So what is there to say about this specification and the ways it might be met?

Firstly it is ambiguous because it does not make clear which of two common methods of using bands (or ranges) is meant. Think of the boxes that are so favoured for questionnaires:

Please Indicate age band:

0-5	6-10	11-18
19-25	26-35	36-45
46-55	56-65	65+

It is quite clear that you must tick only one box. If you are between 26 and 35, then you cannot simultaneously be between 19 and 25.

Next, however painful, think of income tax. Here is the table:

1-16200	30%
16201-19200	40%
19201-24400	45%
24401-32300	50%
32301-40200	55%
40201+	60%

Suppose your income is £16500; will you pay 40% on all of it? Fortunately not, or no one would willingly earn between £16200 and £18901 (take-home pay at 70% of £16200 equals 60% of £18901). We all know that the real tax calculation is:

$$(.3 \times 16200) + ((.4 \times (16500 - 16200)))$$

i.e. that successive slices of income are charged at different rates. In other words more than one 'box' may be ticked.

Head Office Charge, in this case, is meant to be treated like Income Tax – different rates on successive slices of turnover – although, unlike Income Tax, it is regressive (the rate reduces). An accountant would have no difficulty; common sense would tell him to use the second method of calculation.

Programming common sense into computers is another matter entirely and to date common sense languages cannot be bought. So, even if we were able to feed this specification directly into the computer, its action would not be adequately determined.

So how must we write the specification to ensure that it is not misunderstood? How about:

"H.O.C. is the sum of $n(1)\%$ of the first £ $m(1)$ of annual turnover, $n(2)\%$ of the next £ $m(2)$ of annual turnover, . . . and so on. The current values for Grocery turnover are:

m	n	
5000	5	
3000	2.5	
7000	1	
15000	0.5	
rest	0.1	"

Do any of us suppose that this is sufficiently precise for a computer? Probably no one of us is that rash. From which I conclude that if, by some miracle, we had a 4th or 5th generation language which could interpret plain English, we should find the plain (to a computer) English very hard to write and painful to read, because to be precise we should have to write English which is far from plain. The English used in statute law is a good example of what happens when people try to make it precise and unambiguous.

In any case, we cannot wait for miracles; the job requires that we choose some language in which to instruct the computer to make the H.O.C. calculation. What choices do we have? How much code must we write and are the facilities offered by the language sufficient for the job to be cheap to set up and easy to maintain (change)?

While a spread-sheet type of code (Lotus 1-2-3 or Wizard) will look different from a file manipulation type (dBASE II or FOCUS), we can for any of them summarize the job into three steps:

1. find out which band this turnover falls into
2. sum the H.O.C. on all lower bands
3. add the amount due on the remainder.

As it is a small but instructive exercise, I suggest that those among my readers who are familiar with any computer language consider how to code the H.O.C. Of course there is no right way to do it – there are many ways of skinning a cat – but it will be helpful as a particular example of the general characteristics of code which are discussed below.

The spreadsheet approach

All computer languages are able to do simple arithmetic. They can add, subtract, multiply, divide, raise to a power and usually extract a square root. Therefore working out a maximum levy on each band of turnover will present no problem and it will follow closely the steps an accountant would take if using a manual spread-sheet. He will tend to use additional columns in the spread-sheet to hold his intermediate workings, thus:

COL1 FROM	COL2 TO	COL3 %	COL4 TO-FROM	COL5 CHARGE	COL6
0	5000	5	5000	250	/
5000	8000	2.5	3000	75	/
8000	15000	1	7000	20	
15000	30000	0.5	15000	-	
30000	-	0.1	-	-	

				345	

A program would probably not need COL4. The code for COL5 would be simple:

```
COL5(ROWS 1 THRU 4) = (COL2-COL1) * COL3/100
```

But of course we do not necessarily want all the values in COL5, as that will depend on the turnover which is a variable. The accountant working manually has no problem; each time he re-works he knows what the turnover is and chooses accordingly. Suppose the turnover is 10000. He looks down COL2 and says to himself "is 10000 greater than COL2" and mentally ticks that row if the answer is yes (see COL6). Then the first row of COL6 which has no tick is the one which will hold the calculation for the remainder of the charge:

COL5 ROW(COL6 first no tick) = (10000-8000) * 1/100

i.e. 20. So the total charge is $250 + 75 + 20 = 345$. The accountant has used mental ticks to choose the rows, but what is the program equivalent of a tick?

Some packages may be really clever. They may allow the turnover to be tested against a table of ranges without explicit coding. Then the programmer will provide a remainder calculation something like:

COL5 ROW(RANGE=TURNOVER) = (COL2 ROW(RANGE=TURNOVER)-COL1 ROW(RANGE=TURNOVER))
* COL3 ROW(RANGE=TURNOVER)/100

and the sum of the charge rows would be:

COL5 ROW6=SUM COL5 ROW(RANGE=1) TO ROW(RANGE=TURNOVER)

Less sophisticated software would require a set of tests like:

```
FOR ROWS 1 TO 5
  IF COL2>TURNOVER COL5=(TURNOVER-COL1)*COL3/100 GOTO LABEL
  OTHERWISE COL5=(COL2-COL1)*COL3/100
NEXT ROW
LABEL COL5 ROW6 = SUM COL5 ROWS 1 THRU 5
```

All programmers will recognize this old friend. It is the loop and can be a lot less friendly than the RANGE type of code especially if it is part of a nest of loops which all too easily turn into a nest of spaghetti.

So why cannot software have more friendly features like the range test? In principle there is no reason why not and most languages allow subroutines to be called from public libraries to perform frequently-needed calculations. In practice the reason is "where do we stop?". There will be as many friendly features as there are jobs to be done, and the users are bound to want a variation on the theme that no one has thought to provide.

Expert systems are one way to reduce this problem -- they acknowledge that the software has been written for a special purpose and build in particular features, ignoring the general cases. This is an admirable solution for areas where the job to be done is well understood and widespread. The two most successful examples are word-processing and conventional spreadsheet accounting.

Suppose however that your job is not run-of-the-mill. What then will be the language of your choice?

A mathematical approach

The reader will have noticed that the pseudo-code used for illustration contains as many arithmetical symbols as words. Also that the problem centres around the tests (ticks in COL6), which if they are explicit lead to looping code that is difficult to maintain. The code is a clumsy mixture of English or near-English words and a few mathematical symbols, some of which (* for multiply and / for divide) are not the conventional ones that everybody learnt at school.

Clearly more English does not answer the problem so how about using more mathematics?

Commercial users will respond that they do not need mathematics because "all we do is a bit of simple arithmetic – working out a few percentages – that sort of thing". This is true if we attend only to *what* the program does and no attention is paid to *how* it does it.

Think of the one bit of mathematics we can probably all remember: (*Ed: the name at least!*) Pythagoras' Theorem. Somewhere at the beginning of that page in the textbook there will be the phrase "for any right triangle . . ." For any particular triangle, it is easy enough to show by measuring that the square on the hypotenuse equals the sum of the squares on the other two sides, but Pythagoras said it is true of *any* right triangle and that is an altogether different grandmother (maybe as powerful as a wolf?).

Programs resemble wolves in this respect – they should be able to digest any small girl whatever the colour of her hood. That is, they should provide consistent results whatever the data input, after themselves checking that it is well-formed (that is valid, or girl-shaped). Programs are only written because there are many occasions on which the same algorithm is to be applied to different data. In the commercial world, one-off jobs are done more cheaply by hand or with a calculator; only horrendously difficult calculations would warrant writing a program to be run just once.

There is a word to describe this type of computation – it is said to be 'generalized' – and the equivalent mathematical discipline is algebra. In the example above, turnover is a variable, the values in the table are constants, the result is the Head Office Charge and the program is the formula relating them. The difference between the program and ordinary algebra is that a mathematician does not have to describe how to choose the band into which the turnover happens to fall because he is describing the relationship for the benefit of another person and he can assume that the right band will be chosen by "simple inspection". The programmer must substitute an algebra of "ticks" or "true"/"false" for "simple inspection". Such an algebra was developed in the last century by George Boole, but its implementation in most languages is crude.

Of course, a programming language will also need to manipulate text. It must allow tests like DIVISION='GROCERY', but much more power than this is desirable. Sorting is an obvious example but there are many others. Given two lists of names, are all items in list A also contained in list B, and if so where? Which elements of A are not in B? Is every element of list A unique? Left/right justify these lists or centre them. Extract subsets and join them . . . and so on.

Assume that we have a language which has a full range of algebraic and Boolean functions and text manipulation. What would the code look like that does the H.O.C. job, and what advantages would it have? Here is an example:

```
R←TURN HOC TABLE;BOO;ROW;FROM;TO;PC
* HOW TO USE LOGIC TO AVOID LOOPS
FROM TO PC←[1]TABLE
BOO←TURN TO ◊ ROW+1++/BOO
R←/(BOO*(TO-FROM)*PC+100),(TURN-FROM[ROW])*PC[ROW]+100
```

The reduction in quantity of code is small (this has 129 characters as against 136 for the pseudo-code given previously) but look more closely at that code, especially the first line which reads 'FOR ROWS 1 TO 5'. Suppose the number of turnover bands in the table changes. Then the code must be changed to limit the loop to the number of rows in the table. Now look at the new version. The number of rows is nowhere mentioned. It does not need to be mentioned because the code is generalized – it will work with a table of any size.

Here at last is the answer to the problem which data processing managers have spent years looking for – how to avoid maintenance costs *and* allow users to change the shape of their data.

To see the code at work, we can first look at the contents of the variable and the table of constants:

```
TURN
1500
TABLE
0      5000      5
5000   8000      2.5
8000   15000     1
15000  30000     0.5
30000  999999    0.1
```

and then switch on a trace to display the result calculated on each line:

```
TURN HOC TABLE
HOC[1]
HOC[2] (0 500 1200 2000 3000)(500 1200 2000 3000 999999)(4 2.5 1.5 1 .5)
HOC[3] 1 1 0 0 0
◊ 3
HOC[4] 42
```

and run the code:

```
pBIGTABLE
100 3

TURN←1500
TURN HOC BIGTABLE
42
```

and HOC (Head Office Charge), the result (pure chance, I swear) is 42 . . . the answer to the ultimate question about Life, the Universe and Everything!

The action of the code traced was as follows:

line 1: comment only

line 2: split the table into its columns and call them FROM, TO, PC

line 3: create a Boolean variable by comparing the turnover, TURN, with column TO (1500 is greater than 500 & 1200, but less than 2000, 3000 & 999999, so BOO becomes 1 1 0 0 0); the second part of line 3 calculates the number of the remainder row by adding 1 to the sum of the Boolean variable, thus creating the variable ROW.

line 4: uses these intermediate results to sum the remainder (the part after the comma) with the levy on all the rows multiplied by the Boolean which, naturally, reduces those rows corresponding to the Boolean value 0 to zero, while those corresponding to the Boolean value 1 are left unchanged. The Boolean is a mask selecting only those rows where the answer to the question was "yes" – i.e. true or ticked.

To demonstrate that the size of the table is of no consequence, we can run the code with a much larger table – BIGTABLE has 100 rows.

```

      TURNS+550 1300 2000
      ALLHOCs[1]
      ALLHOCs[2] (0 500 1200 2000 3000)(500 1200 2000 3000 999999)(4 2.5 1.5 1 .5)
      ALLHOCs[3]
      1 0 0 0 0
      1 1 0 0 0
      1 1 1 0 0
      0 2 3 4
      ALLHOCs[4] 21.25 39 49.5
  
```

Some serious generalization.

HOC takes a single turnover value and returns the Head Office Charge, but the program of which this is a part will be making the same calculation for many turnover values – perhaps for each department in a store or over successive years for a discounted cash flow. Must we write looping code or can we do all calculations at the same time?

Here is the code to calculate the H.O.C. on any number of turnover values. The code is still 4 lines long, but there is a rather less familiar notation:

- .. means 'each';
- °. ≥ means create a table of Boolean values instead of the vector we had before;
- ⊂ means treat what follows as a single object.

```

R=turns ALLHOCs TABLE;BOO;ROWS;FROM;TO;PC
* HOW TO USE NESTED ARRAYS TO DO MORE AT A TIME
FROM TO PC+1;TABLE
BOO=turns*.2TO 0 ROWS=1+;/BOO
R=(+/(1((TO-FROM)*PC+100))* "+BOO))+(turns-FROM[ROWS])*PC[ROWS]+100

```

To see how this works, here is the trace for only 3 turnover values, so that the size of the Boolean array stays small enough for us to view at a glance. (It has one row for each value in TURNS.

```

TURNS=550 1300 2000

ALLHOCs[1]
ALLHOCs[2] (0 500 1200 2000 3000)(500 1200 2000 3000 999999)(4 2.5 1.5 1 .5)
ALLHOCs[3]
1 0 0 0 0
1 1 0 0 0
1 1 1 0 0
0 2 3 4
ALLHOCs[4] 21.25 39 49.5

```

and here are the three charges: 21.25 39 49.5

However, to see the power of this generalization, we can use a large table of turnovers for 14 departments by 8 years:

```

TURNOVER
238704 298380 340153 367365 385734 405020 425271 446535
38972 48715 55535 59978 62977 66126 69432 72904
25900 32375 36908 39860 41853 43946 46143 48450
23178 28973 33029 35671 37454 39327 41294 43358
0 0 0 0 0 0 0 0
10423 13029 14853 16041 16843 17685 18569 19498
238807 298483 340256 367468 385837 405123 425374 446638
39075 48818 55638 60081 63080 66229 69535 73007
26003 32478 37011 39953 41956 44049 46246 48553
23281 29076 33132 35774 37557 39430 41397 43461
738704 298380 340153 367365 385734 405020 425271 446535
10526 13132 14956 16144 16946 17788 18672 19601
38972 48715 55535 59978 62977 66126 69432 72904
15846 19808 22582 24387 25606 26887 28231 29642

TURNOVER ALLHOCs TABLE
1238.02 1536.4 1745.265 1881.325 1973.17 2069.6 2170.855 2277.175
239.36 288.075 322.175 344.39 359.385 375.13 391.66 409.02
174 206.375 229.04 243.8 253.765 264.23 275.215 286.75
160.39 189.365 209.645 222.855 231.77 241.135 250.97 261.29
0 0 0 0 0 0 0 0
96.615 109.645 118.765 124.705 128.715 132.925 137.345 141.99
1238.535 1536.915 1745.78 1881.84 1973.685 2070.115 2171.37 2277.69
239.875 288.59 322.69 344.905 359.9 375.645 392.175 409.535
174.515 206.89 229.555 244.315 254.28 264.745 275.73 287.265
160.905 189.88 210.16 223.37 232.285 241.65 251.485 261.805
7378.02 1536.4 1745.265 1881.325 1973.17 2069.6 2170.855 2277.175
97.13 110.16 119.28 125.22 129.23 133.44 137.86 142.505
239.36 288.075 322.175 344.39 359.385 375.13 391.66 409.02
123.73 143.54 157.405 166.435 172.53 178.935 185.655 192.71

```

Summary

The purpose of this paper has been twofold. First to persuade the reader that the search for a programming language which can accept plain English will be fruitless. Secondly to demonstrate that the effect of applying a language with the combined power of algebra and logic to a commercial problem is to create concise code which will tolerate major changes in the format of the data offered to it. This has the effect of accommodating change without incurring additional programming costs.

Of course, an intellectual investment must be made in learning the concepts behind the language, but while this will be true whatever software is chosen, there is another factor to be taken into account:

Mathematics is not likely to go out of fashion

whereas we have all seen VISICALC rise and fall in the space of a few years, and the upstart relational database challenge the hierarchical champion.

Whenever new fields in science and technology are opened up, there is a spate of not very well co-ordinated activity as its potential is explored, but this is followed by a more sober consolidation. The consolidation is almost always mediated by the development of mathematical tools to fit the application of the new technology. For the sort of software considered here, this tool is already available and certainly no more difficult to use than the code it replaces.

As illustration, I attach code taken from the FOCUS User's Manual. The purpose of the code is to allow FOCUS to call a subroutine which enables it to translate a month number into a month name (1 becomes 'JANUARY', 2 'FEBRUARY', etc.). Here is an example of FOCUS code which uses it:

FOCUS User's Manual

User written subroutines

Example of running MTHNAM

```
>>DEFINE  
>MONTH/12 = RCOST/1000 ;  
>MONTH_NAME/A9 = MTHNAM(MONTH,MONTH_NAME);  
>END  
>>TABLE FILE CAR  
>PRINT RCOST MONTH MONTH_NAME  
>END
```

The manual supplies a choice of code to do this translation in ASSEMBLER, FORTRAN, PL/1 and COBOL (see Appendices 1 to 4). Here is the equivalent in the language of my choice:

```

R←MTHNAME MTH;⌈IO;NUM
  ⌈ TRANSLATE MONTH NUMBER INTO MONTH NAME
NUM←MTH ⌈ KEEP ORIGINAL NUMBER EVEN IF ERROR
⌈IO←1 ⌈ MTH←(MTH⌈12)×MTH ⌈ ⌈IO←0 ⌈ ZERO TO ERRORS
R←(13 10 ρ'ERROR JANUARY FEBRUARY MARCH APRIL MAY JUNE
    JULY AUGUST SEPTEMBER OCTOBER NOVEMBER DECEMBER '')[MTH;]
R←(νNUM), ' ',R

```

```

      MTHNAME 1
1 JANUARY

```

But it would be much better if the function could be generalized in such a way as to make it unnecessary to specify how many numbers there are without needing to write a loop. With a little more 'mathematical' notation it can be done; the following function takes a list of any length.

```

R←MONTHNAMES MTHS;⌈IO
  ⌈ TRANSLATE MONTH NUMBER INTO MONTH NAME
⌈IO←1
MTHS←1+(MTHS⌈12)×MTHS
R←(13 14 ρ' ERROR 1 JANUARY 2 FEBRUARY 3 MARCH 4 APRIL
    5 MAY 6 JUNE 7 JULY 8 AUGUST 9 SEPTEMBER 10 OCTOBER
    11 NOVEMBER 12 DECEMBER '')[MTHS;]
R←((1+ρR),×/1+ρR)ρR
      MONTHNAMES 1 2 4
1 JANUARY
2 FEBRUARY
4 APRIL

```

But why limit the input to lists? There is no reason why a truly general function should not process a table (a list of lists). Let us follow Pythagoras and make the leap—

“for any array of numbers . . .”

```

      NUMBERS←10 4 ρ13
      NUMBERS
1 2 3 4
5 6 7 8
9 10 11 12
13 1 2 3
4 5 6 7
8 9 10 11
12 13 1 2
3 4 5 6
7 8 9 10
11 12 13 1
      MONTHNAMES NUMBERS
1 JANUARY 2 FEBRUARY 3 MARCH 4 APRIL
5 MAY 6 JUNE 7 JULY 8 AUGUST
9 SEPTEMBER 10 OCTOBER 11 NOVEMBER 12 DECEMBER
ERROR 1 JANUARY 2 FEBRUARY 3 MARCH
4 APRIL 5 MAY 6 JUNE 7 JULY
8 AUGUST 9 SEPTEMBER 10 OCTOBER 11 NOVEMBER
12 DECEMBER ERROR 1 JANUARY 2 FEBRUARY
3 MARCH 4 APRIL 5 MAY 6 JUNE
7 JULY 8 AUGUST 9 SEPTEMBER 10 OCTOBER
11 NOVEMBER 12 DECEMBER ERROR 1 JANUARY

```

Appendix 1. ASSEMBLER User-written Subroutine MTHNAM (FOCUS User's Manual)

MTHNAM	START	0	
	STM	14,12,12(13)	save registers
	BALR	12,0	load base registers
	USING	*,12	
*			
	L	3,0(0,1)	load addr of first arg into R3
	LD	4,=D'0.0'	clear out FPR4 and FPR5
	LE	6,0(0,3)	FP number in FPR6
	LPER	4,6	abs value in FPR4
	AW	4,=D'0.000001'	add rounding constant
	AW	4,DZERO	shift out fraction
	STD	4,FPNUM	move to memory
	L	2,FPNUM+4	integer part in R2
	TM	0(3),B'10000000'	check sign of original number
	BNO	POS	branch if positive
	LCR	2,2	complementr if positive
*			
POS	LR	3,2	copy month number into R3
	C	3,=F'0'	is it zero or less?
	BNP	INVALID	yes, so invalid
	C	3,=F'12'	is it greater than 12?
	BNP	VALID	no, so valid
*			
INVALID	LA	3,13(0,0)	bad, set R3 to item 13
*			
VALID	SR	2,2	clear out R2
	M	2,=F'9'	multiply by shift in table
	LA	6,MTH(3)	get addr of item in R6
	L	4,4(0,1)	get addr of second item in R4
	MVC	0,(9,4),0(6)	move in text
*			
	LM	14,12,12(13)	recover regs
*	BR	14	return
	DS	0D	alignment
FPNUM	DS	D	floating-point number
DZERO	DC	X'4E00000000000000'	shift constant
MTH	DC	CL9'dummyitem'	month table
	DC	CL9'JANUARY'	
	DC	CL9'FEBRUARY'	
	DC	CL9'MARCH'	
	DC	CL9'APRIL'	
	DC	CL9'MAY'	
	DC	CL9'JUNE'	
	DC	CL9'JULY'	
	DC	CL9'AUGUST'	
	DC	CL9'SEPTEMBER'	
	DC	CL9'OCTOBER'	
	DC	CL9'NOVEMBER'	
	DC	CL9'DECEMBER'	
	DC	CL9'***ERROR***'	
	END	MTHNAM	

Appendix 2. PL/1 User-written Subroutine MTHNAME (FOCUS User's Manual)

```

MTHNAM:      PROC(MTH,MONTH)  OPTIONS(COBOL);
DECLARE      MTH DECIMAL FLOAT (16) ;
DECLARE      MONTH CHARACTER (9) ;
DECLARE      IMTH FIXED BIN (15,0) STATIC;
DECLARE      MONTH-TABLE(13) CHARACTER (9) STATIC
              INIT ('JANUARY',
                    'FEBRUARY',
                    'MARCH',
                    'APRIL',
                    'MAY',
                    'JUNE',
                    'JULY',
                    'AUGUST',
                    'SEPTEMBER',
                    'OCTOBER',
                    'NOVEMBER',
                    'DECEMBER',
                    '**ERROR**')

IMTH = MTH + 0.0000001;
IF IMTH < 1 | IMTH > 12 THEN
    IMTH = 13 ;
MONTH = MONTH_TABLE(IMTH);
RETURN;
END MTHNAME;

```

Appendix 3. FORTRAN User-written Subroutine MTHNAM (FOCUS User's Manual)

```

SUBROUTINE MTHNAM (MTH,MONTH)
REAL*8      MTH
INTEGER*4    MONTH(3),A(12,3),IMTH
DATA
+   A( 1,1)/'JANU'/, A( 1,2)/'ARY ', A( 1,3)/' ',
+   A( 2,1)/'FEBR'/, A( 2,2)/'UARY', A( 1,3)/' ',
+   A( 3,1)/'MARC'/, A( 3,2)/'H ', A( 1,3)/' ',
+   A( 4,1)/'APRI'/, A( 4,2)/'L ', A( 1,3)/' ',
+   A( 5,1)/'MAY '/, A( 5,2)/' ', A( 1,3)/' ',
+   A( 6,1)/'JUNE'/, A( 6,2)/' ', A( 1,3)/' ',
+   A( 7,1)/'JULY'/, A( 7,2)/' ', A( 1,3)/' ',
+   A( 8,1)/'AUGU'/, A( 8,2)/'ST ', A( 1,3)/' ',
+   A( 9,1)/'SEPT'/, A( 9,2)/'EMBE', A( 1,3)/'R ',
+   A(10,1)/'OCTO'/, A(10,2)/'BER ', A( 1,3)/' ',
+   A(11,1)/'NOVE'/, A(11,2)/'MBER', A( 1,3)/' ',
+   A(12,1)/'DECE'/, A(12,2)/'MBER', A( 1,3)/' ',
+   A(13,1)/'**ER'/, A(13,2)/'ROR*', A( 1,3)/'* ',
    IMTH=MTH+0.0000001
    IF (IMTH.LT.1.OR.IMTH.GT.12) IMTH=13
    DO 1 I=1,3
1  MONTH(I)=A(IMTH,I)
    RETURN
END

```

Appendix 4. COBOL User-written Subroutine MTHNAM (FOCUS User's Manual)

```

ID DIVISION
PROGRAM-ID.      MTHNAM.
ENVIRONMENT      DIVISION.
CONFIGURATION SECTION.
SOURCE-COMPUTER.  IBM-370.
OBJECT-COMPUTER.  IBM-370.
DATA DIVISION.
WORKING-STORAGE SECTION.
01  MONTH-TABLE.
    05  FILLER PIC X(9) VALUE 'JANUARY '.
    05  FILLER PIC X(9) VALUE 'FEBRUARY '.
    05  FILLER PIC X(9) VALUE 'MARCH '.
    05  FILLER PIC X(9) VALUE 'APRIL '.
    05  FILLER PIC X(9) VALUE 'MAY '.
    05  FILLER PIC X(9) VALUE 'JUNE '.
    05  FILLER PIC X(9) VALUE 'JULY '.
    05  FILLER PIC X(9) VALUE 'AUGUST '.
    05  FILLER PIC X(9) VALUE 'SEPTEMBER'.
    05  FILLER PIC X(9) VALUE 'OCTOBER '.
    05  FILLER PIC X(9) VALUE 'NOVEMBER '.
    05  FILLER PIC X(9) VALUE 'DECEMBER '.
    05  FILLER PIC X(9) VALUE '**ERROR**'.
01  MLIST REDEFINES MONTH-TABLE.
    05  MLINE OCCURS 13 TIMES INDEXED BY IX.
        10 A PIC X(9).
01  IMTH PIC S9(5) COMP.
LINKAGE SECTION.
01  MTH COMP-2.
01  MONTH PIC X(9).
PROCEDURE DIVISION USING MTH, MONTH.
BEG-1.
    ADD 0.000001 TO MTH.
    MOVE MTH TO IMTH.
    IF IMTH<+1 OR>12
        SET IX TO +13
    ELSE
        SET IX TO IMTH.
        MOVE A (IX) TO MONTH.
        GOBACK.

```

Building on a solid past

1976
ISAM/AP for APLSV
VSAM/AP for APLSV
1977
AFM/AP* for APLSV
AFM/AP for VSPC/VSAPL
1978
AFM/AP for CMS/VSAPL
CALL/AP for APLSV
1979
CALL/AP for VSPC/VSAPL
1980
AFM/AP for TSO/VSAPL
1981
CALL/AP for CMS/VSAPL
Enhanced Format for VSAPL
Keyed Access Option
1982
Interactive Link Option
Mail Exchange Option
CALL/AP for TSO/VSAPL
1983
APLPRINT
Input Stack Processor
Output Stack Processor
1985
AFM/AP for CMS/APL2
AFM/AP for TSO/APL2
CALL/AP for APL2
Enhanced Format for APL2
1986
APL2 Display Capture
Indexed File Option

When Interprocess got started ten years ago, we had a lot of ideas about APL.

As far back as APLSV, we believed that APL users needed better file facilities. And we began by providing shared access to ISAM and VSAM.

Then came a component file system that would run on any in-house IBM mainframe APL system.

But our customers wanted more than just component files. They asked for—and got—keyed access and an indexed, PCS-style file structure.

Another thing they needed was a Quad-FMT equivalent that wouldn't change IBM source code. We did it for VSAPL. And it's still there for APL2.

Recently APL2 added the ability to call FORTRAN or BAL from APL. That's something our customers have had since 1978.

Interprocess also pioneered a concept: That APL enhancements don't have to change the APL product itself. That you can achieve results from the IBM interfaces already present.

It's a concept that gives our customers unparalleled continuity from one APL environment to the next. Something solid to build on for the future.



Interprocess Systems

6065 Roswell Road, N.E., Suite 608, Atlanta, Georgia 30328 USA, (404) 257-1110

Steps to a better BASIC

by Anthony Camacho

In this issue's extract from his series of articles introducing APL to BASIC users. Anthony Camacho shows how APL provides powerful selection processes. This article is reproduced with kind permission of Datalink magazine.

A Choice Method of Selection

Conditional operations without IF

In most microcomputer versions of BASIC, the logical value 'true' is represented by a one and 'false' by zero. A few treat any non-zero value for 'true' and/or set true variables to -1. This allows some elegant tricks. If you want an expression which puts B into A if $x=y$ then

$$A = A * (X <> Y) + B * (X = Y)$$

will fill the bill. Either the bracket $X <> Y$ or the bracket $X = Y$ will be a one so either $A * 1 + 0$ or $0 + B * 1$ will go into A.

If your computer renders $\text{PRINT } 1 = 1$ as -1 then the expression you need is

$$A = -A * (X <> Y) - B * (X = Y).$$

If what you want to do is to increment a counter until it reaches, say 50, then the expression is even neater:

$$C = C + (C < 50) \text{ [or } -(C < 50) \text{ to suit]}$$

The power of this kind of selection is greatly developed in APL, partly because it uses arrays of numbers of logical values and partly because it has some extra operators. In particular among these is the 'compression' operator, displayed as an oblique stroke. If the variable BOOLE is an array of ones and zeroes the same shape as an array VALUES containing any kind of item, then the expression 'BOOLE/VALUES' yields another array with as many items as there were ones in BOOLE. The items are, of course, the items from VALUES that corresponded to the ones in BOOLE.

For example suppose you wish to remove all punctuation characters from a line (or page or chapter) of text. In APL it is easy to construct an array of ones and zeroes the same length as the text with zeroes where the punctuation marks are and ones for all the other characters. This is done by using the membership operator which gives ones and zeroes as its result. The result is the same shape as the left argument and has a one whenever the item in the left array occurs in the right argument. Then compression of the original array gives the result you want.

```

TEXT←'DEMONSTRATION. HOW, MEMBERSHIP
      : (c); AND ?COMPRESS? (/) CAN
      REMOVE PUNCTUATION'
TEXT ← '.,,:?!'
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0
  1 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0
    0 0 1 0 0 0 0 0 0 1 0 0 0 0 0 0
    0 0 1 0 0 0 0 0 0 0 0 0 1 0 0 0
    0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
    0
* EXCHANGE 0 AND 1 WITH LOGICAL NOT
  (~)
(~TEXT ← '.,,:?!' )/TEXT
DEMONSTRATION HOW MEMBERSHIP (c) AND
COMPRESS (/) CAN REMOVE PUNCT
UATION

```

A similar line will convert all lower case to upper case. First it converts the text to its numeric form; then creates an array with ones where there are lower case characters (they have codes from 97-122). This array is then multiplied by 32 and subtracted from the numeric form of the text. (The lower case code for each letter is 32 more than the code for the corresponding upper case letter). When the codes are converted back from numbers to letters they are all upper case.

```

* PHR CONTAINS A SENTENCE IN UPPER A
  ND LOWER CASE

  PHR
Now is the time for all good men to
come to the aid of the SDP.

* QAVI CONVERTS CHARACTERS TO NUMBER
  S AND NUMBERS TO CHARACTERS
QAVI(32*(N>96)^(N<123))+N←QAVI PHR
NOW IS THE TIME FOR ALL GOOD MEN TO
COME TO THE AID OF THE SDP.

```

You may feel this messing about with text is trivial. The examples were chosen because text arrays are naturally displayed horizontally and so the examples take less space than numbers in columns would do. But similar expressions are used daily by APL users to select, say, all the accounts whose credit limit is exceeded or all the stock items whose rate of turn is in the bottom quartile or even all the MPs who were elected with the support of less than a third of their electorate.

And of course any action the program can take could easily be made to depend upon such selections.

TECHNICAL SECTION

This section of VECTOR is aimed principally at those of our readers who already know APL. It will contain items to interest people with differing degrees of fluency in APL.

I.P. Sharp Associates provides implementations of the APL language for the largest IBM mainframe through to the IBM personal computer. In addition to the proven benefits of SHARP APL, we offer transportability of applications between machines.

Come see us at APL86 or contact:

**I.P. Sharp Associates
Heron House
10 Dean Farrar Street
London SW1H 0DX
England
Tel: (01) 222-7033**

**I.P. Sharp Associates Ltd.
Suite 1900, Exchange Tower
2 First Canadian Place
Toronto, Ontario M5X 1E3
Canada
Tel: (416) 364-5361**

**I.P. Sharp Associates Pty. Ltd.
8th Floor, Carlton Centre
55 Elizabeth Street
Sydney, New South Wales, 2000
Australia
Tel: (02) 232-6366**

**I.P. Sharp Associates (S) Pte. Ltd
SIA Building, No. 14-00
77 Robinson Road
Singapore 0106
Republic of Singapore
Tel: 2230211**

**I.P. Sharp Associates Inc.
1200 First Federal Plaza
Rochester, New York, 14614
U.S.A.
Tel: (716) 546-7270**

**SHARP APL/XA
APL/370
APL/PCX
APL/PC**

WORLD LEADERS IN APL SYSTEMS

International Offices:

**Amsterdam, Boston, Brisbane, Brussels, Calgary, Canberra, Chicago, Copenhagen, Coventry,
Dallas, Dublin, Dusseldorf, Edmonton, Frankfurt, Halifax, Helsinki, Hong Kong, Houston,
Jeddah, Kuala Lumpur, London, Los Angeles, Madrid, Melbourne, Mexico City, Milan,
Montreal, New York, Oslo, Ottawa, Palo Alto, Paris, Rochester, Rome, Santa Clara, Seoul,
Singapore, Stockholm, Sydney, Taiwan, Tokyo, Toronto, Vancouver, Victoria, Vienna,
Washington, Zurich**



**I.P. Sharp
Associates**

Technical Editorial

by Jonathan Barman and Dave Ziemann

The objective of anyone creating a system is to make it work well and not spend too much time programming it. APL is but one of the tools that a programmer can use, and if other languages or systems can do a specific job better and quicker than APL there is no good reason not to use them. Often a mixture of APL and other languages are used so that we can have the best of both worlds; sometimes the bulk of the processing can be handled by a package and the twiddly bits dealt with in APL, and occasionally APL is just too slow for a particular process, and so it has to be written in a compiled language.

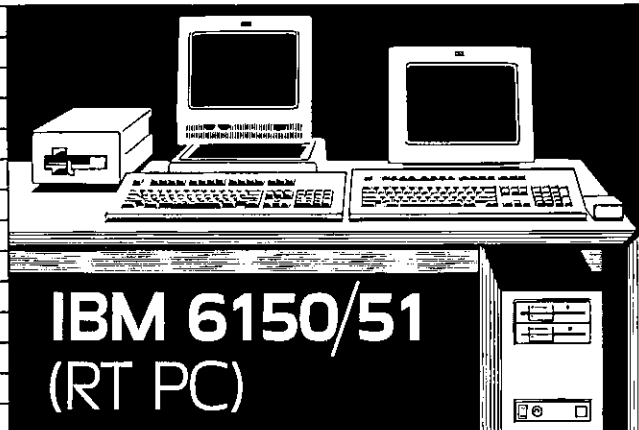
A large amount of APL is already mixed with other systems. The typical PC user does a bit in APL, comes out to the DOS environment to run packages such as DBASE or SYMPHONY, goes back into APL and does more work. Similarly, the IBM VSAPL user running under TSO will intersperse APL with ISPF sessions. The switching process can be automated with BAT, CLIST or EXEC files. If an extensive amount of work is done before switching to the next environment then this method works well. If, however, the amount of work done outside APL is small and the switching between systems is frequent, then this method works very slowly. The APL interpreter is a large beast and loading it takes time, so nipping in and out of APL slows things down. The alternative is to load APL and stay there, running other applications as and when required. This is the best method where small programs need to be run quite frequently. The problem here is that every APL interpreter seems to have a completely different way of accessing the outside world and running programs; some easy, some quite tricky, and some impossible. The impossible is usually deliberate, for example a timesharing bureau will need to guarantee complete security and if assembler programs could be run the security might be compromised. It is normally easy to run programs that are designed to be called directly as independent units, but it is usually much more difficult to run programs that manipulate data in the workspace, as one needs to know quite a lot about the internal structure of the interpreter.

APL2's Name Association facility looks as if it will make calling programs much easier. The quadNA system function is available in Release 2 of the product, and allows external programs to be invoked as if they are locked APL functions.

For some time a shared variable processor has been available for VSAPL which does a similar sort of job, but there does not seem to be anything comparable for the other APL interpreters.

dyalog APL

IS AVAILABLE **NOW** FOR THE HIGH PERFORMANCE



— AND FOR 22 OTHER **UNIX*** SYSTEMS! —

The IBM 6150/51 is a new family of high performance 32-bit multi-user micro computers and workstations. It is based on reduced instruction set (RISC) technology and AIX**, IBM's enhanced version of UNIX System V; and is an excellent basis for a Dyalog APL system.

Dyalog APL on the IBM 6150/51 is fast, especially with the floating-point accelerator, and there is practically no limit on the size of APL objects and workspaces.

The entire range of standard IBM screens and printers for the 6150/51 is fully supported, as are most of the common APL/ASCII vdu's.

All AIX facilities can be accessed from within a Dyalog APL function and results may be captured in an APL array. Compiled subroutines written in C or Fortran can easily be incorporated into applications, and interfaces between Dyalog APL and several IBM supported software packages such as SQL and the Professional Graphics Series are available.

Dyalog APL adheres to the draft ISO standard for APL and includes many of the features of APL2. Nested arrays, and corresponding extensions to primitive functions and operators, greatly simplify the processing of lists and non-rectangular data structures. Dyalog APL contains a host of useful features including a session manager, full-screen editor, and full-screen data manager.

The IBM 6150/51 with Dyalog APL is available direct from Dyadic. Prices, including Dyalog APL, start from around £15,000.

For further information, please contact Peter Donnelly
Dyadic Systems Ltd., Park House, The High Street, Alton,
Hampshire GU34 1EN, United Kingdom
Telephone: Alton (0420) 87024 (10 lines). Telex 858811

*UNIX is a trademark of AT & T Bell Laboratories. IBM is a registered trademark of International Business Machines Corporation.

**AIX is a trademark of International Business Machines Corporation.

Technical Correspondence

From Dick Bowman

Sir:

Incorporated into a graphics package there's the following little snippet of code associated with curve-fitting:

```
COEFF←Y⊞X←.0,1CURVEORDER
```

There are a few twiddly bits around it to ensure that it doesn't fall over too frequently because the user has total control of data values Y and X and of the order of the fitted curve – in the worst case the embedding function reverts to joining the points with a set of straight lines.

Curious as it may sound this had been working fine for some years in a VSAPL/VSPC environment and also in an APL*PLUS/PC incarnation; however implementation in APL2/TSO resulted in a series of phone calls along the approximate lines of "ere, why are my curves all jaggy?". So with a sample set of data to hand we take a look at what goes on in the three incarnations:

```
Y←37 36 39 38 41 39 43 38 43 35 42
X←1984+111
```

Strange as it may sound, our user wanted to fit a third-order curve to this, so let's look at

```
Y⊞X←.0 1 2 3
~6159617.48 92648.90706 ~46.47206775 0.007770014433  R VSAPL
~61569559.53 92648.8197 ~46.47202385 7.77000708E~3  R APL*PLUS PC
DOMAIN ERROR                                         R APL2
```

A fascinating finding; further investigation shows that there is a reasonable agreement between the three cases for orders less than three, and when we go to higher orders both VSAPL and APL*PLUS/PC refuse at six, agree with one another moderately well at four, but diverge wildly at five. Which gives us a potentially very interesting situation that with the same data and the same code within the functions, we can end up with three totally different graphs depending on which system we use to draw them.

Let's come to some conclusions:

- Given sufficiently perverse data there's something different about the domino algorithm in APL2;
- Letting end-users choose the order of a fitted curve is a nice idea in theory and essential sometimes, but quite often all they want is for it to look nice and smooth;

- c) Sometimes a little thought about algorithmic subtlety can repay itself in avoidance of nasty surprises.

Yours sincerely,

Dick Bowman,
C.E.G.B.,
85 Park Street,
London, SE1.

(Editor: The draft ISO APL standard specifies that the algorithm to be used for matrix-divide is implementation-defined. Although no sensible implementer would use an algorithm other than matrix division, it does allow implementers to use different variations of the general technique. This means that different APLs can produce different results and yet still conform to the standard. An acceptable algorithm for matrix division can be found in "Domino - An APL Primitive Function - Its Implementation and Applications", from Quote-Quad Vol.3, No.4, February 1972.)

Surely There Must Be A Better Way

by Dave Ziemann

This issue of VECTOR includes an article by Robert Pullman on apportioning data in APL. You'll find it later on in the technical section. Can anyone find an alternative solution to Robert's <CROSS> function?

Hacker's Corner (1)

by Adrian Smith

Following up on my attempt on the integrity of the TSO/APL Session Manager, I thought it might be time to explore the hacking potential of the workspace access processor (AP102 – authorized personnel only). Here is an alternative route to your TSO Logon Id, for which many thanks to Andy Chadwick of our Technical Support Group:

```

R←TSOID;C;CTLMS;DATHS;BASE;OFF;XX;ZC
-----
[1]  ^DIG OUT YOUR USER ID, HAVING CHAINED DOWN
[2]  ^THREE LOTS OF CONTROL BLOCKS TO FIND IT.
[3]  ^

[4]  C←102 □SVC 2 5 ρ'CTLMSDATHS'
[5]  C← 1 0 1 0 □SVC 2 5 ρ'CTLMSDATHS'
[6]  CTLMS←1,540,4
[7]  BASE←DATHS
[8]  CTLMS←1,(BASE+180),4
[9]  OFF←DATHS
[10] CTLMS←1,(OFF+264),4
[11] OFF←DATHS
[12] CTLMS←1,OFF,8
[13] XX←,⌵(4ρ256)⌵DATHS
[14] ZC←'ABCDEFGHIJAAAAAAAAAJKLHNPQRAAAAAAAAASTUVWXYZ'
[15] ZC←ZC,'AAAAAAAA0123456789?'
[16] R←ZC[(ρZC)⌵192+(⌵1⌵XX)⌵XX]
      ▼

```

Unfortunately, this operates only in □PEEK mode; real hackerdom begins with □POKE, as illustrated .

Hacker's Corner (2)

by Adrian Smith

One of the few minor niggles with APL*PLUS/PC is the way the keyboard reverts to the US version when you shift out of APL mode. This moves a couple of keys round, and (more importantly) leaves you without a £ sign.

Before you start poking around in the key definitions, you need a little function to show you what the current settings are:

```

R←MODE SEE KEY;TAB
-----
[1]  ⍎DISPLAY CURRENT KEY DEFINITIONS
[2]  ⍎NOTE THAT <MODE> DEFAULTS TO ALL 6 DEFINITIONS.
[3]  ⍎
[4]  ⍎(0=⍎HC 'MODE')/'MODE'+0 1 2 3 4 5'
[5]  TAB+256⍎PEEK 198 197
[6]  R←⍎AV[1+⍎PEEK TAB+MODE+.+8×KEY-1]
▽

```

The relevant key numbers (row by row, left to right) are:

	STANDARD PC -----	PC/AT -----
NOS :	1+⍎12	41, (1+⍎12), 43
QH :	15+⍎12	15+⍎12
AS :	29+⍎12	29+⍎11
ZX :	42+⍎11	43+⍎10

To set the text keyboard to match the keytops (and get diamond back where you expect it on the PC/AT) you should:

```

82 KBΔSET ⍎AV[178 49 178 69 49 178]  ⍎SHOP DEFNS ON <INS> KEY

40 41 3 4 KBΔSET ⍎AV[2 4p40 36 51 52, 65 127 35 157]  ... PC

41 3 4 40 KBΔSET ⍎AV[2 4p93 51 52 40, 125 35 157 65]
43      KBΔSET '£~00'  ... PC/AT

```

```
R←KEY KBΔSET DEFN;MODE;TAB
```

```
-----
[1]  *REPLACE CURRENT KEY DEFINITIONS WITH TABLE SUPPLIED.
[2]  *
[3]  * <KEY> IS A VECTOR OF KEY NUMBERS.
[4]  * <DEFN> IS A MATRIX OF CHARS WITH AS MANY COLS AS THERE
[5]  * ARE ELEMENTS IN <KEY>. EACH ROW REDEFINES:
[6]  *
[7]  *                                1) LOWER CASE TEXT
[8]  *                                2) UPPER CASE TEXT
[9]  *                                4) LOWER CASE APL
[10]  *                               5) UPPER CASE APL
[11]  *                               6) ALT-SHIFT APL
[12]  * ACDS NOVEMBER 1985
[13]  *

[14]  MODE←0,~1+1↑ρDEFN
[15]  TAB←256↓DPEEK 198 197
[16]  R←(1+0AV↓DEFN)DPOKE TAB+MODE+.8×KEY-1
[17]  R←'KEYS REDEFINED .... ',,DEFN,[1] ' '
▽
```

Of course this opens up lots of interesting ideas. How about saving some Alt-shifts by copying grade-up and grade-down on to shifted A and W? Maybe transpose could find another home on shift-X? I would certainly like to get the brackets and punctuation back where they belong; on a PC which is mainly used for Lotus and WP, the last thing I want is stickers which ruin the feel of the keyboard and confuse everyone else.

Why not have a go . . . find the layout which best suits your way of working, and then we can all have a jolly good barney about it at APL-86!



APL

ON THE PC

MetaTechnics

We are experts in all aspects of APL from Mainframes to Micros and offer a complete range of services including **consultancy, training, custom software, software packages and hardware.**

Software

APL*PLUS/PC V5.....£695
MetaPack.....£495
Complete set of Utilities
MetaScreen.....£245
AP 124 Emulator for PC
STATGRAPHICS.....£995
Graphics Package
APL — IEEE488 Interface£245
Control Laboratory
Instruments from APL

Hardware

TANDON PCA30M....£2995
8Mhz 80286 AT Compatible
ALLOY 60MByte Tape.....£825
AMPERE APL LAPTOP.....P.O.A
68000, LCD SCREEN LAPTOP
PORTABLE.
NEC V20 Chip Upgrade.....£12
Speeds APL on standard PC by up
to 20%

*all prices ex VAT

MTS MetaTechnics Systems Ltd
Unit 216, 62 Triton Road,
London SE21 8DE, 01-670 7959

The Systems Builders

Prize Competition

Get your directories updated

by Jonathan Barman

Filing systems use directories to keep track of where the data actually resides, and APL is no different – only it is supposed to be easier to manage.

The file system in question has a fixed record size which will accommodate APL objects below a certain size limit. The programmer has decided on a very simple file structure. The first record of the file contains the directory, all other records contain data. In this case the data consists of matrices with fixed width but a varying number of rows. Being a tidy minded sort of soul, the programmer has decided to use as much of the space in each record as possible, so that when new data is added to the file as many records as possible are filled up before new records are added.

The directory has 3 columns: column 1 is the file record number, column 2 positive integers which identify the data matrix and column 3 contains the number of rows in the APL matrix currently held in the record. The first row of the directory refers to itself and therefore has a special meaning. It has a 1 in the first column indicating record 1 and a 0 in the second column because the directory is not a data item. The third column gives the maximum number of data matrix rows that can be held in a record rather than the number of rows in the directory. For example:

```
      DIR
1      0 511
13 100 276
8 100 422
7 100 511
10 200 425
2 200 511
12 300 511
```

The data deletion program has left things in a bit of a mess. Records that are available for use have been removed entirely from the directory (e.g. records 3 to 6), and some records which were full now have space (e.g. records 8,10,13).

When new data is added the programmer wants to fill up records where the data identifiers match, and use the lowest numbered records possible for the new records. He has produced a matrix giving the number of data rows that have to be added; column 1 is the data identifier and column 2 is the total number of rows:

```
      RECS
100 218
200 731
300 134
```

The programmer wants a new directory which has an extra column showing how many rows have to be added to each record:

```
DIR NEWDIRECTORY RECS
1  0 511  0
7 100 511  0
13 100 276 218
8 100 422  0
2 200 511  0
10 200 425 86
3 200  0 511
4 200  0 134
12 300 511  0
5 300  0 134
```

He can then update every record that does not have a zero in column 4, and at the end of the process add columns 3 and 4 together and put the new directory back into the first record.

Can you write the NEWDIRECTORY function?

Answers received before 31 July 1986 will be eligible for a share of the £50 prize.

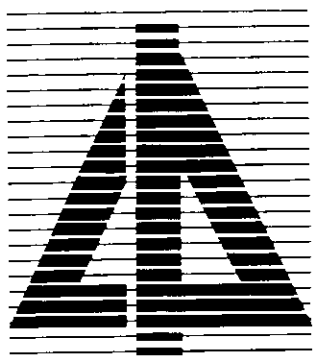
Introduction to Contributed Articles

by Dave Ziemann

Robert Pullman is President of the Rochester Group Inc., who operate out of Rochester, New York. Robert kicks us off with his ideas on how to solve apportionment problems in APL – how to split a total figure over a number of weighted parts. He uses APL to demonstrate informal proofs of his results as well as to provide us with some useful utility functions in his article “Quick and Dirty Apportionment in APL”.

John Sullivan, a Senior Research Officer with National Westminster Bank in London, is our next contributor. He’s taken a new look at an old problem – how to generate the Fibonacci numbers – and found some insights. His message is “Do the analysis before you write the program”. How we agree!

Paul Chapman is author of VIZ::APL, the 8-bit virtual APL interpreter, and an independent consultant. Paul’s heart-felt plea for naming conventions and documentation standards for APL was rather long for a letter, so we’re including it as a contributed article. Although there are no APL symbols in Paul’s contribution it’s in the technical section of VECTOR because we feel that it should receive special attention from our technical readership.



COCKING & DRURY LTD.

A SINGLE SOURCE
FOR APL

▲ Consultancy & Programming

▲ APL*PLUS software

▲ Education

▲ Support

▲ Packages

▲ Turnkey Systems

Quick and Dirty Apportionment in APL

by Robert Pullman

The apportionment problem is to divide an integer n into m integer parts.

This problem occurs in several different sorts of business applications—financial planning, sales plan allocations, and top-down budgeting to name a few. Apportionment might only play a small role in the overall system, but it is likely to be used extensively in the system, and must work correctly.

This paper presents "Quick and Dirty" APL functions for apportionment. The functions are quick and dirty in that they provide a plausible solution regardless of the application.

Example 1: Straight-line calendarization.

A full-year budget of \$ n is to be evenly divided into 12 monthly amounts.

Solution:

```

      V PORTIONS←SUM STRAIGHTLINE PARTS;APPROXIMATIONS;ROUNDERS
[1]  APPROXIMATIONS←PARTS÷[SUM÷PARTS
[2]  ROUNDERS←(SUM-÷/APPROXIMATIONS)DEALBITS PARTS
[3]  PORTIONS←APPROXIMATIONS+ROUNDERS
      V

```

Will it work? SUM and PARTS are assumed to be scalar integers, with PARTS>0. The claim is that:

$SUM = \div / PORTIONS$

Informal proof:

By definition of floor:

$(SUM \div PARTS) \geq \lfloor SUM \div PARTS \rfloor$

So multiplied by PARTS:

$SUM \geq PARTS \times \lfloor SUM \div PARTS \rfloor$

Which is equivalent to:

$SUM \geq \div / APPROXIMATIONS$

So:

$(SUM - \div / APPROXIMATIONS) ? PARTS$

Will work and produce rounders such that:

$(\div / ROUNDERS) = SUM - \div / APPROXIMATIONS$

So:

$SUM = (\div / ROUNDERS) + \div / APPROXIMATIONS$
 $SUM = \div / PORTIONS$

So, with all due apologies for abuses of notation and rigor, we have shown that STRAIGHTLINE will work properly as long as its arguments suit our assumptions about them. So, of all the things that could go wrong with the development of a planning system, we could at least be sure that STRAIGHTLINE will not be among them.

Example 2: Top-down allocation

The company's objective is to sell n units of a product at its five stores. It will allocate the units according to respective percentage shares of 10 20 27 20 23.

Solution:

```

V PORTIONS+SUM PERCENTAGE PROPORTIONS;APPROXIMATIONS;ROUNDERS
[1] APPROXIMATIONS+⌊SUM×PROPORTIONS+÷/PROPORTIONS
[2] ROUNDERS+(SUM-÷/APPROXIMATIONS)DEALBITS,APPROXIMATIONS
[3] PORTIONS+APPROXIMATIONS+ROUNDERS
V

```

Will it work? SUM is again assumed to be a scalar integer, PROPORTIONS is assumed to be a vector of numbers, real or integer, with:

$0 \neq \div / \text{PROPORTIONS}$

We again claim that:

$\text{SUM} = \div / \text{PORTIONS}$

Informal proof: If:

$0 \neq \div / \text{PROPORTIONS}$

then by combination of fractions:

$1 = \div / \text{PROPORTIONS} + \div / \text{PROPORTIONS}$

So we can equivalently write SUM as:

```

SUM × 1
SUM × ÷ / PROPORTIONS + ÷ / PROPORTIONS
+ / SUM × ÷ / PROPORTIONS + ÷ / PROPORTIONS

```

By the definition of floor:

$(\div / \text{SUM} \times \text{PROPORTIONS} + \div / \text{PROPORTIONS}) \geq \div / \lfloor \text{SUM} \times \text{PROPORTIONS} + \div / \text{PROPORTIONS} \rfloor$
 $\text{SUM} \geq \div / \text{APPROXIMATIONS}$

and the rest of the proof follows the same reasoning as Example 1.

There is one condition, not uncommon, that PERCENTAGE does not meet:

$\text{PORTIONS} = 0$ wherever $\text{PROPORTIONS} = 0$

This constraint can be taken care of by:

```

V PORTIONS+SUM NONZERO PROPORTIONS
[1] PORTIONS+((PROPORTIONS≠0)\SUM PERCENTAGE(PROPORTIONS≠0)/PROPORTIONS
V

```

Example 3: Calendarization of a top-down allocation.

The company has allocated n units in two different ways:

1. A full-year allocation to 5 stores.
2. A monthly calendarization of the company's total plan.

It would like to construct a 5×12 matrix containing the calendarized plan for each store.

The row totals of the matrix must be identical to the full-year allocation to the stores, and the column totals of the matrix must be identical to the calendarization of the company's total plan.

Solution:

```

      V MATRIX←ROWS CROSS COLUMNS;I;BITS
[1]  BITS←ROWS÷0
[2]  ROWS←BITS/ROWS
[3]  MATRIX←((ρROWS),ρCOLUMNS)ρ0
[4]  I←1
[5]  LOOP:→LOOPEND IF I=ρROWS
[6]  MATRIX[I;]+ROWS[I]NONZERO COLUMNS
[7]  COLUMNS←COLUMNS-MATRIX[I;]
[8]  I←I+1
[9]  →LOOP
[10] LOOPEND:MATRIX[I;]+COLUMNS
[11] MATRIX←BITS\MATRIX
      V

```

Will it work? ROWS and COLUMNS are assumed to be vectors of non-negative integers, with:

```

      0<+/ROWS
      (+/ROWS)=+/COLUMNS

```

and we claim that:

```

      ROWS≡+/MATRIX
      COLUMNS≡+/MATRIX

```

Informal proof:

The only thing that could go wrong is that some iteration of line 7 leaves some element of COLUMNS negative. At each iteration of line 6 since:

```

      ROWS[(ρROWS)]>0

```

we have the special case of:

```

      ROWS[I] < +/COLUMNS
      SUM < +/PROPORTIONS

```

and so:

```

      (PROPORTIONS[J]+SUM) > PROPORTIONS[J]++/PROPORTIONS
      APROXIMATIONS[J] < PROPORTIONS[J]
      PORTIONS ≤ PROPORTIONS[J]

```

so no element of PORTIONS is greater than its respective PROPORTION, and so:

```
0 A,5 COLUMNS-MATRIX[I;]
```

Examples of use follow:

```

      1000 STRAIGHTLINE 12
83 84 83 83 83 84 84 83 83 84 83 83
      -50 STRAIGHTLINE 3
-16 -17 -17
      1000 PERCENTAGE 10 15 13
264 394 342
      -125 PERCENTAGE 10 15 13
-33 -50 -42
      V+10000 PERCENTAGE 10 15 13 11
      V
2040 3061 2654 2245
      W+10000 PERCENTAGE 10 10 10 12 12 13 15 16 17 17 17 20
      W
592 591 591 710 711 770 888 947 1006 1006 1005 1183
      Z+V CROSS W
      Z
120 121 120 144 145 157 182 193 205 205 206 242
181 180 181 218 218 235 271 290 309 309 308 361
158 157 157 189 188 204 235 252 267 267 265 315
133 133 133 159 160 174 200 212 225 225 226 265
      +/Z
2040 3061 2654 2245
      V=+/Z
1
      W=+/Z
1
      W CROSS V
121 182 157 132
121 180 157 133
120 181 157 133
144 218 188 160
146 218 188 159
158 236 204 172
181 272 235 200
193 289 252 213
205 308 268 225
205 307 267 227
205 308 266 226
241 362 315 265
```

Fast Fibbing

by John Sullivan

I was most amused to read the outrageous leg-pull 'Steps towards a better APL' in VECTOR Vol.2 No.2. Not all of it is humorous, however, as there is an important point hidden behind all of the coding, and I feel it is a point that ought to be brought to everybody's attention, especially the amateur programmers who use Basic.

The problem behind the program on pp 92-3 is simply stated: Write a program to print the first n terms of the Fibonacci series.

So off we go to the terminal and without more ado we write something like this:

```

      V R←FIB1 N
      [1] R←1 1
      [2] →(N>⌈R+R.+/~2+R)/⌈LC
      V

```

(which works OK unless $N < 3$ in which case you get the first three terms which is not what you asked for).

But, and it is a big but, we can do better than that. You will notice the branch arrow and the reference to $\lceil LC$, which announces to everybody in 120-point type that 'MY PROGRAM CONTAINS A LOOP!' This can be eliminated by a different approach i.e. that of analysing the problem.

The Fibonacci series is an example of the recurrence relation:

$$u_{r+1} = u_r + u_{r-1}$$

with the initial conditions $u_1 = 1, u_2 = 1$. To solve the recurrence relation we solve the auxiliary equation $x^2 = x + 1$, or $x^2 - x - 1 = 0$, which has roots $\frac{1}{2}(1 \pm \sqrt{5})$, so the general solution is:-

$$u_n = A \left(\frac{1 + \sqrt{5}}{2} \right)^n + B \left(\frac{1 - \sqrt{5}}{2} \right)^n$$

and A and B are found from the initial conditions to be $1/\sqrt{5}$ and $-1/\sqrt{5}$ respectively. Thus the formula for the nth term in the Fibonacci series is:

$$u_n = \frac{1}{\sqrt{5}} \left(\frac{1 + \sqrt{5}}{2} \right)^n - \frac{1}{\sqrt{5}} \left(\frac{1 - \sqrt{5}}{2} \right)^n$$

or in APL:

$$R \leftarrow (1R) \times ((.5 \times 1 + R) * N) - (.5 \times 1 - R + 5 * .5) * N$$

You may well ask how we get a series of integers from a formula using an irrational number ($\sqrt{5}$). That's easy – all the $\sqrt{5}$'s cancel out leaving only integer values; try it for yourself with a few simple values, say $n = 2$ or 3 and expand $(1 \pm \sqrt{5})^n$ by the binomial theorem. Don't use your calculator or you will hit APL's problem: sooner or later you will end up with a non-integer ending in .999 . . . or .000 . . . , which is caused by the way computers store decimal numbers in binary storage. This problem is easily overcome: the theory says that these numbers should be integers so we force them to be so by coding

$$R \leftarrow (.5 + (\sqrt{5}) \times ((.5 \times 1 + R) * N) - (.5 \times 1 - R + 5 * .5) * N$$

Now we can write our Fibonacci function Mark 2:

```

      V R←FIB2 N
[1]   R←10.5+(R)*((.5*1+R)*N)-(0.5*1-R+5*0.5)*N+1N
      V

```

Look! – No loops!

I executed each function a hundred times on an IBM 3084 running APL2 release 1.0 under MVS/XA and the following average timings were recorded.

N	=	75	10
FIB1		23.5 milliseconds	1.6 milliseconds
FIB2		2.6 milliseconds	0.8 milliseconds

The amazing result from this is that the application of a little mathematical theory has speeded up the execution of this trivial function by a factor of as much as 9. So the hidden point referred to in my first paragraph is, if you haven't guessed it already,

Always do the systems analysis **before** you write the program.

*Editor: The speed of John's excellent algorithm depends a lot on the APL interpreter used. The average timings for $n = 75$ and $n = 10$ running under APL*PLUS PC release 5.0 on an Olivetti M21 were 0.85 and 0.08 seconds for FIB1 and a surprising 3.32 and 0.20 seconds for FIB2. A floating point co-processor was not used.*

A Plea for Naming Conventions and Documentation Standards for APL

by Paul Chapman

APL enjoys the privilege, both in Britain and internationally, of being the focal point of a community of advocates, academics and acolytes. Most of the members of the British APL Association derive some or all of their professional income from the disciplines of APL, and we are all aware that the BAA provides, in part, an umbrella for political activities to promote understanding and use of the language.

Romilly Cocking, of Cocking and Drury Limited, made a short impromptu speech during the presentation of an APL micro at the November 1985 meeting of the BAA, and particularly addressed a major problem facing the British APL community at present. In a plea to encourage the availability of APL to schools and colleges, he pointed out that the use of APL cannot grow unless the number of capable APL programmers grows also to match the demand. He felt that APL programmers could easily price themselves out of the market.

A great deal of commercial APL programming in this country is performed by contractors, working either through agencies or specialist software houses. In fact, there seems to be a current trend among companies that employ full-time APL staff to reduce or eliminate these sections of their data processing departments.

APL and DP are traditional enemies. This is partly because of the way APL has grown over its twenty-nine year history. The adoption of APL has been a programmer-driven rather than management-driven phenomenon. APL programmers tend to be undisciplined, and this seems to be a function of the language itself rather than of the sort of people that are drawn to APL in the first place. DP departments do not like undisciplined programmers, particularly when the unfair advantages of the language they use leads them to behave like smartasses as well.

Why does APL lead to undisciplined programming? Its remarkable ease of use leads to two problems. Firstly, it is possible for an APL programmer to begin work on a system before a single page of specification has been produced. The interpretive environment of restartability provided by the active workspace, the ease of language extension, the function-base design, component file systems, and adaptability to new requirements, enable the programmer to design the system as he goes along, altering the overall approach and introducing new ideas as fast as the requirements change and develop. Secondly, it is possible to design an entire system without a comment being written, or a listing being printed. Every one of the advantages of the language listed above becomes a disadvantage in this respect.

Provided that development of a computer system is continuous, and the programmer or team remains constant, the problems are not serious. But the high turnover and relatively weak loyalties of all programmers are well-known, and APL programmers suffer more from these shortcomings because of the lack of stability of APL within DP and the large relative number of contractors. It is therefore especially important that APL systems are maintainable.

The draft international standard has been hailed, as all standards are, as a great advance in stabilising APL itself, and thus hopefully improving its image as an acceptable implementation language among DP departments. By contrast with COBOL however, which became popular primarily because of demands for a (to a certain extent) self-documenting and maintainable language system, and many other languages like PL/1, PASCAL, FOCUS, etc, APL code can be very obscure. The answer to this criticism has been that APL can be used in a highly structured manner, and that code can be as clear as a bell.

In practice, it hardly ever is, and this seems to me to be very sad.

In compiled languages, routines can be grouped to emphasise functional interdependence, and can be ordered to indicate structure and flow of control. Declarations can also be grouped to show structure, and their mere presence can give a deep insight into the purpose of the block in which they appear. Even the naive use of the letters I to N to indicate integers in FORTRAN, or the use of \$ in BASIC to indicate strings, gives a certain clarity. And the presence of the underline character for use in names makes identification of their purpose a little clearer.

APL has none of these features. The only apparent ordering of objects in a workspace is alphabetic, except local declarations in function headers, where virtually no use is made of order. With the obsolescence of groups, which in any case were difficult to maintain and used almost exclusively for)COPY, and hardly ever for the organised production of listings, there remain only artificial means of grouping objects.

Some attempts to correct the course of development of APL to include grouping and ordering of objects have been published in the various proceedings from past international APL conferences, but language development has a high inertia, and recent developments among the big guns of the APL world have shown that perhaps it is better to leave well alone than to try and put in every good, new idea that comes along.

So it is left to us, the programmers, to develop standards of our own. I hope that, like me, most programmers have some self-imposed standards, including naming conventions for functional groups, labels, function arguments and particularly global arrays, and also some commenting standards, like at least one line of general description per function, plus a description of the domain of each of the arguments and the result (and global used or affected).

The problem is that, even where such conventions are used, there is very little agreement. The delta particularly is used in as many ways as there are programmers. (My personal convention is to use it to start the names of global functions and arrays needed only for development of the workspaces, e.g. screen editors, but not in the production version.) In fact the only popular standard I know is the use of the first component of a file to contain a description, and this is often forgotten, over-brief, or months out of date.

In a field where many programmers spend a great deal of time trying to find out what the previous developers were up to, it seems vital to our own interests, and to our credibility as a group to our employers, to develop some standards. And because, as a group, we are very much united and organised, it seems that we should have an advantage over others. A language standard is barely half the answer to the establishment of APL as a competitor with other languages. We need a voluntary, and perhaps ultimately formal, national or international standard of conventions truly to exploit the claimed maintainability of APL systems.

I would not begin to promote such a standard in detail. It has taken many years of different jobs, in different environments on different hardware with different teams, even to begin to get together a set of rules which I am happy with myself. It would be chauvinistic for any of us to attempt to force our own pet ideas down each others' throats. I should think every APLer in the world has a useful and usable idea to contribute, and we must all listen more than we talk.

This task is several orders of magnitude harder than the language standard was, since thanks to Iverson and IBM in the early days (and no thanks to them now), there is very little disputable content of the language itself. By contrast, there are as many different documentation conventions as there are APL programmers who use them, and even those programmers are lazy. We can only begin by thinking how we each develop code, use comments and names, and then try to formalise that. Then throw the hundreds of ideas into a hat and pull out something which everyone will (reluctantly) accept.

I have heard that British Telecom engineers can always recognise when a phone has been tampered with, because they have their own special way of twisting wires and fastening screws. Perhaps in a few years APL programmers too will be able to examine a workspace and spot the difference between professional work and amateur meddling.

In every industry, there has to be a leader

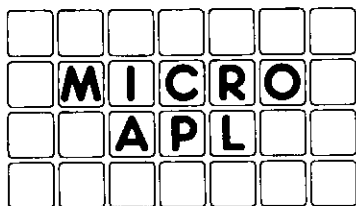
Personal APL – Our version of APL for the **Sinclair QL** brings full-powered APL at the lowest possible price.

PC-APL – MicroAPL supplies STSC's **APL*Plus/PC** with full backup, support, and ancillary software.

Departmental APL – For the ultimate in **performance and power**, our multi-user APL supermicros can handle even the largest applications.

Company-wide APL – With unrivalled experience in networking and communications, MicroAPL can offer a **comprehensive APL facility** linking PCs, supermicros and mainframes.

Consultancy in APL – MicroAPL's commitment to quality doesn't stop at our hardware products. Our uniquely experienced team of APL consultants will undertake any APL software project, from one day's help with a specific problem, to major team developments.



MicroAPL Limited

Unit 1F, Nine Elms Industrial Estate, 87 Kirtling Street, London SW8 5BP.
Telephone: 01-622 0395

The Use of `□WIN` in APL*PLUS PC Release 5

by David Piper

Abstract

One of the problems of using APL*PLUS PC in the past has been the relative difficulty of accessing the screen via some form of "full-screen" mode. Release 5.0 includes two new system functions which enable this form of application to be created much more easily – `□WIN` and its associated function `□WKEY`.

Using these functions, an easy to use, full-screen interface can be constructed without the need to resort to packages such as Toolkit 1 or Power Tools. A full screen interface which closely emulates the full-screen interface supplied by IBM in the FSC12x workspaces can also be implemented very easily. This will allow systems implemented on VS APL to be transported to the PC much more easily.

Introducing `□WIN`

The `□WIN` system function emulates the facilities provided by the `SFOR`, `SCUR` and `SREA` functions in the FSC12X workspaces supplied by IBM for fullscreen processing in VS APL. That is, it establishes the fields on the screen, positions the cursor in a specific field and then waits for data to be typed and an interrupt made (ENTER or PF-Key).

The only remaining functions to be implemented are those which write and read data to and from fields on the screen – `SWRI` and `<SGET>`. These can be emulated using the `□WPUT` and `□WGET` functions implemented in earlier releases of APL*PLUS PC.

The Format Array.

The screen layout is defined in terms of the "format array", which is passed as the right argument to the `□WIN` function. The format array is similar to that used in mainframe applications, but some differences are apparent.

COLUMN	Description
[1 2]	Field start position, row and column. The top right of the screen is position 0,0 on the PC, but position 1,1 on the mainframe.
[3 4]	Field width and depth in rows and columns. Identical to mainframe implementation.
[5]	Field type – 0=Protected (mf=2) 1=Alphameric (mf=0) 2=Numeric (mf=1) Additional field types can be user defined, to only accept specific characters from the keyboard, e.g. to set up a field which will only accept the Y or N keys.

- [6] Special feature code: 1=Autoskip when field is full (ASFEND under GDDM). Other features are available, such as automatic numeric validation etc.
- [7 8] Attributes to be used with/without the cursor. The PC attribute combines colour, intensity and extended attributes in one value. It is therefore more difficult to manipulate than the separate values used on the mainframe. Additionally different attribute values are required for colour or monochrome screens.

Since the attributes on the PC are more difficult to handle than on the mainframe, and it is desirable to be able to convert from a mainframe type format to a PC format, a function <fCNVT> has been written to perform the conversion. This function takes a mainframe format array as its right argument and returns a PC format array which will emulate the mainframe functionality as closely as possible. The function is capable of handling all the GDDM extended attributes (colour, reverse video etc).

The <fCNVT> function can also take advantage of column seven of the PC format array. If a left argument is supplied, it is assumed to be an array of attributes to be adopted when the cursor enters the field. The columns must be supplied in the same order as in the mainframe format array (intensity, colour, extended attribute). The ability to change attributes depending on the presence or otherwise of the cursor makes it very easy to implement 1-2-3 or Symphony type menus. The attribute in column 8 (without cursor) displays the field normally, that in column 7 puts it into reverse video when the cursor enters the field.

Positioning the Cursor

The position of the cursor is initialised by the left argument of ☐WIN. This argument is a numeric vector of up to 4 elements, the first three define the current cursor position in terms of a field number, and the row and column within that field. The field number is the row index of the desired field in the format array, and is therefore ☐IO dependent.

The fourth element is a time-out factor – if no keys are pressed before the time-out is exceeded, control will be returned to the application. If the time-out is omitted, an infinite value is assumed.

Detecting an interrupt

Once ☐WIN has been called, control is normally only returned to the user's application when a key defined for that purpose is pressed. By default, the ESCAPE and CTRL-ENTER keys are defined. The meaning of keys on the keyboard can be changed by using the ☐WKEY function (described over page).

The result of `□WIN` includes the following information:

ELEMENT	Description
[1]	Reason for exiting <code>□WIN</code> , normally zero if a valid exit key is used.
[2]	Numeric value of the key causing the exit.
[3]	Number of field containing the cursor (in origin-zero).
[4 5]	Row and column in field of cursor position (origin 0).
[6...]	Boolean elements indicating if the equivalent field in the right argument was modified or not.

If the special features (such as numeric validation) are specified in column 6 of the format array, then other reason codes may be set in the first element of the return vector.

Emulating Mainframe Screen Processing

One of the major features of screen processing via GDDM is that all I/O to the screen is buffered by GDDM until a request to update the screen is received (ASREAD or FSRCE). Since, on the PC, no shared variables are involved, I/O must either occur immediately, or must be buffered by being stored in an APL variable. The second approach allows the panel to be placed on the screen using just one call to `□WPUT`, which is very quick and efficient.

Similarly, by establishing attributes on the physical screen as late as possible, multiple uses of `□WIN` can be avoided. This is achieved by storing the requested format array in a global variable. Any changes to attributes can be effected simply by changing the relevant element of the global format array.

This process is made even easier if the format array is kept in mainframe format until it is output to the screen. The `<SINT>` function listed below gives an example of this. The format array is only processed through the `<fCNVT>` function immediately before use by `□WIN`.

Reading the Screen

When a screen read is requested, two calls to `□WIN` are required. The first is made to establish attributes for all fields on the screen. This call cannot be used for the actual screen read, since the cursor can be moved to any field on the screen, even to protected fields. To avoid a delay while the attributes are established, a zero time-out is requested. After this, `□WPUT` is used to write the text of the panel image to the screen.

The second call to `□WIN` is then made, this time only INPUT fields are passed in the argument. Since only these fields are defined, the cursor cannot be moved to any protected part of the screen. Once control is returned to the application, the updated panel is retrieved using `□WGET`.

In order to retrieve the data in the input fields, the information is read from the global panel image variable. This is illustrated by the <SGET> function listed below.

Using Function Keys

□WIN cannot detect the keyboard codes associated with the PF keys. In order to allow the PF keys to be detected, an alternative key code must be assigned using □PFKEY. This alternative key code is then detected by □WIN and can be interpreted by the application as a PF key. The best key codes to use appear to be 480-510.

In order to detect these keys, □WKEY must be used in order to change the interpretation of the key. □WKEY allows the interpretation of any key on the keyboard to be changed, either to an alternative character or some other special meaning such as TAB. The <initialise> function listed below sets up some required global variables and changes the codes associated with the function keys. These new codes are then reinterpreted by □WKEY to mean "return control to the application". In addition the CAPS-LOCK key is given the same meaning, since CAPS-LOCK is in a similar position to ENTER on the IBM 327X keyboard.

Conclusions

The presence of □WIN in APL*PLUS PC release 5.0 makes the implementation of full-screen processing on the PC very simple. This style of processing can be made as close to the mainframe style as is desired, even to the extent of redefining the keyboard.

The format conversion function, in conjunction with a suitable set of cover functions on the PC, means that systems can be transferred directly from mainframe implementations of APL to the PC with very little conversion work being involved.

The ambivalent implementation of the format conversion function also allows full use to be made of the PC's extended features, such as changing field characteristics as the cursor enters the field.

```

V OF+AA fCNVT FF;AT;BO
[1]  A Convert Mainframe format to PC format array
[2]  A FF - Numeric array      MF style format array
[3]  A AA - Numeric array      Alternate attributes with cursor
[4]  A OF - Numeric array      Equivalent PC stype format array
[5]  A Make sure input array has 10 columns
[6]  FF+((1+pFF),10)+FF
[7]  A If AA not specified, default is FF[; 6 7 9]
[8]  +(2=□NC 'AA')/dy
[9]  AA+FF[; 6 7 9]
[10] A If AA has missing columns, replace from FF
[11] dy:AA+AA,((1+pAA),(~1+pAA)-3)+FF[; 6 7 9]
[12] A Adjust start position to origin zero.
[13] OF+{FF[; 1 2]-1}, 0 ~5 + 0 2 +FF
[14] A Add in 'feature' col
[15] OF+OF,~FF[;10]

```

```

[16] A Change field type value.
[17] OF[;5]+ 1 2 0 [3[1+FF[;5]]
[18] A Check for colour/mono screen
[19] →(3=[CRT[7]])/co
[20] A Add in hi-lighting/intensity options
[21] OF+OF,(ΔMONO AA[; 1 3]),ΔMONO FF[; 6 9]
[22] →0
[23] A Add in hi-lighting/colour/intensity options
[24] co:OF+OF,(ΔCOLOUR FF[;5],AA),ΔCOLOUR FF[; 5 6 7 9]

```

▽

```

▽ FN SINT IN;AT;AV;BO
[1] A Set intensity of fields
[2] A IN - Numeric vector      Intensities to be set
[3] A FN - Char/num vector    Field names/numbers
[4] A Convert to field numbers
[5] FN+FIELDS FN
[6] A Reassign intensities.
[7] format[FN;6]+(ρFN)+IN

```

▽

```

▽ DA+SGET FL;CO;CL;RW
[1] A Get data in screen fields <FL>
[2] A FL - Num/Char array      fields to be read(names or numbers)
[3] A DA - Character array     data entered in fields
[4] DA+' '
[5] A Convert to field numbers
[6] →(0=ρFL+FIELDS FL)/0
[7] CO+1
[8] A Calculate Row and Column indexes
[9] 10:RW+(-1+format[FL[CO];1])+format[FL[CO];3]
[10] CL+(-1+format[FL[CO];2])+format[FL[CO];4]
[11] A Retrieve text
[12] DA+DA,TCNL,screen[RW;CL]
[13] →((ρFL)≥CO+CO+1)/10
[14] A Convert to array
[15] DA+TCNL box 1+DA

```

▽

```

▽ initialise;CO;RC
[1] A Initialise globals and keyboard
[2] A Set up status line window.
[3] RC+ 1 0 0 0 [WIN fCNVT 1 9 p 25 1 1 80 2 1 0 0 2
[4] A Output processing message.
[5] sSTATUS 'PROCESSING'
[6] A Initialise <alarm>.
[7] →(2=[NC 'alarm']/cp
[8] alarm+0
[9] A Turn APL keyboard off.
[10] cp:[KEYW+1
[11] A Initialise pf keys to return codes 480-510.
[12] ck:→(Λ/1=[WKEY 428,479+130])/0
[13] CO+1
[14] 10:(479+CO)[PFKEY CO
[15] →(30≥CO+CO+1)/10
[16] A CAPS LOCK and PF Keys return to application.
[17] RC+1 [WKEY 428,479+130

```

▽

APL Trivia: The functions they couldn't print

by Mark Bassett

Following your call for more trivial APL functions (so much more interesting than the useful sort) here are a pair of self reproducing functions; one reproduces its $\square$ CR form, the other its $\square$ VR.

```

      V r←CRSELFREP
[1]  r← 2 89 ρ(38ϕ127+r).1ϕ⊖AV[⊖IO+2ρ39].r←'r← 2 89 ρ(38ϕ127+r).1ϕ⊖AV[
      ⊖IO+2ρ39].r←r+CRSELFREP'
      V

```

```

      V r←VRSELFREP
[1]  r←7ϕ⊖TCNL,⊖AV[⊖IO+ 32 32 32 32 236],⊖TCNL,(~17ϕ⊖TCNL,r).1ϕ⊖AV[⊖IO+
      2ρ39].r←'[1]  r←7ϕ⊖TCNL,⊖AV[⊖IO+ 32 32 32 32 236],⊖TCNL,(~17ϕ⊖TCNL,r),
      1ϕ⊖AV[⊖IO+2ρ39].r←'      V r←VRSELFREP'
      V

```

Both these functions use the same trick, first formulated by the logician E.Quine. If we have a function QUINE with the property that QUINE x is $x'x'$, then we can create a self-reproducing expression by writing QUINE'QUINE'.

```

      V r←QUINE x
[1]  r←x,1ϕ⊖AV[⊖IO+2ρ39].x
      V
( ⊖AV[⊖IO+39] is the character ' ' )

```

Try evaluating the expression:

QUINE'Applying QUINE to the following vector creates a false statement'.

Do you believe what it says or not?

Although this may seem a rather fanciful function it is not without its serious side; it has profound applications in both logic and computer science. Godel relied upon a version of it to prove the incompleteness of formal mathematics, and see how it fits into the following rather curious result.

LOOPY is a function of one argument, x , that returns 1 if executing x would cause an infinite loop, and 0 otherwise.

```

      V r←LOOPY x
[1]  A
[2]  A (left as an exercise for the astute reader)
[3]  A
      V

```

Obviously LOOPY can't actually execute *x* itself, because if it did go into an infinite loop there would be no way it could return a result to say so; instead it must perform a clever syntactic analysis, coupled with the execution of carefully selected bits of code (looking for lines that perform an unconditional branch to themselves would probably be a good place to start).

But now consider the following function, named after its discoverer, Alan Turing:

```

      V TURING x
[1]   →(r←LOOPY QUINE x)ρ0
[2]   L:→L
      V

```

What is the value of TURING'TURING'? In particular, does it go into an infinite loop or not?

The only solution to the above dilemma is to conclude that the function LOOPY is impossible, it just can't be written to behave as described. But if there's no algorithm we can express in APL that will let a computer tell if a function goes into an infinite loop or not, how can we, as mere humans, do it? Presumably we can't, which makes debugging programs a rather fraught exercise. By building on Turing's work logicians were able to show that for every function specification we can realise in APL (or any other language), there are literally infinitely many we can't – which makes writing programs sound like an even more futile activity than debugging them.

All this is beginning to sound dangerously non-trivial, so I shall move on from functions that can't be written to functions that never should have been. Those of you with access to STSC APL*PLUS/PC (I haven't tried this on other systems) might like to try the following little function, coyly named BUG.

```

      V BUG;f
[1]   f← 2 4 ρ'r←L9','r←L8'
[2]   □WA←□FX f
[3]   →L9
[4]   L8:□+ 'LINE EIGHT'
[5]   →0
[6]   L9:□+ 'LINE NINE'
[7]   →0
      V

```

As Gilman and Rose explain on page 150, labels are like local variables but you can't assign new values to them; as you can see however it is possible to □FX a function of the same name, and this can return any value you like! I showed this to Martyn Adams who came up with the bright idea of expunging the newly created function, this 'works' as you would expect too (imagine the effect of a virus function that spent the morning infecting other pieces of code and then during lunch re-assigned or removed all their line labels!).

```

      V BUGTWO;f
[1]  f← 1 2 ρ'L9'
[2]  □WA+□FX f
[3]  □WA+□EX 'L9'
[4]  →L9
[5]  L8:□+'LINE EIGHT'
[6]  →0
[7]  L9:□+'LINE NINE'
[8]  →0
      V

```

Now that the line label had thoroughly disappeared I tried re-introducing it by direct assignment, i.e.

```

      V BUGTHREE;f
[1]  f← 1 2 ρ'L9'
[2]  □WA+□FX f
[3]  □WA+□EX 'L9'
[4]  L9+L8
[5]  →L9
[6]  L8:□+'LINE EIGHT'
[7]  →0
[8]  L9:□+'LINE NINE'
[9]  →0
      V

```

Oddly, at this point the interpreter 'remembered' that L9 was a line label and came back at me with a domain error. It proved a lot more obliging, however, if the branch and the assignment took place on the same line.

```

      V BUGFOUR;f
[1]  f← 1 2 ρ'L9'
[2]  □WA+□FX f
[3]  □WA+□EX 'L9'
[4]  →L9+L8
[5]  L8:□+'LINE EIGHT'
[6]  →0
[7]  L9:□+'LINE NINE'
[8]  →0
      V

```

Even more oddly, I discovered that this trick failed when the following line contained a simple branch, even though it was never called.

```

      V BUGFIVE;f
[1]  f← 1 2 ρ'L9'
[2]  □WA+□FX f
[3]  □WA+□EX 'L9'
[4]  →L9+L8
[5]  →L9
[6]  L8:□+'LINE EIGHT'
[7]  →0
[8]  L9:□+'LINE NINE'
[9]  →0
      V

```

Before you all go rushing off to incorporate these tricks into your latest devilishly cunning lines of code, I should point out that these are real bugs, not features. Every time I have run these functions I have had some sort of system crash a few moments later, ranging from the spectacular vision of all the characters rushing to the centre of the screen and vanishing, while the disk drive lights glowed fiercely (a software black hole ?) to the quiet but resolute announcement that my attempt to issue)SAVE was a domain error!

P.S. Would it be possible to suggest that the trivia section of VECTOR be renamed 'Jota-zero' ? (Because it's a vector with trivial content. Oh well!)

BACK NUMBERS OF VECTOR

Back numbers of VECTOR are available from the BCS. If you don't have them all, now is the time to complete your collection. Apart from the technical contents, every issue includes book and product reviews, letters, news and a competition.. Send in your order before they run out. These will one day be unobtainable collectors' items, like the early issues of Quote Quad.

The prices inclusive of postage and packing are as follows:

	UK	Prices in Pounds Sterling	
		Surface (inc. Europe)	Airmail (outside Europe)
Single issues	3	3.75	5.75
Volume 1	10	14.00	22.00
Volume 2	10	14.00	22.00

Please send sterling cheques or money orders (payable to The British APL Association) to the Treasurer:

Mel Chapman, 12 Garden Street, Stafford ST17 4BT.

Don't forget to include your name and address and to be clear which VECTORS you want.

VECTOR – contents of back-numbers

Title	Author(s)	Vol	No	Page
Meetings & Conferences:				
APL and graphics (1984)	Adrian Smith	1	1	35
A graphic vision of APL	Graeme Robertson	1	1	36
APL and graphics (1985)	Adrian Smith	1	4	63
APL and GKS	Richard Nabavi	1	4	63
Duck a la carte	Peter Donnelly	1	4	65
Getting the best out of GDDM	Adrian Smith	1	4	65
APL & Knowledge-based systems	Adrian Smith	1	3	48
Expert Systems	M.J. Winfield	1	3	49
APL in the Information Centre	Adrian Smith	1	1	41
Setting up a company I.C.	Romilly Cocking	1	1	42
The Global Information Centre	Fred Perkins	1	1	45
APL in practice	Bern King	2	3	60
Sales Forecasting System	Patrick Rushton	2	3	60
Large Commercial Database	Christine Brewster	2	3	61
APL Statistics Users Group	Jake Ansell	2	3	63
APL systems on micros	Eileen Dyson	2	3	50
Developments in PEFAC	Chris Staffurth	2	3	51
Corporate Planning	Adam Dakin	2	3	54
Integrated Manufacturing	Ed Shaw	2	3	56
Migration from mainframe	John Birch	2	3	57
APL vs packages & 4GLs	Adrian Smith	1	2	48
Are 4GLs killing APL?	Desmond Booth	1	2	49
APL84 in Focus	Smith, Bowman & Lyus	1	2	52
An APL compiler	Roy Sykes	1	2	54
Dyalog APL	John Scholes	1	2	54
Ampere: a portable APL m/c	Philip van Cleave	1	2	55
APL2	Jim Brown	1	2	56
The APL Machine	Jim Ryan	1	2	57
APL85				
A brief impression	Dick Bowman	2	1	64
Report from APL85	John Adams	2	1	67
Second thoughts on Seattle	Dick Gray	2	1	68
Photographic review	David Ziemann	2	1	70
Communicating with APL	Adrian Smith	1	1	48
Networking APL micros	Richard Nabavi	1	1	49
Interfacing Viewdata & APL	John Pym	1	1	59
IBM-based APL communications	Tim Perry	1	1	62
Computer Graphics '85	Williamson & Hollamby	2	4	69
Expert Systems '85	Peter Davies	2	4	77
IPSA Users' Meeting 1984	Adrian Smith	1	3	65
Mainframes, micros & co-existence	Adrian Smith	2	1	59
My first date with IRMA	Graham Fieldsend	2	1	60
Nested Arrays Workshop	Adrian Smith	2	1	46
Importance of being nested	David Crossley	2	1	48
O. R. Conference 85	Adrian Smith	2	3	65
Surviving in APL/XX		1	3	60
What's New for 1985	Dick Bowman	1	3	63

General Articles:

An APL for the teacher	Dick Gray	2	1	79
APL and graphics standards	Dick Bowman	1	2	75
APLIM - Generalized Linear Models	Jake Ansell	2	2	63
APL lap-sized computer	Phil van Cleave	2	2	55
Do you dig keywords?	Dick Gray	2	2	61
External Databases	Williamson & Wells	2	3	79
FGL: Fifth Generation Language	Robert Bittlestone	1	1	89
Financial Planning at Imperial	Steve Lyus	2	4	93
GKS - opportunity for APL	Richard Nabavi	1	2	78
Graphics for decision-makers	David Preedy	1	2	83
Information Centre as strategy	Fred Perkins	1	4	90
Menu-oriented dialogue	Gosta Olavi	1	3	83
Mr Babbage's Secret	Ole Franksen	1	2	61
QL/APL - popularising APL	David Eastwood	1	4	81
Recursion revisited	David Preedy	2	2	47
Re-inventing Man	Robert Bittlestone	1	4	75
Steps towards a better BASIC	Anthony Camacho	1	1	77
(part 2)		1	2	73
(part 3)		1	4	95
(part 4)		2	1	81
(part 5)		2	3	89
The paper they dared not print	Anon	1	1	87
Towards a teachable interface	Adrian Smith	1	3	91
Why APL?	Robert Bittlestone	1	1	65
Why GKS is unsuitable . .	Martyn Adams	2	3	83
XPL - an expert systems framework	Robert Bittlestone	1	2	65

Case-studies:

Bedford School	Adrian Smith	2	1	84
Imperial Group graphics	Adrian Smith	1	2	89
Matchmakers simulation	Adrian Smith	1	1	79
PATTIE: a practical expert system	Christopher Harvey	1	3	101
(part 2)		1	4	98
VSPC: for whom the bell tolls?	Adrian Smith	2	2	69
Whitbread: large scale database	Christine Brewster	2	4	81

Technical Articles:

An APL Dialogue	Graeme Robertson	1	3	135
APL and partitioned data	Jonathan Barman	1	1	129
APL Linguistics	Graeme Robertson	2	2	118
APLpip	L. Llewellyn-Jones	1	2	109
APL2 or LISP	Keppel & Kropp	2	2	97
Complex numbers in APL	Alan Hawkes	1	1	107
DIF-file interface	Romilly Cocking	1	4	149
GDDM and AP126	David Piper	2	3	109
Generic read & replace in IPSA	Robert Pullman	1	3	128
Guide to APL2 nested arrays	Norman Thompson	2	1	108
International APL standard	David Ziemann	1	1	143
ISO Standards update	David Ziemann	1	2	125

Migration to APL2	Mark Lenihan	2	4	123
Modelling fuzzy decisions	Adrian Smith	2	2	109
Moving data from 1-2-3 to APL	Michael Carmichael	1	4	130
Numeric integration & probability	Alan Hawkes	1	2	117
Operators & Nested Arrays	John Scholes	2	1	117
Power to APL	Hultin & Hagger	2	1	123
QL/APL - a leap forward or back?	van Batenburg & Prins	2	2	115
Quotidian Jottings	Micheal Carmichael	1	3	156
Random Contingency Tables	A.G.Prys Williams	1	3	160
SCREENIO: IBM full-screen manager	David Doherty	1	1	121
Statistical computing with APL	Alan Hawkes	1	2	113
Using operators in APL2	Norman Thompson	2	3	118
Using VSAPL under TSO	David Piper	2	4	127
When domino is not sufficient	Sykes & Ansell	1	4	155

Reviews:

APAT - APL utilities	Bernard King	1	2	44
"APL - an interactive approach"	David Preedy	1	2	41
APL on micros	Adrian Smith	1	4	54
"Application systems in APL"	Anthony Camacho	2	2	41
"Artificial Intelligence"	Robert Bittlestone	1	3	43
"Build your own Expert Systems"	Robert Bittlestone	1	3	41
"Decision Support Systems"	David Preedy	2	2	42
"Diagnosing the system"	David Preedy	2	3	33
"Expert Systems: Concepts & examples"	Robert Bittlestone	1	3	42
"Fuzzy Sets & Decision Analysis"	Robert Bittlestone	1	4	51
"Graphics . . in Office Systems"	David Preedy	2	3	36
H-P 7550 plotter	David Preedy	2	1	40
Keyword APL/QL	David Ziemann	1	4	121
LaserJet printer	David Preedy	2	1	42
"Learning & applying APL"	Anthony Camacho	1	3	39
"LISP: a gentle introduction . ."	Robert Bittlestone	1	4	51
MicroSpan		2	4	41
"Mr. Babbage's Secret"	Ole Franksen	1	2	42
"Powers of TEN"	Robert Bittlestone	1	4	53
U-REG - regression package	Teresa Jones	1	3	45
"Visual display . ."	David Preedy	2	3	34

Index to Advertisers

Ampere	26
APL-385	12
APL People	78
APL Software Technology	20
Cocking & Drury	2, 38, 108
Dyadic Systems Ltd	22, 98
HMW Programming Consultants	6, 58
I.P.S.A.	96
IBM (Sweden)	16
Inner Product Ltd	66
Interprocess Systems	92
MetaTechnics	49, 104
MicroAPL	50, 118
Mine of Information (APL Booklist)	37
Vector - Back Numbers	127

All queries regarding advertising in VECTOR should be made to the advertising editor, Steve Lyus, at the following address:

2 Howard Road
Westbury Park
Bristol
BS6 7UT

Tel: 0272-40716

Advertisements should be submitted in typeset, camera-ready A5 portrait format with a 20mm blank border. Illustrations should be black-and-white photographs or line drawings. Rates are £250 per page. A6 and A7 sizes (at £150 and £75 respectively) are available, subject to layout constraints.

BRITISH APL ASSOCIATION**Membership Application Form**

Please read the membership information in the inside front cover of VECTOR before completing this form. Use photocopies of this form for multiple applications. The membership year runs from 1st May – 30th April.

Name: _____
 Department: _____
 Organisation: _____
 Address Line 1: _____
 Address Line 2: _____
 Address Line 3: _____
 Address Line 4: _____
 Post or zip code: _____
 Country: _____
 Telephone Number: _____

Membership category applied for (tick one):	86/87	
Non-voting student membership (UK only)	£ 5	
UK private membership	£ 10	
Overseas private membership	£ 18	\$ 27
Airmail supplement (not needed for Europe)	£ 8	\$ 12
Corporate membership	£ 85	
Corporate membership Overseas	£140	\$210
Sustaining membership	£360	

For student applicants:

Name of course: _____
 Name and title of supervisor: _____
 Signature of supervisor: _____

PAYMENT

Payment should be enclosed with membership applications in the form of a UK sterling cheque or postal order made payable to "The British APL Association". Corporate or sustaining member applicants should contact the Treasurer in advance if an invoice is required. Please enclose a stamped addressed envelope if you require a receipt.

Send the completed form to the Treasurer at this address:

Mel Chapman, 12 Garden Street, Stafford ST17 4BT, UK.

The British APL Association

The British APL Association is a Specialist Group of the British Computer Society and a member of EuroAPL, an organisation supported by the Commission of the European Communities. It is administered by a Committee of eight officers who are elected by the vote of Association members at the Annual General Meeting. Working groups are also established in areas such as activity planning and journal production. Offers of assistance and involvement with any Association matters are welcomed and should be addressed in the first instance to the Secretary.

1986/87 Committee

See AGM report for details of the 1986/87 committee

Activities Working Group

Peter Donnelly	0420-87024
Steve Margolis	01-670 7959
Tim Perry	04626-77375
Roy Tallis	01-405 7841
Stan Wilkinson	01-286 7068

Journal Working Group

Jonathan Barman	01-493 6172
Anthony Camacho	0727(56 from London)-60130
Steve Lyus	0272-40716
David Preedy	01-541 1696
Adrian Smith	0904-53071
David Ziemann	01-493 6172

WP disks converted to photsetting by Capella House, Stowmarket (0449) 677663.

Printed in England by Short-Run Press Ltd., Exeter.

VECTOR

VECTOR is the quarterly Journal of the British APL Association and is distributed to Association members in the UK and overseas. The British APL Association is a Specialist Group of the British Computer Society and a member of EuroAPL, an organisation supported by the Commission of the European Communities. APL stands for "A Programming Language" - an interactive computer programming language noted for its elegance, conciseness and fast development speed. It is supported on many timesharing bureaux and on most mainframe, mini and micro computers.

SUSTAINING MEMBERS

The Committee of the British APL Association wish to acknowledge the generous financial support of the following Association Sustaining Members. In many cases these organisations also provide manpower and administrative assistance to the Association at their own cost.

APL People

17 Barton Street
Bath, Avon
Tel: 0225 62601

APL Software Technology

14 Rosewood Avenue
Avaston, Bristol, BS12 2PP
Tel: 0454 415737

Cocking & Drury Ltd.

16 Berkeley Street, London W1X 5AE
Tel: 01-493 6172

Dyadic Systems Ltd.

Park House, The High Street, Alton, Hants.
Tel: 0420 67024

Inner Product Ltd.

Eagle House, 73 Clapham Common Southside,
London SW4 9DG
Tel: 01-676 8854

MetaTechnics Systems Ltd.

Unit 216, 62 Triton Road,
London SE21 8DE
Tel: 01-670 7959

MicroAPL Limited

Unit 1F, Nine Elms Industrial Estate,
87 Kirling Street, London SW8 5BP
Tel: 01-622 0395

HP Sharp Associates

10 Dean Farrar Street,
London SW1H 0DX
Tel: 01-222 7033



The British Computer Society, 13 Mansfield Street, London W1M 0BD.